1

Preliminaries

1.1 Reasons for Studying Concepts of Programming Languages
1.2 Programming Domains
1.3 Language Evaluation Criteria
1.4 Influences on Language Design
1.5 Language Categories
1.6 Language Design Trade-Offs
1.7 Implementation Methods
1.8 Programming Environments

Before we begin discussing the concepts of programming languages, we must consider a few preliminaries. First, we explain some reasons why computer science students and professional software developers should study general approaches to language design and evaluation. This discussion is especially valuable for those who believe that a working knowledge of one or two programming languages is sufficient for computer scientists. Then, we briefly describe the major programming domains. Next, because the book evaluates language constructs and features, we present a list of criteria that can serve as a basis for such judgments. Then, we discuss the two major influences on language design: machine architecture and program design methodologies. After that, we introduce the major categories of programming languages. Next, we describe a few of the most important trade-offs that must be considered during language design.

Because this book is also about the implementation of programming languages, this chapter includes an overview of the most common general approaches to implementation. Finally, we briefly describe a few examples of programming environments and discuss their impact on software production.

1.1 Reasons for Studying Concepts of Programming Languages

It is natural for students to wonder how they will benefit from the study of programming language concepts. After all, many other topics in computer science are worthy of serious study. In fact, many now believe that there are more important areas of computing for study than can be covered in a four-year college curriculum. The following is what we believe to be a compelling list of potential benefits of studying concepts of programming languages:

- *Increased capacity to express ideas.* It is widely believed that the depth at which people can think is influenced by the expressive power of the language in which they communicate their thoughts. Those with only a weak understanding of natural language are limited in the complexity of their thoughts, particularly in depth of abstraction. In other words, it is difficult for people to conceptualize structures they cannot describe, verbally or in writing.

  Programmers, in the process of developing software, are similarly constrained. The language in which they develop software places limits on the kinds of control structures, data structures, and abstractions they can use; thus, the forms of algorithms they can construct are likewise limited. Awareness of a wider variety of programming language features can reduce such limitations in software development. Programmers can increase the range of their software development thought processes by learning new language constructs.

  It might be argued that learning the capabilities of other languages does not help a programmer who is forced to use a language that lacks those capabilities. That argument does not hold up, however, because often, language constructs can be simulated in other languages that do not support those constructs directly. For example, a C (Harbison and Steele, 2002) programmer who had learned the structure and uses of associative

arrays in Perl (Christianson et al., 2013) might design structures that simulate associative arrays in that language. In other words, the study of programming language concepts builds an appreciation for valuable language features and constructs and encourages programmers to use them, even when the language they are using does not directly support such features and constructs.

- *Improved background for choosing appropriate languages.* Some professional programmers have had little formal education in computer science; rather, they have developed their programming skills independently or through in-house training programs. Such training programs often limit instruction to one or two languages that are directly relevant to the current projects of the organization. Other programmers received their formal training years ago. The languages they learned then are no longer widely used, and many features now available in programming languages were not commonly known at the time. The result is that many programmers, when given a choice of languages for a new project, use the language with which they are most familiar, even if it is poorly suited for the project at hand. If these programmers were familiar with a wider range of languages and language constructs, they would be better able to choose the language with the features that best address the problem.

  Some of the features of one language often can be simulated in another language. However, it is preferable to use a feature whose design has been integrated into a language than to use a simulation of that feature, which is often less elegant, more cumbersome, and less safe.

- *Increased ability to learn new languages.* Computer programming is still a relatively young discipline, and design methodologies, software development tools, and programming languages are still in a state of continuous evolution. This makes software development an exciting profession, but it also means that continuous learning is essential. The process of learning a new programming language can be lengthy and difficult, especially for someone who is comfortable with only one or two languages and has never examined programming language concepts in general. Once a thorough understanding of the fundamental concepts of languages is acquired, it becomes far easier to see how these concepts are incorporated into the design of the language being learned. For example, programmers who understand the concepts of object-oriented programming will have a much easier time learning Ruby (Thomas et al., 2013) than those who have never used those concepts.

  The same phenomenon occurs in natural languages. The better you know the grammar of your native language, the easier it is to learn a second language. Furthermore, learning a second language has the benefit of teaching you more about your first language.

  The TIOBE Programming Community issues an index (`http://www.tiobe.com/index.php/content/paperinfo/tpci/index.htm`) that is an indicator of the relative popularity of programming languages. For example, according to the index, Java, C, C++ (Lippman et al., 2012), and

C# (Albahari and Abrahari, 2012) were the four most popular languages in use in February 2017.[1] However, dozens of other languages were widely used at the time. The index data also show that the distribution of usage of programming languages is always changing. The number of languages in use and the dynamic nature of the statistics imply that every software developer must be prepared to learn different languages.

Finally, it is essential that practicing programmers know the vocabulary and fundamental concepts of programming languages so they can read and understand programming language descriptions and evaluations, as well as promotional literature for languages and compilers. These are the sources of information needed in order to choose and learn a language.

- *Better understanding of the significance of implementation.* In learning the concepts of programming languages, it is both interesting and necessary to touch on the implementation issues that affect those concepts. In some cases, an understanding of implementation issues leads to an understanding of why languages are designed the way they are. In turn, this knowledge leads to the ability to use a language more intelligently, as it was designed to be used. We can become better programmers by understanding the choices among programming language constructs and the consequences of those choices.

  Certain kinds of program bugs can be found and fixed only by a programmer who knows some related implementation details. Another benefit of understanding implementation issues is that it allows us to visualize how a computer executes various language constructs. In some cases, some knowledge of implementation issues provides hints about the relative efficiency of alternative constructs that may be chosen for a program. For example, programmers who know little about the complexity of the implementation of subprogram calls often do not realize that a small subprogram that is frequently called can be a highly inefficient design choice.

  Because this book touches on only a few of the issues of implementation, the previous two paragraphs also serve well as rationale for studying compiler design.
- *Better use of languages that are already known.* Most contemporary programming languages are large and complex. Accordingly, it is uncommon for a programmer to be familiar with and use all of the features of a language he or she uses. By studying the concepts of programming languages, programmers can learn about previously unknown and unused parts of the languages they already use and begin to use those features.
- *Overall advancement of computing.* Finally, there is a global view of computing that can justify the study of programming language concepts. Although it is usually possible to determine why a particular programming language became popular, many believe, at least in retrospect, that the most popular

1. Note that this index is only one measure of the popularity of programming languages, and its accuracy is not universally accepted.

languages are not always the best available. In some cases, it might be concluded that a language became widely used, at least in part, because those in positions to choose languages were not sufficiently familiar with programming language concepts.

For example, many people believe it would have been better if ALGOL 60 (Backus et al., 1963) had displaced Fortran (ISO/IEC 1539-1, 2010) in the early 1960s, because it was more elegant and had much better control statements, among other reasons. That it did not, is due partly to the programmers and software development managers of that time, many of whom did not clearly understand the conceptual design of ALGOL 60. They found its description difficult to read (which it was) and even more difficult to understand. They did not appreciate the benefits of block structure, recursion, and well-structured control statements, so they failed to see the benefits of ALGOL 60 over Fortran.

Of course, many other factors contributed to the lack of acceptance of ALGOL 60, as we will see in Chapter 2. However, the fact that computer users were generally unaware of the benefits of the language played a significant role.

In general, if those who choose languages were well informed, perhaps better languages would eventually squeeze out poorer ones.

1.2 Programming Domains

Computers have been applied to a myriad of different areas, from controlling nuclear power plants to providing video games in mobile phones. Because of this great diversity in computer use, programming languages with very different goals have been developed. In this section, we briefly discuss a few of the most common areas of computer applications and their associated languages.

1.2.1 Scientific Applications

The first digital computers, which appeared in the late 1940s and early 1950s, were invented and used for scientific applications. Typically, the scientific applications of that time used relatively simple data structures, but required large numbers of floating-point arithmetic computations. The most common data structures were arrays and matrices; the most common control structures were counting loops and selections. The early high-level programming languages invented for scientific applications were designed to provide for those needs. Their competition was assembly language, so efficiency was a primary concern. The first language for scientific applications was Fortran. ALGOL 60 and most of its descendants were also intended to be used in this area, although they were designed to be used in related areas as well. For some scientific applications where efficiency is the primary concern, such as those that were common in the 1950s and 1960s, no subsequent language is significantly better than Fortran, which explains why Fortran is still used.

1.2.2 Business Applications

The use of computers for business applications began in the 1950s. Special computers were developed for this purpose, along with special languages. The first successful high-level language for business was COBOL (ISO/IEC, 2002), the initial version of which appeared in 1960. It probably still is the most commonly used language for these applications. Business languages are characterized by facilities for producing elaborate reports, precise ways of describing and storing decimal numbers and character data, and the ability to specify decimal arithmetic operations.

There have been few developments in business application languages outside the development and evolution of COBOL. Therefore, this book includes only limited discussions of the structures in COBOL.

1.2.3 Artificial Intelligence

Artificial intelligence (AI) is a broad area of computer applications characterized by the use of symbolic rather than numeric computations. Symbolic computation means that symbols, consisting of names rather than numbers, are manipulated. Also, symbolic computation is more conveniently done with linked lists of data rather than arrays. This kind of programming sometimes requires more flexibility than other programming domains. For example, in some AI applications the ability to create and execute code segments during execution is convenient.

The first widely used programming language developed for AI applications was the functional language Lisp (McCarthy et al., 1965), which appeared in 1959. Most AI applications developed prior to 1990 were written in Lisp or one of its close relatives. During the early 1970s, however, an alternative approach to some of these applications appeared—logic programming using the Prolog (Clocksin and Mellish, 2013) language. More recently, some AI applications have been written in systems languages such as Python (Lutz, 2013). Scheme (Dybvig, 2011), a dialect of Lisp, and Prolog are introduced in Chapters 15 and 16, respectively.

1.2.4 Web Software

The World Wide Web is supported by an eclectic collection of languages, ranging from markup languages, such as HTML, which is not a programming language, to general-purpose programming languages, such as Java. Because of the pervasive need for dynamic Web content, some computation capability is often included in the technology of content presentation. This functionality can be provided by embedding programming code in an HTML document. Such code is often in the form of a scripting language, such as JavaScript (Flanagan, 2011) or PHP (Tatroe et al., 2013). There are also some markup-like languages that have been extended to include constructs that control document processing, which are discussed in Section 1.5 and in Chapter 2.

1.3 Language Evaluation Criteria

As noted previously, the purpose of this book is to examine carefully the underlying concepts of the various constructs and capabilities of programming languages. We will also evaluate these features, focusing on their impact on the software development process, including maintenance. To do this, we need a set of evaluation criteria. Such a list of criteria is necessarily controversial, because it is difficult to get even two computer scientists to agree on the value of some given language characteristic relative to others. In spite of these differences, most would agree that the criteria discussed in the following subsections are important.

Some of the characteristics that influence three of the four most important of these criteria are shown in Table 1.1, and the criteria themselves are discussed in the following sections.[2] Note that only the most important characteristics are included in the table, mirroring the discussion in the following subsections. One could probably make the case that if one considered less important characteristics, virtually all table positions could include "bullets."

Note that some of these characteristics are broad and somewhat vague, such as writability, whereas others are specific language constructs, such as exception handling. Furthermore, although the discussion might seem to imply that the criteria have equal importance, that implication is not intended, and it is clearly not the case.

1.3.1 Readability

One of the most important criteria for judging a programming language is the ease with which programs can be read and understood. Before 1970, software development was largely thought of in terms of writing code. The primary

Table 1.1 Language evaluation criteria and the characteristics that affect them

	CRITERIA		
Characteristic	READABILITY	WRITABILITY	RELIABILITY
Simplicity	•	•	•
Orthogonality	•	•	•
Data types	•	•	•
Syntax design	•	•	•
Support for abstraction		•	•
Expressivity		•	•
Type checking			•
Exception handling			•
Restricted aliasing			•

2. The fourth primary criterion is cost, which is not included in the table because it is only slightly related to the other criteria and the characteristics that influence them.

positive characteristic of programming languages was efficiency. Language constructs were designed more from the point of view of the computer than of the computer users. In the 1970s, however, the software life-cycle concept (Booch, 1987) was developed; coding was relegated to a much smaller role, and maintenance was recognized as a major part of the cycle, particularly in terms of cost. Because ease of maintenance is determined in large part by the readability of programs, readability became an important measure of the quality of programs and programming languages. This was an important juncture in the evolution of programming languages. There was a distinct crossover from a focus on machine orientation to a focus on human orientation.

Readability must be considered in the context of the problem domain. For example, if a program that describes a computation is written in a language not designed for such use, the program may be unnatural and convoluted, making it unusually difficult to read.

The following subsections describe characteristics that contribute to the readability of a programming language.

1.3.1.1 Overall Simplicity

The overall simplicity of a programming language strongly affects its readability. A language with a large number of basic constructs is more difficult to learn than one with a smaller number. Programmers who must use a large language often learn a subset of the language and ignore its other features. This learning pattern is sometimes used to excuse the large number of language constructs, but that argument is not valid. Readability problems occur whenever the program's author has learned a different subset from that subset with which the reader is familiar.

A second complicating characteristic of a programming language is **feature multiplicity**—that is, having more than one way to accomplish a particular operation. For example, in Java, a user can increment a simple integer variable in four different ways:

```
count = count + 1
count += 1
count++
++count
```

Although the last two statements have slightly different meanings from each other and from the others in some contexts, all of them have the same meaning when used as stand-alone expressions. These variations are discussed in Chapter 7.

A third potential problem is **operator overloading**, in which a single operator symbol has more than one meaning. Although this is often useful, it can lead to reduced readability if users are allowed to create their own overloading and do not do it sensibly. For example, it is clearly acceptable to overload + to use it for both integer and floating-point addition. In fact, this overloading

simplifies a language by reducing the number of operators. However, suppose the programmer defined + used between single-dimensioned array operands to mean the sum of all elements of both arrays. Because the usual meaning of vector addition is quite different from this, this unusual meaning could confuse both the author and the program's readers. An even more extreme example of program confusion would be a user defining + between two vector operands to mean the difference between their respective first elements. Operator overloading is further discussed in Chapter 7.

Simplicity in languages can, of course, be carried too far. For example, the form and meaning of most assembly language statements are models of simplicity, as you can see when you consider the statements that appear in the next section. This very simplicity, however, makes assembly language programs less readable. Because they lack more complex control statements, program structure is less obvious; because the statements are simple, far more of them are required than in equivalent programs in a high-level language. These same arguments apply to the less extreme case of high-level languages with inadequate control and data-structuring constructs.

1.3.1.2 Orthogonality

Orthogonality in a programming language means that a relatively small set of primitive constructs can be combined in a relatively small number of ways to build the control and data structures of the language. Furthermore, every possible combination of primitives is legal and meaningful. For example, consider data types. Suppose a language has four primitive data types (integer, float, double, and character) and two type operators (array and pointer). If the two type operators can be applied to themselves and the four primitive data types, a large number of data structures can be defined.

The meaning of an orthogonal language feature is independent of the context of its appearance in a program. (The word *orthogonal* comes from the mathematical concept of orthogonal vectors, which are independent of each other.) Orthogonality follows from a symmetry of relationships among primitives. A lack of orthogonality leads to exceptions to the rules of the language. For example, in a programming language that supports pointers, it should be possible to define a pointer to point to any specific type defined in the language. However, if pointers are not allowed to point to arrays, many potentially useful user-defined data structures cannot be defined.

We can illustrate the use of orthogonality as a design concept by comparing one aspect of the assembly languages of the IBM mainframe computers and the VAX series of minicomputers. We consider a single simple situation: adding two 32-bit integer values that reside in either memory or registers and replacing one of the two values with the sum. The IBM mainframes have two instructions for this purpose, which have the forms

```
A Reg1, memory_cell
AR Reg1, Reg2
```

where `Reg1` and `Reg2` represent registers. The semantics of these are

```
Reg1 ← contents(Reg1) + contents(memory_cell)
Reg1 ← contents(Reg1) + contents(Reg2)
```

The VAX addition instruction for 32-bit integer values is

```
ADDL operand_1, operand_2
```

whose semantics is

```
operand_2 ← contents(operand_1) + contents(operand_2)
```

In this case, either operand can be a register or a memory cell.

The VAX instruction design is orthogonal in that a single instruction can use either registers or memory cells as its operands. There are two ways to specify operands, which can be combined in all possible ways. The IBM design is not orthogonal. Only two out of four operand combinations possibilities are legal, and the two require different instructions, `A` and `AR`. The IBM design is more restricted and therefore less writable. For example, you cannot add two values and store the sum in a memory location. Furthermore, the IBM design is more difficult to learn because of the restrictions and the additional instruction.

Orthogonality is closely related to simplicity: The more orthogonal the design of a language, the fewer exceptions the language rules require. Fewer exceptions mean a higher degree of regularity in the design, which makes the language easier to learn, read, and understand. Anyone who has learned a significant part of the English language can testify to the difficulty of learning its many rule exceptions (for example, *i* before *e* except after *c*).

As examples of the lack of orthogonality in a high-level language, consider the following rules and exceptions in C. Although C has two kinds of structured data types, arrays and records (**`struct`**s), records can be returned from functions but arrays cannot. A member of a structure can be any data type except **`void`** or a structure of the same type. An array element can be any data type except **`void`** or a function. Parameters are passed by value, unless they are arrays, in which case they are, in effect, passed by reference (because the appearance of an array name without a subscript in a C program is interpreted to be the address of the array's first element).

As an example of context dependence, consider the C expression

```
a + b
```

This expression often means that the values of `a` and `b` are fetched and added together. However, if `a` happens to be a pointer and `b` is an integer, it affects the value of `b`. For example, if `a` points to a float value that occupies four bytes, then the value of `b` must be scaled—in this case multiplied by 4—before it is

added to a. Therefore, the type of a affects the treatment of the value of b. The context of b affects its meaning.

Too much orthogonality can also cause problems. Perhaps the most orthogonal programming language is ALGOL 68 (van Wijngaarden et al., 1969). Every language construct in ALGOL 68 has a type, and there are no restrictions on those types. In addition, most constructs produce values. This combinational freedom allows extremely complex constructs. For example, a conditional can appear as the left side of an assignment, along with declarations and other assorted statements, as long as the result is an address. This extreme form of orthogonality leads to unnecessary complexity. Furthermore, because languages require a large number of primitives, a high degree of orthogonality results in an explosion of combinations. So, even if the combinations are simple, their sheer numbers lead to complexity.

Simplicity in a language, therefore, is at least in part the result of a combination of a relatively small number of primitive constructs and a limited use of the concept of orthogonality.

Some believe that functional languages offer a good combination of simplicity and orthogonality. A functional language, such as Lisp, is one in which computations are made primarily by applying functions to given parameters. In contrast, in imperative languages such as C, C++, and Java, computations are usually specified with variables and assignment statements. Functional languages offer potentially the greatest overall simplicity, because they can accomplish everything with a single construct, the function call, which can be combined simply with other function calls. This simple elegance is the reason why some language researchers are attracted to functional languages as the primary alternative to complex nonfunctional languages such as Java. Other factors, the most important of which is probably efficiency, however, have prevented functional languages from becoming more widely used.

1.3.1.3 Data Types

The presence of adequate facilities for defining data types and data structures in a language is another significant aid to readability. For example, suppose a numeric type is used for an indicator flag because there is no Boolean type in the language. In such a language, for example, in the original version of C, we might have an assignment such as the following:

```
timeout = 1
```

The meaning of this statement is unclear, whereas in a language that includes Boolean types, we would have the following:

```
timeout = true
```

The meaning of this statement is perfectly clear.

1.3.1.4 Syntax Design

The syntax, or form, of the elements of a language has a significant effect on the readability of programs. Following are some examples of syntactic design choices that affect readability:

- *Special words.* Program appearance and thus program readability are strongly influenced by the forms of a language's special words (for example, **`while`**, **`class`**, and **`for`**). Especially important is the method of forming compound statements, or statement groups, primarily in control constructs. Some languages have used matching pairs of special words or symbols to form groups. C and its descendants use braces to specify compound statements. All of these languages have diminished readability because statement groups are always terminated in the same way, which makes it difficult to determine which group is being ended when an **`end`** or a right brace appears. Fortran 95 and Ada (ISO/IEC, 2014) make this clearer by using a distinct closing syntax for each type of statement group. For example, Ada uses **`end if`** to terminate a selection construct and **`end loop`** to terminate a loop construct. This is an example of the conflict between simplicity that results in fewer reserved words, as in Java, and the greater readability that can result from using more reserved words, as in Ada.

  Another important issue is whether the special words of a language can be used as names for program variables. If so, then the resulting programs can be very confusing. For example, in Fortran 95, special words, such as `Do` and `End`, are legal variable names, so the appearance of these words in a program may or may not connote something special.

- *Form and meaning.* Designing statements so that their appearance at least partially indicates their purpose is an obvious aid to readability. Semantics, or meaning, should follow directly from syntax, or form. In some cases, this principle is violated by two language constructs that are identical or similar in appearance but have different meanings, depending perhaps on context. In C, for example, the meaning of the reserved word **`static`** depends on the context of its appearance. If used on the definition of a variable inside a function, it means the variable is created at compile time. If used on the definition of a variable that is outside all functions, then it means the variable is visible only in the file in which its definition appears; that is, it is not exported from that file.

  One of the primary complaints about the shell commands of UNIX (Robbins, 2005) is that their appearance does no t always suggest their function. For example, the meaning of the UNIX command `grep` can be deciphered only through prior knowledge, or perhaps cleverness and familiarity with the UNIX editor, `ed`. The appearance of `grep` connotes nothing to UNIX beginners. (In `ed`, the command */regular_expression/* searches for a substring that matches the regular expression. Preceding this with `g` makes it a global command, specifying that the scope of the search is the whole file being edited. Following the command with `p` specifies that lines with the matching substring are to be printed. So `g`*/regular_expression/*`p`, which can obviously be abbreviated as `grep`, prints all lines in a file that contain substrings that match its operand, which is a regular expression.)

1.3.2 Writability

Writability is a measure of how easily a language can be used to create programs for a chosen problem domain. Most of the language characteristics that affect readability also affect writability. This follows directly from the fact that the process of writing a program requires the programmer frequently to reread the part of the program that is already written.

As is the case with readability, writability must be considered in the context of the target problem domain of a language. It simply is not fair to compare the writability of two languages in the realm of a particular application when one was designed for that application and the other was not. For example, the writabilities of Visual BASIC (VB) (Halvorson, 2013) and C are dramatically different for creating a program that has a graphical user interface (GUI), for which VB is ideal. Their writabilities are also quite different for writing systems programs, such as an operating system, for which C was designed.

The following subsections describe the most important characteristics influencing the writability of a language.

1.3.2.1 Simplicity and Orthogonality

If a language has a large number of different constructs, some programmers who use the language might not be familiar with all of them. This situation can lead to a misuse of some features and a disuse of others that may be either more elegant or more efficient, or both, than those that are used. It may even be possible, as noted by Hoare (1973), to use unknown features accidentally, with bizarre results. Therefore, a smaller number of primitive constructs and a consistent set of rules for combining them (that is, orthogonality) is much better than simply having a large number of primitives. A programmer can design a solution to a complex problem after learning only a simple set of primitive constructs.

On the other hand, too much orthogonality can be a detriment to writability. Errors in programs can go undetected when nearly any combination of primitives is legal. This can lead to code absurdities that cannot be discovered by the compiler.

1.3.2.2 Expressivity

Expressivity in a language can refer to several different characteristics. In a language such as APL (Gilman and Rose, 1983), it means that there are very powerful operators that allow a great deal of computation to be accomplished with a very small program. More commonly, it means that a language has relatively convenient, rather than cumbersome, ways of specifying computations. For example, in C, the notation `count++` is more convenient and shorter than `count = count + 1`. Also, the **`and then`** Boolean operator in Ada is a convenient way of specifying short-circuit evaluation of a Boolean expression. The inclusion of the **`for`** statement in Java makes writing counting loops easier than with the use of **`while`**, which is also possible. All of these increase the writability of a language.

1.3.3 Reliability

A program is said to be reliable if it performs to its specifications under all conditions. The following subsections describe several language features that have a significant effect on the reliability of programs in a given language.

1.3.3.1 Type Checking

Type checking is simply testing for type errors in a given program, either by the compiler or during program execution. Type checking is an important factor in language reliability. Because run-time type checking is expensive, compile-time type checking is more desirable. Furthermore, the earlier errors in programs are detected, the less expensive it is to make the required repairs. The design of Java requires checks of the types of nearly all variables and expressions at compile time. This virtually eliminates type errors at run time in Java programs. Types and type checking are discussed in depth in Chapter 6.

One example of how failure to type check, at either compile time or run time, has led to countless program errors is the use of subprogram parameters in the original C language (Kernighan and Ritchie, 1978). In this language, the type of an actual parameter in a function call was not checked to determine whether its type matched that of the corresponding formal parameter in the function. An `int` type variable could be used as an actual parameter in a call to a function that expected a `float` type as its formal parameter, and neither the compiler nor the run-time system would detect the inconsistency. For example, because the bit string that represents the integer 23 is essentially unrelated to the bit string that represents a floating-point 23, if an integer 23 is sent to a function that expects a floating-point parameter, any uses of the parameter in the function will produce nonsense. Furthermore, such problems are often difficult to diagnose.[3] The current version of C has eliminated this problem by requiring all parameters to be type checked. Subprograms and parameter-passing techniques are discussed in Chapter 9.

1.3.3.2 Exception Handling

The ability of a program to intercept run-time errors (as well as other unusual conditions detectable by the program), take corrective measures, and then continue is an obvious aid to reliability. This language facility is called **exception handling**. Ada, C++, Java, and C# include extensive capabilities for exception handling, but such facilities are practically nonexistent in some widely used languages, for example, C. Exception handling is discussed in Chapter 14.

1.3.3.3 Aliasing

Loosely defined, **aliasing** is having two or more distinct names in a program that can be used to access the same memory cell. It is now generally accepted that aliasing is a dangerous feature in a programming language. Most

3. In response to this and other similar problems, UNIX systems include a utility program named `lint` that checks C programs for such problems.

programming languages allow some kind of aliasing—for example, two pointers (or references) set to point to the same variable, which is possible in most languages. In such a program, the programmer must always remember that changing the value pointed to by one of the two changes the value referenced by the other. Some kinds of aliasing, as described in Chapters 5 and 9, can be prohibited by the design of a language.

In some languages, aliasing is used to overcome deficiencies in the language's data abstraction facilities. Other languages greatly restrict aliasing to increase their reliability.

1.3.3.4 Readability and Writability

Both readability and writability influence reliability. A program written in a language that does not support natural ways to express the required algorithms will necessarily use unnatural approaches. Unnatural approaches are less likely to be correct for all possible situations. The easier a program is to write, the more likely it is to be correct.

Readability affects reliability in both the writing and maintenance phases of the life cycle. Programs that are difficult to read are difficult both to write and to modify.

1.3.4 Cost

The total cost of a programming language is a function of many of its characteristics.

First, there is the cost of training programmers to use the language, which is a function of the simplicity and orthogonality of the language and the experience of the programmers. Although more powerful languages are not necessarily more difficult to learn, they often are.

Second, there is the cost of writing programs in the language. This is a function of the writability of the language, which depends in part on its closeness in purpose to the particular application. The original efforts to design and implement high-level languages were driven by the desire to lower the costs of creating software.

Both the cost of training programmers and the cost of writing programs in a language can be significantly reduced in a good programming environment. Programming environments are discussed in Section 1.8.

Third, the cost of executing programs written in a language is greatly influenced by that language's design. A language that requires many run-time type checks will prohibit fast code execution, regardless of the quality of the compiler. Although execution efficiency was the foremost concern in the design of early languages, it is now considered to be less important.

A simple trade-off can be made between compilation cost and execution speed of the compiled code. **Optimization** is the name given to the collection of techniques that compilers may use to decrease the size and/or increase the execution speed of the code they produce. If little or no optimization is done, compilation can be done much faster than if a significant effort is made to

produce optimized code. The choice between the two alternatives is influenced by the environment in which the compiler will be used. In a laboratory for beginning programming students, who often compile their programs many times during development but use little execution time (their programs are small and they must execute correctly only once), little or no optimization should be done. In a production environment, where compiled programs are executed many times after development, it is better to pay the extra cost to optimize the code.

Fourth, there is the cost of poor reliability. If the software fails in a critical system, such as a nuclear power plant or an X-ray machine for medical use, the cost could be very high. The failures of noncritical systems can also be very expensive in terms of lost future business or lawsuits over defective software systems.

The final consideration is the cost of maintaining programs, which includes both corrections and modifications to add new functionality. The cost of software maintenance depends on a number of language characteristics, primarily readability. Because maintenance is often done by individuals other than the original author of the software, poor readability can make the task extremely challenging.

The importance of software maintainability cannot be overstated. It has been estimated that for large software systems with relatively long lifetimes, maintenance costs can be as high as two to four times as much as development costs (Sommerville, 2010).

Of all the contributors to language costs, three are most important: program development, maintenance, and reliability. Because these are functions of writability and readability, these two evaluation criteria are, in turn, the most important.

Of course, a number of other criteria could be used for evaluating programming languages. One example is **portability**, or the ease with which programs can be moved from one implementation to another. Portability is most strongly influenced by the degree of standardization of the language. Some languages are not standardized at all, making programs in these languages very difficult to move from one implementation to another. This problem is alleviated in some cases by the fact that implementations for some languages now have single sources. Standardization is a time-consuming and difficult process. A committee began work on producing a standard version of C++ in 1989. It was approved in 1998.

Generality (the applicability to a wide range of applications) and **well-definedness** (the completeness and precision of the language's official defining document) are two other criteria.

Most criteria, particularly readability, writability, and reliability, are neither precisely defined nor accurately measurable. Nevertheless, they are useful concepts and they provide valuable insight into the design and evaluation of programming languages.

A final note on evaluation criteria: language design criteria are weighed differently from different perspectives. Language implementors are concerned

primarily with the difficulty of implementing the constructs and features of the language. Language users are worried about writability first and readability later. Language designers are likely to emphasize elegance and the ability to attract widespread use. These characteristics often conflict with one another.

1.4 Influences on Language Design

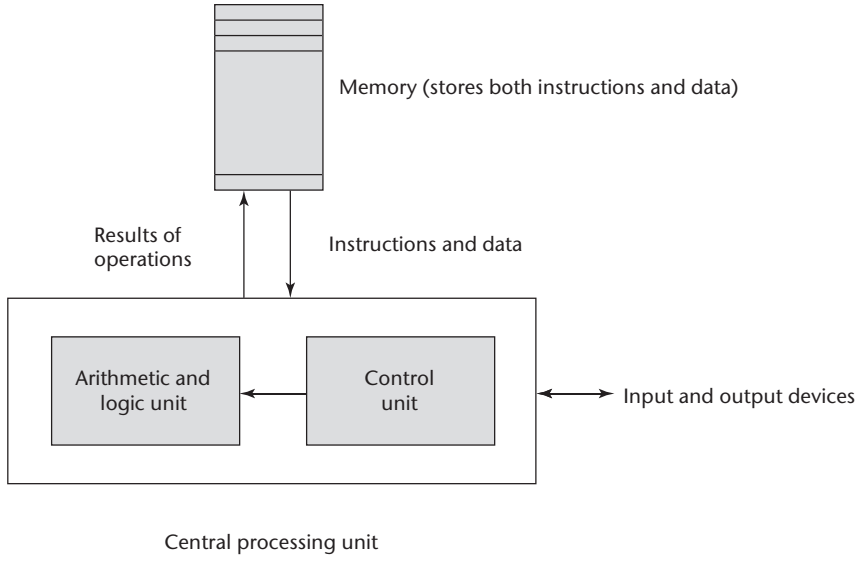
In addition to those factors described in Section 1.3, several other factors influence the basic design of programming languages. The most important of these are computer architecture and programming design methodologies.

1.4.1 Computer Architecture

The basic architecture of computers has had a profound effect on language design. Most of the popular languages of the past 60 years have been designed around the prevalent computer architecture, called the **von Neumann architecture**, after one of its originators, John von Neumann (pronounced "von Noyman"). These languages are called **imperative** languages. In a von Neumann computer, both data and programs are stored in the same memory. The central processing unit (CPU), which executes instructions, is separate from the memory. Therefore, instructions and data must be transmitted, or piped, from memory to the CPU. Results of operations in the CPU must be moved back to memory. Nearly all digital computers built since the 1940s have been based on the von Neumann architecture. The overall structure of a von Neumann computer is shown in Figure 1.1.

Figure 1.1

The von Neumann computer architecture



Because of the von Neumann architecture, the central features of imperative languages are variables, which model the memory cells; assignment statements, which are based on the piping operation; and the iterative form of repetition, which is the most efficient way to implement repetition on this architecture. Operands in expressions are piped from memory to the CPU, and the result of evaluating the expression is piped back to the memory cell represented by the left side of the assignment. Iteration is fast on von Neumann computers because instructions are stored in adjacent cells of memory and repeating the execution of a section of code requires only a branch instruction. This efficiency discourages the use of recursion for repetition, although recursion is sometimes more natural.

The execution of a machine code program on a von Neumann architecture computer occurs in a process called the **fetch-execute cycle**. As stated earlier, programs reside in memory but are executed in the CPU. Each instruction to be executed must be moved from memory to the processor. The address of the next instruction to be executed is maintained in a register called the **program counter**. The fetch-execute cycle can be simply described by the following algorithm:

```
initialize the program counter
repeat forever
    fetch the instruction pointed to by the program counter
    increment the program counter to point at the next instruction
    decode the instruction
    execute the instruction
end repeat
```

The "decode the instruction" step in the algorithm means the instruction is examined to determine what action it specifies. Program execution terminates when a stop instruction is encountered, although on an actual computer a stop instruction is rarely executed. Rather, control transfers from the operating system to a user program for its execution and then back to the operating system when the user program execution is complete. In a computer system in which more than one user program may be in memory at a given time, this process is far more complex.

As stated earlier, a functional, or applicative, language is one in which the primary means of computation is applying functions to given parameters. Programming can be done in a functional language without the kind of variables that are used in imperative languages, without assignment statements, and without iteration. Although many computer scientists have expounded on the myriad benefits of functional languages, such as Scheme, it is unlikely that they will displace the imperative languages until a non–von Neumann computer is designed that allows efficient execution of programs in functional languages. Among those who have bemoaned this fact, perhaps the most eloquent was John Backus (1978), the principal designer of the original version of Fortran.

In spite of the fact that the structure of imperative programming languages is modeled on a machine architecture, rather than on the abilities and inclinations of the users of programming languages, some believe that using imperative languages is somehow more natural than using a functional language. So, these people believe that even if functional programs were as efficient as imperative programs, the use of imperative programming languages would still dominate.

1.4.2 Programming Design Methodologies

The late 1960s and early 1970s brought an intense analysis, begun in large part by the structured-programming movement, of both the software development process and programming language design.

An important reason for this research was the shift in the major cost of computing from hardware to software, as hardware costs decreased and programmer costs increased. Increases in programmer productivity were relatively small. In addition, progressively larger and more complex problems were being solved by computers. Rather than simply solving sets of equations to simulate satellite tracks, as in the early 1960s, programs were being written for large and complex tasks, such as controlling large petroleum-refining facilities and providing worldwide airline reservation systems.

The new software development methodologies that emerged as a result of the research of the 1970s were called top-down design and stepwise refinement. The primary programming language deficiencies that were discovered were incompleteness of type checking and inadequacy of control statements (requiring the extensive use of gotos).

In the late 1970s, a shift from procedure-oriented to data-oriented program design methodologies began. Simply put, data-oriented methods emphasize data design, focusing on the use of abstract data types to solve problems.

For data abstraction to be used effectively in software system design, it must be supported by the languages used for implementation. The first language to provide even limited support for data abstraction was SIMULA 67 (Birtwistle et al., 1973), although that language certainly was not propelled to popularity because of it. The benefits of data abstraction were not widely recognized until the early 1970s. However, most languages designed since the late 1970s support data abstraction, which is discussed in detail in Chapter 11.

The latest step in the evolution of data-oriented software development, which began in the early 1980s, is object-oriented design. Object-oriented methodology begins with data abstraction, which encapsulates processing with data objects and controls access to data, and adds inheritance and dynamic method binding. Inheritance is a powerful concept that greatly enhances the potential reuse of existing software, thereby providing the possibility of significant increases in software development productivity. This is an important factor in the increase in popularity of object-oriented languages. Dynamic (run-time) method binding allows more flexible use of inheritance.

Object-oriented programming developed along with a language that supported its concepts: Smalltalk (Goldberg and Robson, 1989). Although Smalltalk never became as widely used as many other languages, support for object-oriented programming is now part of most popular imperative languages, including Java, C++, and C#. Object-oriented concepts have also found their way into functional programming in CLOS (Bobrow et al., 1988) and F# (Syme et al., 2010), as well as logic programming in Prolog++ (Moss, 1994). Language support for object-oriented programming is discussed in detail in Chapter 12.

Procedure-oriented programming is, in a sense, the opposite of data-oriented programming. Although data-oriented methods now dominate software development, procedure-oriented methods have not been abandoned. On the contrary, in recent years, a good deal of research has occurred in procedure-oriented programming, especially in the area of concurrency. These research efforts brought with them the need for language facilities for creating and controlling concurrent program units. Java and C# include such capabilities. Concurrency is discussed in detail in Chapter 13.

All of these evolutionary steps in software development methodologies led to new language constructs to support them.

1.5 Language Categories

Programming languages are often categorized into four bins: imperative, functional, logic, and object oriented. However, we do not consider languages that support object-oriented programming to form a separate category of languages. We have described how the most popular languages that support object-oriented programming grew out of imperative languages. Although the object-oriented software development paradigm differs significantly from the procedure-oriented paradigm usually used with imperative languages, the extensions to an imperative language required to support object-oriented programming are not intensive. For example, the expressions, assignment statements, and control statements of C and Java are nearly identical. (On the other hand, the arrays, subprograms, and semantics of Java are very different from those of C.) Similar statements can be made for functional languages that support object-oriented programming.

Some authors refer to scripting languages as a separate category of programming languages. However, languages in this category are bound together more by their implementation method, partial or full interpretation, than by a common language design. The languages that are typically called scripting languages, among them Perl, JavaScript, and Ruby (Flanagan and Matsumoto, 2008), are imperative languages in every sense.

A logic programming language is an example of a rule-based language. In an imperative language, an algorithm is specified in great detail, and the specific order of execution of the instructions or statements must be included. In a rule-based language, however, rules are specified in no particular order, and the language implementation system must choose an order in which the

rules are used to produce the desired result. This approach to software development is radically different from those used with the other two categories of languages and clearly requires a completely different kind of language. Prolog, the most commonly used logic programming language, and logic programming are discussed in Chapter 16.

In recent years, a new category of languages has emerged, the markup/programming hybrid languages. Markup languages are not programming languages. For instance, HTML, the most widely used markup language, is used to specify the layout of information in Web documents. However, some programming capability has crept into some extensions to HTML and XML. Among these are the Java Server Pages Standard Tag Library (JSTL) and eXtensible Stylesheet Language Transformations (XSLT). Both of these are briefly introduced in Chapter 2. Those languages cannot be compared to any of the complete programming languages and therefore will not be discussed after Chapter 2.

1.6 Language Design Trade-Offs

The programming language evaluation criteria described in Section 1.3 provide a framework for language design. Unfortunately, that framework is self-contradictory. In his insightful paper on language design, Hoare (1973) stated that "there are so many important but conflicting criteria, that their reconciliation and satisfaction is a major engineering task."

Two criteria that conflict are reliability and cost of execution. For example, the Java language definition demands that all references to array elements be checked to ensure that the index or indices are in their legal ranges. This step adds a great deal to the cost of execution of Java programs that contain large numbers of references to array elements. C does not require index range checking, so C programs execute faster than semantically equivalent Java programs, although Java programs are more reliable. The designers of Java traded execution efficiency for reliability.

As another example of conflicting criteria that leads directly to design trade-offs, consider the case of APL. APL includes a powerful set of operators for array operands. Because of the large number of operators, a significant number of new symbols had to be included in APL to represent the operators. Also, many APL operators can be used in a single, long, complex expression. One result of this high degree of expressivity is that, for applications involving many array operations, APL is very writable. Indeed, a huge amount of computation can be specified in a very small program. Another result is that APL programs have very poor readability. A compact and concise expression has a certain mathematical beauty but it is difficult for anyone other than the programmer to understand. Well-known author Daniel McCracken (1970) once noted that it took him four hours to read and understand a four-line APL program. The designer of APL traded readability for writability.

The conflict between writability and reliability is a common one in language design. The pointers of C++ can be manipulated in a variety of ways, which supports highly flexible addressing of data. Because of the potential reliability problems with pointers, they are not included in Java.

Examples of conflicts among language design (and evaluation) criteria abound; some are subtle, others are obvious. It is therefore clear that the task of choosing constructs and features when designing a programming language requires many compromises and trade-offs.

1.7 Implementation Methods

As described in Section 1.4.1, two of the primary components of a computer are its internal memory and its processor. The internal memory is used to store programs and data. The processor is a collection of circuits that provides a realization of a set of primitive operations, or machine instructions, such as those for arithmetic and logic operations. In most computers, some of these instructions, which are sometimes called macroinstructions, are actually implemented with a set of instructions called microinstructions, which are defined at an even lower level. Because microinstructions are never seen by software, they will not be discussed further here.

The machine language of the computer is its set of instructions. In the absence of other supporting software, its own machine language is the only language that most hardware computers "understand." Theoretically, a computer could be designed and built with a particular high-level language as its machine language, but it would be very complex and expensive. Furthermore, it would be highly inflexible, because it would be difficult (but not impossible) to use it with other high-level languages. The more practical machine design choice implements in hardware a very low-level language that provides the most commonly needed primitive operations and requires system software to create an interface to programs in higher-level languages.

A language implementation system cannot be the only software on a computer. Also required is a large collection of programs, called the operating system, which supplies higher-level primitives than those of the machine language. These primitives provide system resource management, input and output operations, a file management system, text and/or program editors, and a variety of other commonly needed functions. Because language implementation systems need many of the operating system facilities, they interface with the operating system rather than directly with the processor (in machine language).

The operating system and language implementations are layered over the machine language interface of a computer. These layers can be thought of as virtual computers, providing interfaces to the user at higher levels. For example, an operating system and a C compiler provide a virtual C computer. With other compilers, a machine can become other kinds of virtual computers. Most computer systems provide several different virtual computers. User programs form another layer over the top of the layer of virtual computers. The layered view of a computer is shown in Figure 1.2.

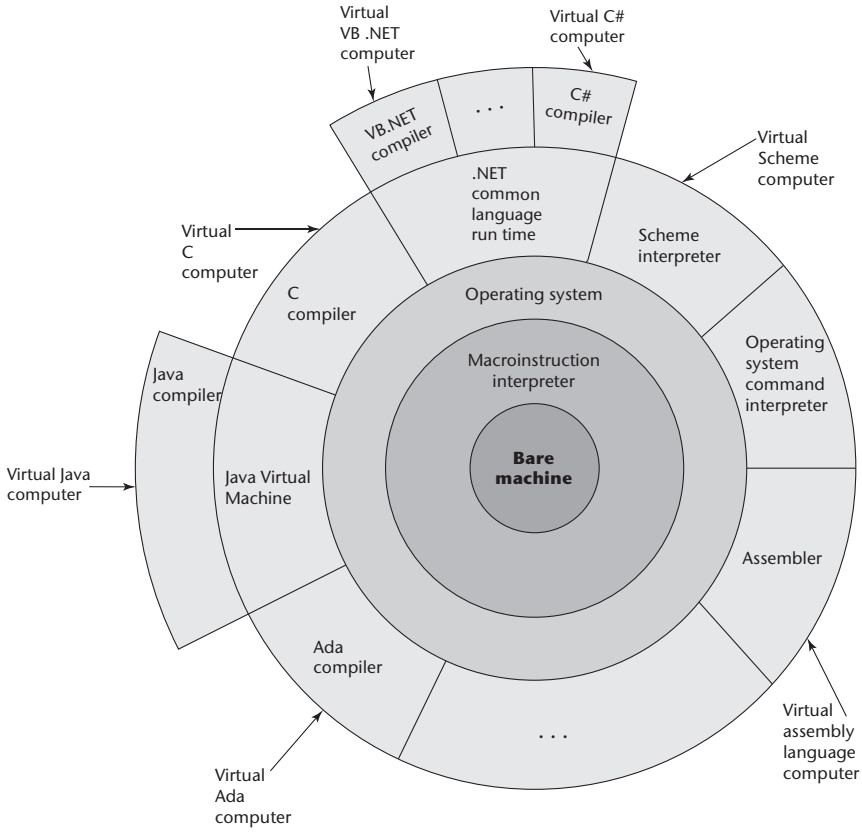


Figure 1.2

Layered interface of virtual computers, provided by a typical computer system

The implementation systems of the first high-level programming languages, constructed in the late 1950s, were among the most complex software systems of that time. In the 1960s, intensive research efforts were made to understand and formalize the process of constructing these high-level language implementations. The greatest success of those efforts was in the area of syntax analysis, primarily because that part of the implementation process is an application of parts of automata theory and formal language theory that were then well understood.

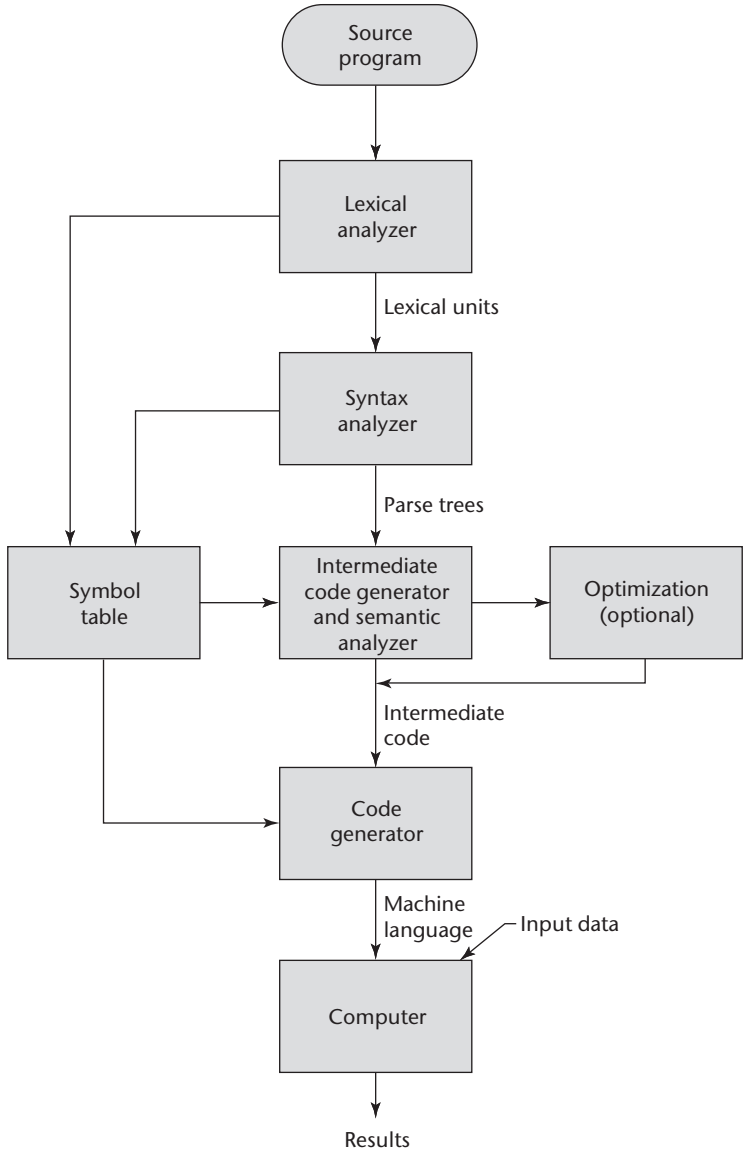
1.7.1 Compilation

Programming languages can be implemented by any of three general methods. At one extreme, programs can be translated into machine language, which can be executed directly on the computer. This method is called a **compiler implementation** and has the advantage of very fast program execution, once the translation process is complete. Most production implementations of languages, such as C, COBOL, and C++, are by compilers.

The language that a compiler translates is called the **source language**. The process of compilation and program execution takes place in several phases, the most important of which are shown in Figure 1.3.

Figure 1.3

The compilation process



The lexical analyzer gathers the characters of the source program into lexical units. The lexical units of a program are identifiers, special words, operators, and punctuation symbols. The lexical analyzer ignores comments in the source program because the compiler has no use for them.

The syntax analyzer takes the lexical units from the lexical analyzer and uses them to construct hierarchical structures called **parse trees**. These parse trees represent the syntactic structure of the program. In many cases, no actual parse tree structure is constructed; rather, the information that would be required to

build a tree is generated and used directly. Both lexical units and parse trees are discussed further in Chapter 3. Lexical analysis and syntax analysis, or parsing, are discussed in Chapter 4.

The intermediate code generator produces a program in a different language, at an intermediate level between the source program and the final output of the compiler: the machine language program.[4] Intermediate languages sometimes look very much like assembly languages, and in fact, sometimes are actual assembly languages. In other cases, the intermediate code is at a level somewhat higher than an assembly language. The semantic analyzer is an integral part of the intermediate code generator. The semantic analyzer checks for errors, such as type errors, that are difficult, if not impossible, to detect during syntax analysis.

Optimization, which improves programs (usually in their intermediate code version) by making them smaller or faster or both. Because many kinds of optimization are difficult to do on machine language, most optimization is done on the intermediate code.

The code generator translates the optimized intermediate code version of the program into an equivalent machine language program.

The symbol table serves as a database for the compilation process. The primary contents of the symbol table are the type and attribute information of each user-defined name in the program. This information is placed in the symbol table by the lexical and syntax analyzers and is used by the semantic analyzer and the code generator.

As stated previously, although the machine language generated by a compiler can be executed directly on the hardware, it must nearly always be run along with some other code. Most user programs also require programs from the operating system. Among the most common of these are programs for input and output. The compiler builds calls to required system programs when they are needed by the user program. Before the machine language programs produced by a compiler can be executed, the required programs from the operating system must be found and linked to the user program. The linking operation connects the user program to the system programs by placing the addresses of the entry points of the system programs in the calls to them in the user program. The user and system code together are sometimes called a **load module**, or **executable image**. The process of collecting system programs and linking them to user programs is called **linking and loading**, or sometimes just **linking**. It is accomplished by a systems program called a **linker**.

In addition to systems programs, user programs must often be linked to previously compiled programs that reside in libraries. So the linker not only links a given program to system programs, but also it may link it to other user or system-supplied programs.

The speed of the connection between a computer's memory and its processor often determines the speed of the computer, because instructions often can be executed faster than they can be moved to the processor for execution.

---

4. Note that the words *program* and *code* are often used interchangeably.

This connection is called the **von Neumann bottleneck**; it is the primary limiting factor in the speed of von Neumann architecture computers. The von Neumann bottleneck has been one of the primary motivations for the research and development of parallel computers.

1.7.2 Pure Interpretation

Pure interpretation lies at the opposite end (from compilation) among implementation methods. With this approach, programs are interpreted by another program called an interpreter, with no translation whatever. The interpreter program acts as a software simulation of a machine whose fetch-execute cycle deals with high-level language program statements rather than machine instructions. This software simulation obviously provides a virtual machine for the language.

Pure interpretation has the advantage of allowing easy implementation of many source-level debugging operations, because all run-time error messages can refer to source-level units. For example, if an array index is found to be out of range, the error message can easily indicate the source line of the error and the name of the array. On the other hand, this method has the serious disadvantage that execution is 10 to 100 times slower than in compiled systems. The primary source of this slowness is the decoding of the high-level language statements, which are far more complex than machine language instructions (although there may be fewer statements than instructions in equivalent machine code). Furthermore, regardless of how many times a statement is executed, it must be decoded every time. Therefore, statement decoding, rather than the connection between the processor and memory, is the bottleneck of a pure interpreter.

Another disadvantage of pure interpretation is that it often requires more space. In addition to the source program, the symbol table must be present during interpretation. Furthermore, the source program may be stored in a form designed for easy access and modification rather than one that provides for minimal size.

Although some simple early languages of the 1960s (APL, SNOBOL (Griswold et al., 1971), and Lisp) were purely interpreted, by the 1980s, the approach was rarely used on high-level languages. However, in recent years, pure interpretation has made a significant comeback with some Web scripting languages, such as JavaScript and PHP, which are now widely used. The process of pure interpretation is shown in Figure 1.4.

1.7.3 Hybrid Implementation Systems

Some language implementation systems are a compromise between compilers and pure interpreters; they translate high-level language programs to an intermediate language designed to allow easy interpretation. This method is faster than pure interpretation because the source language statements are decoded only once. Such implementations are called **hybrid implementation systems**.

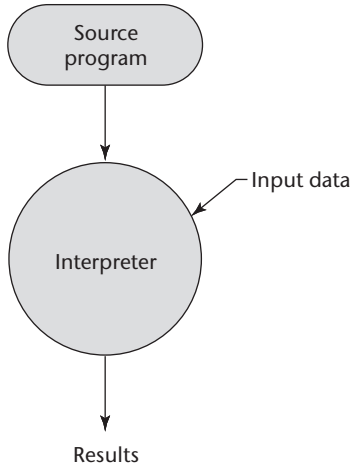


Figure 1.4
Pure interpretation

The process used in a hybrid implementation system is shown in Figure 1.5. Instead of translating intermediate language code to machine code, it simply interprets the intermediate code.

Perl is implemented with a hybrid system. Perl programs are partially compiled to detect errors before interpretation and to simplify the interpreter.

Initial implementations of Java were all hybrid. Its intermediate form, called **byte code**, provides portability to any machine that has a byte code interpreter and an associated run-time system. Together, these are called the Java Virtual Machine. There are now systems that translate Java byte code into machine code for faster execution.

A Just-in-Time (JIT) implementation system initially translates programs to an intermediate language. Then, during execution, it compiles intermediate language methods into machine code when they are called. The machine code version is kept for subsequent calls. JIT systems now are widely used for Java programs. Also, the .NET languages are all implemented with a JIT system.

Sometimes an implementor may provide both compiled and interpreted implementations for a language. In these cases, the interpreter is used to develop and debug programs. Then, after a (relatively) bug-free state is reached, the programs are compiled to increase their execution speed.

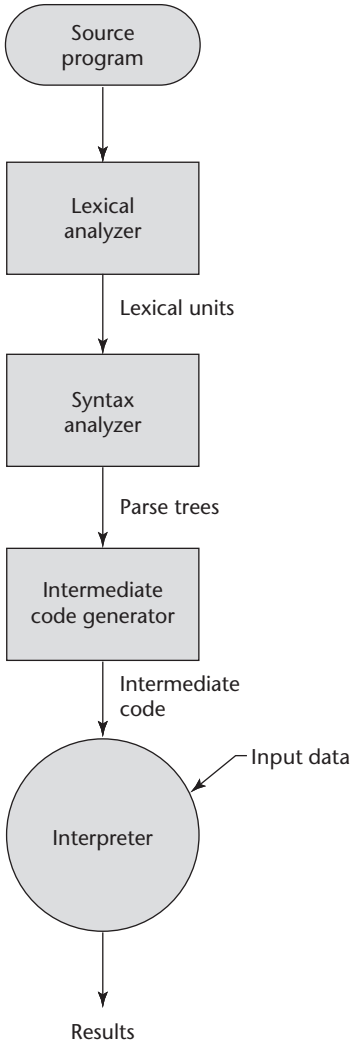
1.7.4 Preprocessors

A **preprocessor** is a program that processes a program just before the program is compiled. Preprocessor instructions are embedded in programs. The preprocessor is essentially a macro expander. Preprocessor instructions are commonly used to specify that the code from another file is to be included. For example, the C preprocessor instruction

```
#include "myLib.h"
```

Figure 1.5

Hybrid implementation system



causes the preprocessor to copy the contents of `myLib.h` into the program at the position of the `#include`.

Other preprocessor instructions are used to define symbols to represent expressions. For example, one could use

```
#define max(A, B) ((A) > (B) ? (A) : (B))
```

to determine the largest of two given expressions. For example, the expression

```
x = max(2 * y, z / 1.73);
```

would be expanded by the preprocessor to

```
x = ((2 * y) > (z / 1.73) ? (2 * y) : (z / 1.73);
```

Notice that this is one of those cases where expression side effects can cause trouble. For example, if either of the expressions given to the `max` macro have side effects—such as `z++`—it could cause a problem. Because one of the two expression parameters is evaluated twice, this could result in `z` being incremented twice by the code produced by the macro expansion.

1.8 Programming Environments

A programming environment is the collection of tools used in the development of software. This collection may consist of only a file system, a text editor, a linker, and a compiler. Or it may include a large collection of integrated tools, each accessed through a uniform user interface. In the latter case, the process of the development and maintenance of software is greatly enhanced. Therefore, the characteristics of a programming language are not the only measure of the software development capability of a system. We now briefly describe several programming environments.

UNIX is an older programming environment, first distributed in the middle 1970s, built around a portable multiprogramming operating system. It provides a wide array of powerful support tools for software production and maintenance in a variety of languages. In the past, the most important feature absent from UNIX was a uniform interface among its tools. This made it more difficult to learn and to use. However, UNIX is now often used through a GUI that runs on top of UNIX. Examples of UNIX GUIs are the Solaris Common Desktop Environment (CDE), GNOME, and KDE. These GUIs make the interface to UNIX appear similar to that of Windows and Macintosh systems.

Borland JBuilder is a programming environment that provides an integrated compiler, editor, debugger, and file system for Java development, where all four are accessed through a graphical interface. JBuilder is a complex and powerful system for creating Java software.

Microsoft Visual Studio .NET is a relatively recent step in the evolution of software development environments. It is a large and elaborate collection of software development tools, all used through a windowed interface. This system can be used to develop software in any one of the five .NET languages: C#, Visual Basic.NET, JScript (Microsoft's version of JavaScript), F# (a functional language), and C++/CLI.

NetBeans is a development environment that is primarily used for Java application development but also supports JavaScript, Ruby, and PHP. Both Visual Studio and NetBeans are more than development environments—they are also frameworks, which means they actually provide common parts of the code of the application.

SUMMARY

The study of programming languages is valuable for some important reasons: It increases our capacity to use different constructs in writing programs, enables us to choose languages for projects more intelligently, and makes learning new languages easier.

Computers are used in a wide variety of problem-solving domains. The design and evaluation of a particular programming language is highly dependent on the domain in which it is to be used.

Among the most important criteria for evaluating languages are readability, writability, reliability, and overall cost. These will be the basis on which we examine and judge the various language features discussed in the remainder of the book.

The major influences on language design have been machine architecture and software design methodologies.

Designing a programming language is primarily an engineering feat, in which a long list of trade-offs must be made among features, constructs, and capabilities.

The major methods of implementing programming languages are compilation, pure interpretation, and hybrid implementation.

Programming environments have become important parts of software development systems, in which the language is just one of the components.

REVIEW QUESTIONS

1. Why is it useful for a programmer to have some background in language design, even though he or she may never actually design a programming language?
2. How can knowledge of programming language characteristics benefit the whole computing community?
3. What programming language has dominated scientific computing over the past 60 years?
4. What programming language has dominated business applications over the past 60 years?
5. What programming language has dominated artificial intelligence over the past 60 years?
6. In what language is most of UNIX written?
7. What is the disadvantage of having too many features in a language?
8. How can user-defined operator overloading harm the readability of a program?
9. What is one example of a lack of orthogonality in the design of C?

10. What language used orthogonality as a primary design criterion?
11. What primitive control statement is used to build more complicated control statements in languages that lack them?
12. What does it mean for a program to be reliable?
13. Why is type checking the parameters of a subprogram important?
14. What is aliasing?
15. What is exception handling?
16. Why is readability important to writability?
17. How is the cost of compilers for a given language related to the design of that language?
18. What have been the strongest influences on programming language design over the past 60 years?
19. What is the name of the category of programming languages whose structure is dictated by the von Neumann computer architecture?
20. What two programming language deficiencies were discovered as a result of the research in software development in the 1970s?
21. What are the three fundamental features of an object-oriented programming language?
22. What language was the first to support the three fundamental features of object-oriented programming?
23. What is an example of two language design criteria that are in direct conflict with each other?
24. What are the three general methods of implementing a programming language?
25. Which produces faster program execution, a compiler or a pure interpreter?
26. What role does the symbol table play in a compiler?
27. What does a linker do?
28. Why is the von Neumann bottleneck important?
29. What are the advantages in implementing a language with a pure interpreter?

PROBLEM SET

1. Do you believe our capacity for abstract thought is influenced by our language skills? Support your opinion.
2. What are some features of specific programming languages you know whose rationales are a mystery to you?

3. What arguments can you make for the idea of a single language for all programming domains?
4. What arguments can you make against the idea of a single language for all programming domains?
5. Name and explain another criterion by which languages can be judged (in addition to those discussed in this chapter).
6. What common programming language statement, in your opinion, is most detrimental to readability?
7. Java uses a right brace to mark the end of all compound statements. What are the arguments for and against this design?
8. Many languages distinguish between uppercase and lowercase letters in user-defined names. What are the pros and cons of this design decision?
9. Explain the different aspects of the cost of a programming language.
10. What are the arguments for writing efficient programs even though hardware is relatively inexpensive?
11. Describe some design trade-offs between efficiency and safety in some language you know.
12. In your opinion, what major features would a perfect programming language include?
13. Was the first high-level programming language you learned implemented with a pure interpreter, a hybrid implementation system, or a compiler? (You may have to research this.)
14. Describe the advantages and disadvantages of some programming environment you have used.
15. How do type declaration statements for simple variables affect the readability of a language, considering that some languages do not require them?
16. Write an evaluation of some programming language you know, using the criteria described in this chapter.
17. Some programming languages—for example, Pascal—have used the semicolon to separate statements, while Java uses it to terminate statements. Which of these, in your opinion, is most natural and least likely to result in syntax errors? Support your answer.
18. Many contemporary languages allow two kinds of comments: one in which delimiters are used on both ends (multiple-line comments), and one in which a delimiter marks only the beginning of the comment (one-line comments). Discuss the advantages and disadvantages of each of these with respect to our criteria.

2

Evolution of the Major Programming Languages

2.1 Zuse's Plankalkül
2.2 Pseudocodes
2.3 The IBM 704 and Fortran
2.4 Functional Programming: Lisp
2.5 The First Step Toward Sophistication: ALGOL 60
2.6 Computerizing Business Records: COBOL
2.7 The Beginnings of Timesharing: Basic
2.8 Everything for Everybody: PL/I
2.9 Two Early Dynamic Languages: APL and SNOBOL
2.10 The Beginnings of Data Abstraction: SIMULA 67
2.11 Orthogonal Design: ALGOL 68
2.12 Some Early Descendants of the ALGOLs
2.13 Programming Based on Logic: Prolog
2.14 History's Largest Design Effort: Ada
2.15 Object-Oriented Programming: Smalltalk
2.16 Combining Imperative and Object-Oriented Features: C++
2.17 An Imperative-Based Object-Oriented Language: Java
2.18 Scripting Languages
2.19 The Flagship .NET Language: C#
2.20 Markup-Programming Hybrid Languages

This chapter describes the development of a collection of programming languages. It explores the environment in which each was designed and focuses on the contributions of the language and the motivation for its development. Overall language descriptions are not included; rather, we discuss only some of the new features introduced by each language. Of particular interest are the features that most influenced subsequent languages or the field of computer science.

This chapter does not include an in-depth discussion of any language feature or concept; that is left for later chapters. Brief, informal explanations of features will suffice for our trek through the development of these languages.

We discuss a wide variety of languages and language concepts that will not be familiar to many readers. These topics are described in detail only in later chapters. Those who find this unsettling may prefer to delay reading this chapter until the rest of the book has been studied.

The choice as to which languages to discuss here was subjective, and some readers will unhappily note the absence of one or more of their favorites. However, to keep this historical coverage to a reasonable size, it was necessary to leave out some languages that some regard highly. The choices were based on our estimate of each language's importance to language development and the computing world as a whole. We also include brief discussions of some other languages that are referenced later in the book.

The organization of this chapter is as follows: The initial versions of languages generally are discussed in chronological order. However, subsequent versions of languages appear with their initial version, rather than in later sections. For example, Fortran 2003 is discussed in the section with Fortran I (1956). Also, in some cases, languages of secondary importance that are related to a language that has its own section appear in that section.

This chapter includes listings of 14 complete example programs, each in a different language. These programs are not described in this chapter; they are meant to illustrate the appearance of programs in these languages. Readers familiar with any of the common imperative languages should be able to read and understand most of the code in these programs, except those in Lisp, COBOL, and Smalltalk. (A Scheme function similar to the Lisp example is discussed in Chapter 15.) The same problem is solved by the Fortran, ALGOL 60, PL/I, Basic, Pascal, C, Perl, Ada, Java, JavaScript, and C# programs. Note that most of the contemporary languages in this list support dynamic arrays, but because of the simplicity of the example problem, we did not use them in the example programs. Also, in the Fortran 95 program, we avoided using the features that could have avoided the use of loops altogether, in part to keep the program simple and readable and in part just to illustrate the basic loop structure of the language.

Figure 2.1 is a chart of the genealogy of the high-level languages discussed in this chapter.

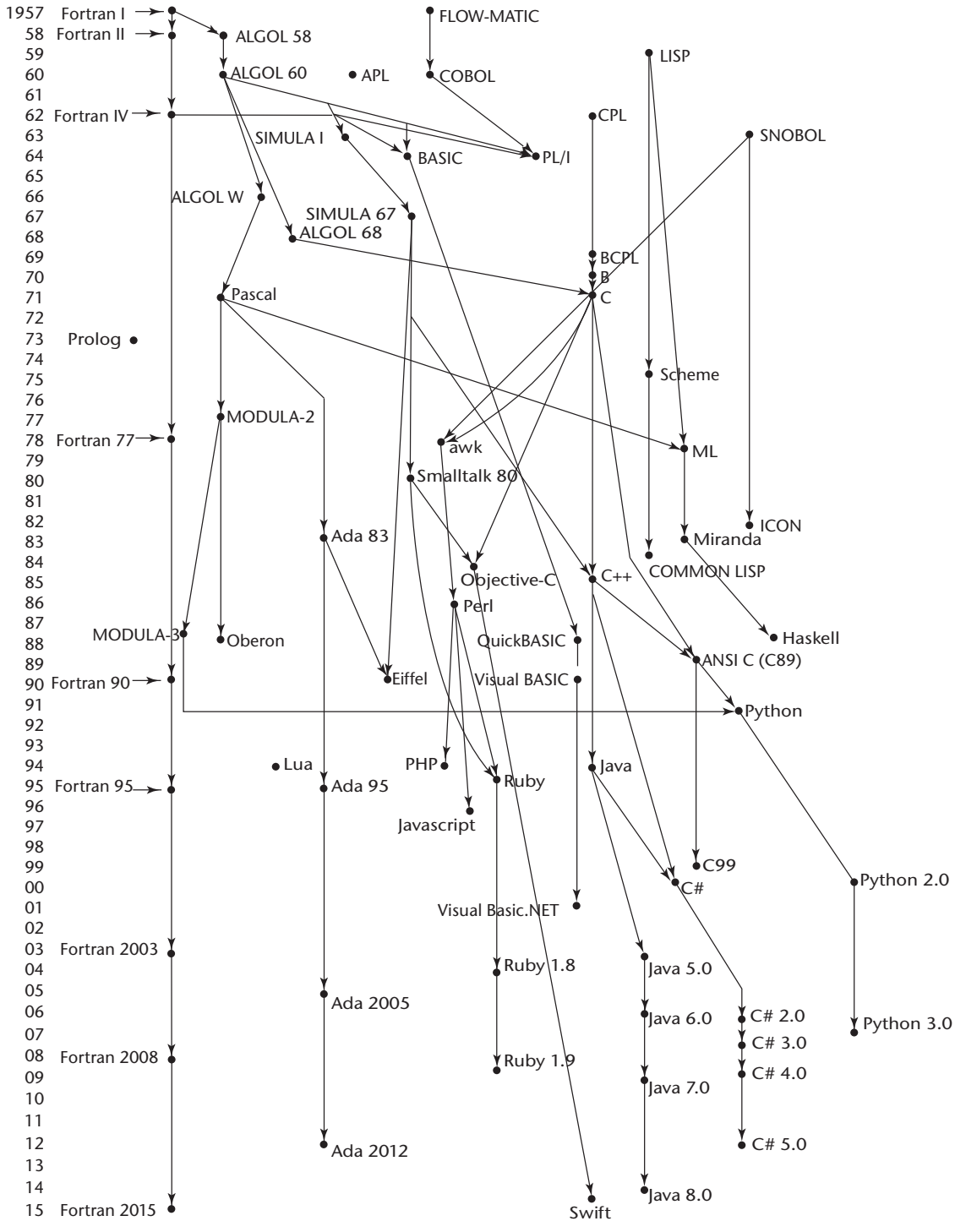


Figure 2.1

Genealogy of common high-level programming languages

2.1 Zuse's Plankalkül

The first programming language discussed in this chapter is highly unusual in several respects. For one thing, it was never implemented. Furthermore, although developed in 1945, its description was not published until 1972. Because so few people were familiar with the language, some of its capabilities did not appear in other languages until 15 years after its development.

2.1.1 Historical Background

Between 1936 and 1945, German scientist Konrad Zuse (pronounced "Tsoo-zuh") built a series of complex and sophisticated computers from electromechanical relays. By early 1945, Allied bombing had destroyed all but one of his latest models, the Z4, so he moved to a remote Bavarian village, Hinterstein, and his research group members went their separate ways.

Working alone, Zuse embarked on an effort to develop a language for expressing computations for the Z4, a project he had begun in 1943 as a proposal for his Ph.D. dissertation. He named this language Plankalkül, which means *program calculus*. In a lengthy manuscript dated 1945 but not published until 1972 (Zuse, 1972), Zuse defined Plankalkül and wrote algorithms in the language to solve a wide variety of problems.

2.1.2 Language Overview

Plankalkül was remarkably complete, with some of its most advanced features in the area of data structures. The simplest data type in Plankalkül was the single bit. Integer and floating-point numeric types were built from the bit type. The floating-point type used twos-complement notation and the "hidden bit" scheme currently used to avoid storing the most significant bit of the normalized fraction part of a floating-point value.

In addition to the usual scalar types, Plankalkül included arrays and records (called *structs* in the C-based languages). The records could include nested records.

Although the language had no explicit goto, it did include an iterative statement similar to the Ada **`for`**. It also had the command **`Fin`** with a superscript that specified an exit out of a given number of iteration loop nestings or to the beginning of a new iteration cycle. Plankalkül included a selection statement, but it did not allow an `else` clause.

One of the most interesting features of Zuse's programs was the inclusion of mathematical expressions showing the current relationships between program variables. These expressions stated what would be true during execution at the points in the code where they appeared. These are very similar to the assertions of Java and in those in axiomatic semantics, which is discussed in Chapter 3.

Zuse's manuscript contained programs of far greater complexity than any written prior to 1945. Included were programs to sort arrays of numbers; test the connectivity of a given graph; carry out integer and floating-point operations, including square root; and perform syntax analysis on logic formulas that had parentheses and operators in six different levels of precedence. Perhaps most remarkable were his 49 pages of algorithms for playing chess, a game in which he was not an expert.

If a computer scientist had found Zuse's description of Plankalkül in the early 1950s, the single aspect of the language that would have hindered its implementation as defined would have been the notation. Each statement consisted of either two or three lines of code. The first line was most like the statements of current languages. The second line, which was optional, contained the subscripts of the array references in the first line. The same method of indicating subscripts was used by Charles Babbage in programs for his Analytical Engine in the middle of the nineteenth century. The last line of each Plankalkül statement contained the type names for the variables mentioned in the first line. This notation is quite intimidating when first seen.

The following example assignment statement, which assigns the value of the expression `A[4} + 1` to `A[5]`, illustrates this notation. The row labeled `V` is for subscripts, and the row labeled `S` is for the data types. In this example, `1.n` means an integer of *n* bits:

```
  | A + 1 =>  A
V | 4         5
S | 1.n       1.n
```

We can only speculate on the direction that programming language design might have taken if Zuse's work had been widely known in 1945 or even 1950. It is also interesting to consider how his work might have been different had he done it in a peaceful environment surrounded by other scientists, rather than in Germany in 1945 in virtual isolation.

2.2 Pseudocodes

First, note that the word *pseudocode* is used here in a different sense than its contemporary meaning. We call the languages discussed in this section pseudocodes because that's what they were named at the time they were developed and used (the late 1940s and early 1950s). However, they are clearly not pseudocodes in the contemporary sense.

The computers that became available in the late 1940s and early 1950s were far less usable than those of today. In addition to being slow, unreliable, expensive, and having extremely small memories, the machines of that time were difficult to program because of the lack of supporting software.

There were no high-level programming languages or even assembly languages, so programming was done in machine code, which is both tedious and

error prone. Among its problems is the use of numeric codes for specifying instructions. For example, an ADD instruction might be specified by the code 14 rather than a connotative textual name, even if only a single letter. This makes programs very difficult to read. A more serious problem is absolute addressing, which makes program modification tedious and error prone. For example, suppose we have a machine language program stored in memory. Many of the instructions in such a program refer to other locations within the program, usually to reference data or to indicate the targets of branch instructions. Inserting an instruction at any position in the program other than at the end invalidates the correctness of all instructions that refer to addresses beyond the insertion point, because those addresses must be increased to make room for the new instruction. To make the addition correctly, all instructions that refer to addresses that follow the addition must be found and modified. A similar problem occurs with deletion of an instruction. In this case, however, machine languages often include a "no operation" instruction that can replace deleted instructions, thereby avoiding the problem.

These are standard problems with all machine languages and were the primary motivations for inventing assemblers and assembly languages. In addition, most programming problems of that time were numerical and required floating-point arithmetic operations and indexing of some sort to allow the convenient use of arrays. Neither of these capabilities, however, was included in the architecture of the computers of the late 1940s and early 1950s. These deficiencies naturally led to the development of somewhat higher-level languages.

2.2.1 Short Code

The first of these new languages, named Short Code, was developed by John Mauchly in 1949 for the BINAC computer, which was one of the first successful stored-program electronic computers. Short Code was later transferred to a UNIVAC I computer (the first commercial electronic computer sold in the United States) and, for several years, was one of the primary means of programming those machines. Although little is known of the original Short Code because its complete description was never published, a programming manual for the UNIVAC I version did survive (Remington-Rand, 1952). It is safe to assume that the two versions were very similar.

The words of the UNIVAC I's memory had 72 bits, grouped as 12 six-bit bytes. Short Code consisted of coded versions of mathematical expressions that were to be evaluated. The codes were byte-pair values, and many equations could be coded in a word. The following operation codes were included:

```
01 -    06 abs value      1n (n+2)nd power
02 )    07 +              2n (n+2)nd root
03 =    08 pause          4n if <= n
04 /    09 (              58 print and tab
```

Variables were named with byte-pair codes, as were locations to be used as constants. For example, `X0` and `Y0` could be variables. The statement

```
X0 = SQRT(ABS(Y0))
```

would be coded in a word as `00 X0 03 20 06 Y0`. The initial `00` was used as padding to fill the word. Interestingly, there was no multiplication code; multiplication was indicated by simply placing the two operands next to each other, as in algebra.

Short Code was not translated to machine code; rather, it was implemented with a pure interpreter. At the time, this process was called *automatic programming*. It clearly simplified the programming process, but at the expense of execution time. Short Code interpretation was approximately 50 times slower than machine code.

2.2.2 Speedcoding

In other places, interpretive systems were being developed that extended machine languages to include floating-point operations. The Speedcoding system developed by John Backus for the IBM 701 is an example of such a system (Backus, 1954). The Speedcoding interpreter effectively converted the 701 to a virtual three-address floating-point calculator. The system included pseudoinstructions for the four arithmetic operations on floating-point data, as well as operations such as square root, sine, arc tangent, exponent, and logarithm. Conditional and unconditional branches and input/output conversions were also part of the virtual architecture. To get an idea of the limitations of such systems, consider that the remaining usable memory after loading the interpreter was only 700 words and that the add instruction took 4.2 milliseconds to execute. On the other hand, Speedcoding included the novel facility of automatically incrementing address registers. This facility did not appear in hardware until the UNIVAC 1107 computers of 1962. Because of such features, matrix multiplication could be done in 12 Speedcoding instructions. Backus claimed that problems that could take two weeks to program in machine code could be programmed in a few hours using Speedcoding.

2.2.3 The UNIVAC "Compiling" System

Between 1951 and 1953, a team led by Grace Hopper at UNIVAC developed a series of "compiling" systems named A-0, A-1, and A-2 that expanded a pseudocode into machine code subprograms in the same way as macros are expanded into assembly language. The pseudocode source for these "compilers" was still quite primitive, although even this was a great improvement over machine code because it made source programs much shorter. Wilkes (1952) independently suggested a similar process.

2.2.4 Related Work

Other means of easing the task of programming were being developed at about the same time. At Cambridge University, David J. Wheeler (1950) developed a method of using blocks of relocatable addresses to solve, at least partially, the problem of absolute addressing, and later, Maurice V. Wilkes (also at Cambridge) extended the idea to design an assembly program that could combine chosen subroutines and allocate storage (Wilkes et al., 1951, 1957). This was indeed an important and fundamental advance.

We should also mention that assembly languages, which are quite different from the pseudocodes discussed, evolved during the early 1950s. However, they had little impact on the design of high-level languages.

2.3 The IBM 704 and Fortran

Certainly one of the greatest single advances in computing came with the introduction of the IBM 704 in 1954, in large measure because its capabilities prompted the development of Fortran. One could argue that if it had not been IBM with the 704 and Fortran, it would soon thereafter have been some other organization with a similar computer and related high-level language. However, IBM was the first with both the foresight and the resources to undertake these developments.

2.3.1 Historical Background

One of the primary reasons why the slowness of interpretive systems was tolerated from the late 1940s to the mid-1950s was the lack of floating-point hardware in the available computers. All floating-point operations had to be simulated in software, a very time-consuming process. Because so much processor time was spent in software floating-point processing, the overhead of interpretation and the simulation of indexing were relatively insignificant. As long as floating-point had to be done by software, interpretation was an acceptable expense. However, many programmers of that time never used interpretive systems, preferring the efficiency of hand-coded machine (or assembly) language. The announcement of the IBM 704 system, with both indexing and floating-point instructions in hardware, heralded the end of the interpretive era, at least for scientific computation. The inclusion of floating-point hardware removed the hiding place for the cost of interpretation.

Although Fortran is often credited with being the first compiled high-level language, the question of who deserves credit for implementing the first such language is somewhat open. Knuth and Pardo (1977) give the credit to Alick E. Glennie for his Autocode compiler for the Manchester Mark I computer. Glennie developed the compiler at Fort Halstead, Royal Armaments Research Establishment, in England. The compiler was operational by September 1952. However, according to John Backus (Wexelblat, 1981, p. 26), Glennie's

Autocode was so low level and machine oriented that it should not be considered a compiled system. Backus gives the credit to Laning and Zierler at the Massachusetts Institute of Technology.

The Laning and Zierler system (Laning and Zierler, 1954) was the first algebraic translation system to be implemented. By algebraic, we mean that it translated arithmetic expressions, used separately coded subprograms to compute transcendental functions (e.g., sine and logarithm), and included arrays. The system was implemented on the MIT Whirlwind computer, in experimental prototype form, in the summer of 1952 and in a more usable form by May 1953. The translator generated a subroutine call to code each formula, or expression, in the program. The source language was easy to read, and the only actual machine instructions included were for branching. Although this work preceded the work on Fortran, it never escaped MIT.

In spite of these earlier works, the first widely accepted compiled high-level language was Fortran. The following subsections chronicle this important development.

2.3.2 Design Process

Even before the 704 system was announced in May 1954, plans were begun for Fortran. By November 1954, John Backus and his group at IBM had produced the report titled "The IBM Mathematical FORmula TRANslating System: FORTRAN" (IBM, 1954). This document described the first version of Fortran, which we refer to as Fortran 0, prior to its implementation. It also boldly stated that Fortran would provide the efficiency of hand-coded programs and the ease of programming of the interpretive pseudocode systems. In another burst of optimism, the document stated that Fortran would eliminate coding errors and the debugging process. Based on this premise, the first Fortran compiler included little syntax error checking.

The environment in which Fortran was developed was as follows: (1) Computers had small memories and were slow and relatively unreliable; (2) the primary use of computers was for scientific computations; (3) there were no existing efficient and effective ways to program computers; and (4) because of the high cost of computers compared to the cost of programmers, speed of the generated object code was the primary goal of the first Fortran compilers. The characteristics of the early versions of Fortran follow directly from this environment.

2.3.3 Fortran I Overview

Fortran 0 was modified during the implementation period, which began in January 1955 and continued until the release of the compiler in April 1957. The implemented language, which we call Fortran I, is described in the first Fortran *Programmer's Reference Manual*, published in October 1956 (IBM, 1956). Fortran I included input/output formatting, variable names of up to six characters (it had been just two in Fortran 0), user-defined subroutines, although they

could not be separately compiled, the `If` selection statement, and the `Do` loop statement.

All of Fortran I's control statements were based on 704 instructions. It is not clear whether the 704 designers dictated the control statement design of Fortran I or whether the designers of Fortran I suggested these instructions to the 704 designers.

There were no data-typing statements in the Fortran I language. Variables whose names began with `I`, `J`, `K`, `L`, `M`, and `N` were implicitly integer type, and all others were implicitly floating-point. The choice of the letters for this convention was based on the fact that at that time scientists and engineers used letters as variable subscripts, usually *i*, *j*, and *k*. In a gesture of generosity, Fortran's designers threw in the three additional letters.

The most audacious claim made by the Fortran development group during the design of the language was that the machine code produced by the compiler would be about half as efficient as what could be produced by hand.[1] This, more than anything else, made skeptics of potential users and prevented a great deal of interest in Fortran before its actual release. To almost everyone's surprise, however, the Fortran development group nearly achieved its goal in efficiency. The largest part of the 18 worker-years of effort used to construct the first compiler had been spent on optimization, and the results were remarkably effective.

The early success of Fortran is shown by the results of a survey made in April 1958. At that time, roughly half of the code being written for 704s was being written in Fortran, in spite of the skepticism of most of the programming world only a year earlier.

2.3.4 Fortran II

The Fortran II compiler was distributed in the spring of 1958. It fixed many of the bugs in the Fortran I compilation system and added some significant features to the language, the most important being the independent compilation of subroutines. Without independent compilation, any change in a program required that the entire program be recompiled. Fortran I's lack of independent-compilation capability, coupled with the poor reliability of the 704, placed a practical restriction on the length of programs to about 300 to 400 lines (Wexelblat, 1981, p. 68). Longer programs had a poor chance of being compiled completely before a machine failure occurred. The capability of including precompiled machine language versions of subprograms shortened the compilation process considerably and made it practical to develop much larger programs.

1. In fact, the Fortran team believed that the code generated by their compiler could be no less than half as fast as handwritten machine code, or the language would not be adopted by users.

2.3.5 Fortrans IV, 77, 90, 95, 2003, and 2008

A Fortran III was developed, but it was never widely distributed. Fortran IV, however, became one of the most widely used programming languages of its time. It evolved over the period 1960 to 1962 and was standardized as Fortran 66 (ANSI, 1966), although that name was rarely used. Fortran IV was an improvement over Fortran II in many ways. Among its most important additions were explicit type declarations for variables, a logical `If` construct, and the capability of passing subprograms as parameters to other subprograms.

Fortran IV was replaced by Fortran 77, which became the new standard in 1978 (ANSI, 1978a). Fortran 77 retained most of the features of Fortran IV and added character string handling, logical loop control statements, and an `If` with an optional `Else` clause.

Fortran 90 (ANSI, 1992) was dramatically different from Fortran 77. The most significant additions were dynamic arrays, records, pointers, a multiple selection statement, and modules. In addition, Fortran 90 subprograms could be recursively called.

A new concept that was included in the Fortran 90 definition was that of removing some language features from earlier versions. While Fortran 90 included all of the features of Fortran 77, the language definition included a list of constructs that were recommended for removal in the next version of the language.

Fortran 90 included two simple syntactic changes that altered the appearance of both programs and the literature describing the language. First, the required fixed format of code, which required the use of specific character positions for specific parts of statements, was dropped. For example, statement labels could appear only in the first five positions and statements could not begin before the seventh position. This rigid formatting of code was designed around the use of punch cards. The second change was that the official spelling of FORTRAN became Fortran. This change was accompanied by the change in convention of using all uppercase letters for keywords and identifiers in Fortran programs. The new convention was that only the first letter of keywords and identifiers would be uppercase.

Fortran 95 (INCITS/ISO/IEC, 1997) continued the evolution of the language, but only a few changes were made. Among other things, a new iteration construct, `Forall`, was added to ease the task of parallelizing Fortran programs.

Fortran 2003 (Metcalf et al., 2004) added support for object-oriented programming, parameterized derived types, procedure pointers, and interoperability with the C programming language.

The latest version of Fortran, Fortran 2008 (ISO/IEC 1539-1, 2010), added support for blocks to define local scopes, co-arrays, which provide a parallel execution model, and the `DO CONCURRENT` construct, to specify loops without interdependencies. A new version, Fortran 2015, is under development and is scheduled for release in 2018.

2.3.6 Evaluation

The original Fortran design team thought of language design only as a necessary prelude to the critical task of designing the translator. Furthermore, it never occurred to them that Fortran would be used on computers not

manufactured by IBM. However, they were forced to consider building Fortran compilers for other IBM machines only because the successor to the 704, the 709, was announced before the 704 Fortran compiler was released. The effect that Fortran has had on the use of computers, along with the fact that all subsequent programming languages owe a debt to Fortran, is indeed impressive in light of the modest goals of its designers.

One of the features of Fortran I, and all of its successors before 90, that allows highly optimizing compilers was that the types and storage for all variables are fixed before run time. No new variables or space could be allocated during execution. This was a sacrifice of flexibility to simplicity and efficiency. It eliminated the possibility of recursive subprograms and made it difficult to implement data structures that grow or change shape dynamically. Of course, the kinds of programs that were being built at the time of the development of the early versions of Fortran were primarily numerical in nature and were simple in comparison with more recent software projects. Therefore, the sacrifice was not a great one.

The overall success of Fortran is difficult to overstate: It dramatically changed the way computers are used. This is, of course, in large part due to its being the first widely used high-level language. In comparison with concepts and languages developed later, early versions of Fortran suffer in a variety of ways, as should be expected. After all, it would not be fair to compare the performance and comfort of a 1910 Model T Ford with the performance and comfort of a 2017 Ford Mustang. Nevertheless, in spite of the inadequacies of Fortran, the momentum of the huge investment in Fortran software, among other factors, has kept it in use for 60 years.

Alan Perlis, one of the designers of ALGOL 60, said of Fortran in 1978, "Fortran is the *lingua franca* of the computing world. It is the language of the streets in the best sense of the word, not in the prostitutional sense of the word. And it has survived and will survive because it has turned out to be a remarkably useful part of a very vital commerce" (Wexelblat, 1981, p. 161).

The following is an example of a Fortran 95 program:

```
! Fortran 95 Example program
!  Input:   An integer, List_Len, where List_Len is less
!           than 100, followed by List_Len-Integer values
!  Output: The number of input values that are greater
!          than the average of all input values
Implicit none
Integer Dimension(99) :: Int_List
Integer :: List_Len, Counter, Sum, Average, Result
Result= 0
Sum = 0
Read *, List_Len
If ((List_Len > 0) .AND. (List_Len < 100)) Then
! Read input data into an array and compute its sum
   Do Counter = 1, List_Len
      Read *, Int_List(Counter)
      Sum = Sum + Int_List(Counter)
   End Do
```

```
! Compute the average
   Average = Sum / List_Len
! Count the values that are greater than the average
   Do Counter = 1, List_Len
      If (Int_List(Counter) > Average) Then
         Result = Result + 1
      End If
   End Do
! Print the result
   Print *, 'Number of values > Average is:', Result
Else
   Print *, 'Error - list length value is not legal'
End If
End Program Example
```

2.4 Functional Programming: Lisp

The first functional programming language was invented to provide language features for list processing, the need for which grew out of the first applications in the area of artificial intelligence (AI).

2.4.1 The Beginnings of Artificial Intelligence (AI) and List Processing

Interest in AI appeared in the mid-1950s in a number of places. Some of this interest grew out of linguistics, some from psychology, and some from mathematics. Linguists were concerned with natural language processing. Psychologists were interested in modeling human information storage and retrieval, as well as other fundamental processes of the brain. Mathematicians were interested in mechanizing certain intelligent processes, such as theorem proving. All of these investigations arrived at the same conclusion: Some method must be developed to allow computers to process symbolic data in linked lists. At the time, nearly all computation was on numeric data in arrays.

The concept of list processing was developed by Allen Newell, J. C. Shaw, and Herbert Simon at the RAND Corporation. It was first published in a classic paper that describes one of the first AI programs, the Logic Theorist,[2] and a language in which it could be implemented (Newell and Simon, 1956). The language, named IPL-I (Information Processing Language I), was never implemented. The next version, IPL-II, was implemented on a RAND Johnniac computer. Development of IPL continued until 1960, when the description of IPL-V was published (Newell and Tonge, 1960). The low level of the IPL languages prevented their widespread use. They were actually assembly languages for a hypothetical computer, implemented with an interpreter, in which

2. Logic Theorist discovered proofs for theorems in propositional calculus.

list-processing instructions were included. Another factor that kept the IPL languages from becoming popular was their implementation on the obscure Johnniac machine.

The contributions of the IPL languages were in their list design and their demonstration that list processing was feasible and useful.

IBM became interested in AI in the mid-1950s and chose theorem proving as a demonstration area. At the time, the Fortran project was still underway. The high cost of the Fortran I compiler convinced IBM that their list processing should be attached to Fortran, rather than in the form of a new language. Thus, the Fortran list processing language (FLPL) was designed and implemented as an extension to Fortran. FLPL was used to construct a theorem prover for plane geometry, which was then considered the easiest area for mechanical theorem proving.

2.4.2 Lisp Design Process

John McCarthy of MIT took a summer position at the IBM Information Research Department in 1958. His goal for the summer was to investigate symbolic computations and to develop a set of requirements for doing such computations. As a pilot example problem area, he chose differentiation of algebraic expressions. From this study came a list of language requirements. Among them were the control flow methods of mathematical functions: recursion and conditional expressions. The only available high-level language of the time, Fortran I, had neither of these.

Another requirement that grew from the symbolic-differentiation investigation was the need for dynamically allocated linked lists and some kind of implicit deallocation of abandoned lists. McCarthy simply would not allow his elegant algorithm for differentiation to be cluttered with explicit deallocation statements.

Because FLPL did not support recursion, conditional expressions, dynamic storage allocation, or implicit deallocation, it was clear to McCarthy that a new language was needed.

When McCarthy returned to MIT in the fall of 1958, he and Marvin Minsky formed the MIT AI Project, with funding from the Research Laboratory for Electronics at MIT. The first important effort of the project was to produce a software system for list processing. It was to be used initially to implement a program proposed by McCarthy called the Advice Taker.[3] This application became the impetus for the development of the list-processing language Lisp. The first version of Lisp is sometimes called "pure Lisp" because it is a purely functional language. In the following section, we describe the development of pure Lisp.

---

3. Advice Taker represented information with sentences written in a formal language and used a logical inferencing process to decide what to do.

2.4.3 Language Overview

2.4.3.1 Data Structures

Pure Lisp has only two kinds of data structures: atoms and lists. Atoms are either symbols, which have the form of identifiers, or numeric literals. The concept of storing symbolic information in linked lists is natural and was used in IPL-II. Such structures allow insertions and deletions at any point, operations that were then thought to be a necessary part of list processing. It was eventually determined, however, that Lisp programs rarely require these operations.

Lists are specified by delimiting their elements with parentheses. Simple lists, in which elements are restricted to atoms, have the form

```
(A B C D)
```

Nested list structures are also specified by parentheses. For example, the list

```
(A (B C) D (E (F G)))
```

is composed of four elements. The first is the atom `A`; the second is the sublist `(B C)`; the third is the atom `D`; the fourth is the sublist `(E (F G))`, which has as its second element the sublist `(F G)`.

Internally, lists are stored as single-linked list structures, in which each node has two pointers and represents a list element. A node containing an atom has its first pointer pointing to some representation of the atom, such as its symbol or numeric value, or a pointer to a sublist. A node for a sublist element has its first pointer pointing to the first node of the sublist. In both cases, the second pointer of a node points to the next element of the list. A list is referenced by a pointer to its first element.

The internal representations of the two lists shown earlier are depicted in Figure 2.2. Note that the elements of a list are shown horizontally. The last element of a list has no successor, so its link is `NIL`, which is represented in Figure 2.2 as a diagonal line in the element. Sublists are shown with the same structure.

2.4.3.2 Processes in Functional Programming

Lisp was designed as a functional programming language. All computation in a purely functional program is accomplished by applying functions to arguments. Neither the assignment statements nor the variables that abound in imperative language programs are necessary in functional language programs. Furthermore, repetitive processes can be specified with recursive function calls, making iteration (loops) unnecessary. These basic concepts of functional programming make it significantly different from programming in an imperative language.

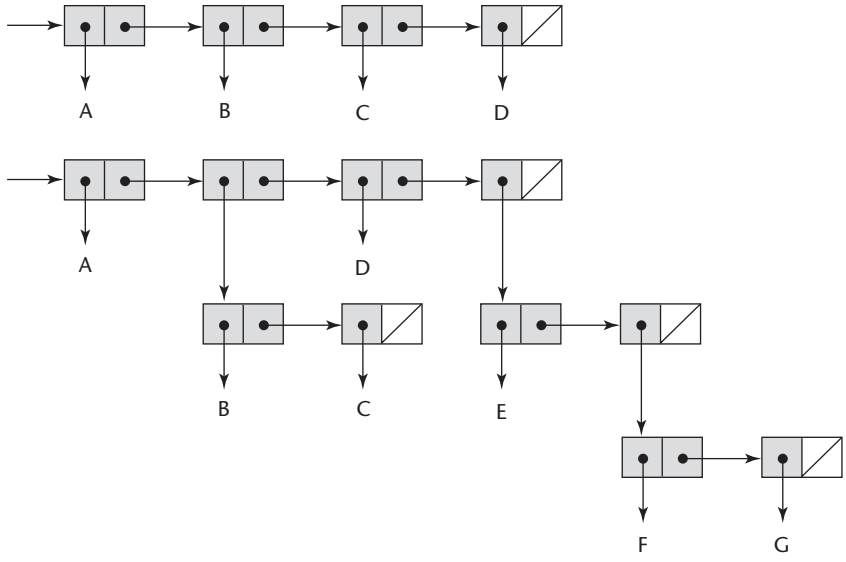


Figure 2.2
Internal representation of two Lisp lists

2.4.3.3 The Syntax of Lisp

Lisp is very different from the imperative languages, both because it is a functional programming language and because the appearance of Lisp programs is so different from those in languages like Java or C++. For example, the syntax of Java is a complicated mixture of English and algebra, while Lisp's syntax is a model of simplicity. Program code and data have exactly the same form: parenthesized lists. Consider again the list

```
(A B C D)
```

When interpreted as data, it is a list of four elements. When viewed as code, it is the application of the function named `A` to the three parameters `B`, `C`, and `D`.

2.4.4 Evaluation

Lisp completely dominated AI applications for a quarter century. Much of the cause of Lisp's reputation for being highly inefficient has been eliminated. Many contemporary implementations are compiled, and the resulting code is much faster than running the source code on an interpreter. In addition to its success in AI, Lisp pioneered functional programming, which has proven to be a lively area of research in programming languages. As stated in Chapter 1, many programming language researchers believe functional programming is a much better approach to software development than procedural programming using imperative languages.

The following is an example of a Lisp program:

```
; Lisp Example function
; The following code defines a Lisp predicate function
; that takes two lists as arguments and returns True
; if the two lists are equal, and NIL (false) otherwise
  (DEFUN equal_lists (lis1 lis2)
   (COND
     ((ATOM lis1) (EQ lis1 lis2))
     ((ATOM lis2) NIL)
     ((equal_lists (CAR lis1) (CAR lis2))
                 (equal_lists (CDR lis1) (CDR lis2)))
     (T NIL)
   )
)
```

2.4.5 Two Descendants of Lisp

Two dialects of Lisp are now widely used, Scheme and Common Lisp. These are briefly discussed in the following subsections.

2.4.5.1 Scheme

The Scheme language emerged from MIT in the mid-1970s. It is characterized by its small size, its exclusive use of static scoping (discussed in Chapter 5), and its treatment of functions as first-class entities. As first-class entities, Scheme functions can be assigned to variables, passed as parameters, and returned as the values of function applications. They can also be the elements of lists. Early versions of Lisp did not provide all of these capabilities, nor did they use static scoping.

As a small language with simple syntax and semantics, Scheme is well suited to educational applications, such as courses in functional programming and general introductions to programming. Scheme is described in some detail in Chapter 15.

2.4.5.2 Common Lisp

During the 1970s and early 1980s, a large number of different dialects of Lisp were developed and used. This led to the familiar problem of lack of portability among programs written in the various dialects. Common Lisp (Graham, 1996) was created in an effort to rectify this situation. Common Lisp was designed by combining the features of several dialects of Lisp developed in the early 1980s, including Scheme, into a single language. Being such an amalgam, Common Lisp is a relatively large and complex language. Its basis, however, is pure Lisp, so its syntax, primitive functions, and fundamental nature come from that language.

Recognizing the flexibility provided by dynamic scoping as well as the simplicity of static scoping, Common Lisp allows both. The default scoping for variables is static, but by declaring a variable to be `special`, that variable becomes dynamically scoped.

Common Lisp has a large number of data types and structures, including records, arrays, complex numbers, and character strings. It also has a form of packages for modularizing collections of functions and data providing access control.

Common Lisp is further described in Chapter 15.

2.4.6 Related Languages

ML (*M*eta*L*anguage; Ullman, 1998) was originally designed in the 1980s by Robin Milner at the University of Edinburgh as a metalanguage for a program verification system named Logic for Computable Functions (LCF; Milner et al., 1997). ML is primarily a functional language, but it also supports imperative programming. Unlike Lisp and Scheme, the type of every variable and expression in ML can be determined at compile time. Types are associated with objects rather than names. Types of names and expressions are inferred from their context.

Unlike Lisp and Scheme, ML does not use the parenthesized functional syntax that originated with lambda expressions. Rather, the syntax of ML resembles that of the imperative languages, such as Java and C++.

Miranda was developed by David Turner (1986) at the University of Kent in Canterbury, England, in the early 1980s. Miranda is based partly on the languages ML, SASL, and KRC. Haskell (Hudak and Fasel, 1992) is based in large part on Miranda. Like Miranda, it is a purely functional language, having no variables and no assignment statement. Another distinguishing characteristic of Haskell is its use of lazy evaluation. This means that no expression is evaluated until its value is required. This leads to some surprising capabilities in the language.

Caml (Cousineau et al., 1998) and its dialect that supports object-oriented programming, OCaml (Smith, 2006), descended from ML and Haskell. Finally, F# is a relatively new typed language based directly on OCaml. F# (Syme et al., 2010) is a .NET language with direct access to the whole .NET library. Being a .NET language also means it can smoothly interoperate with any other .NET language. F# supports both functional programming and procedural programming. It also fully supports object-oriented programming.

ML, Haskell, and F# are further discussed in Chapter 15.

2.5 The First Step Toward Sophistication: ALGOL 60

ALGOL 60 strongly influenced subsequent programming languages and is therefore of central importance in any historical study of languages.

2.5.1 Historical Background

ALGOL 60 was the result of efforts to design a universal programming language for scientific applications. By late 1954, the Laning and Zierler algebraic system had been in operation for over a year, and the first report on Fortran had been published. Fortran became a reality in 1957, and several other high-level languages were being developed. Most notable among them were IT, which was designed by Alan Perlis at Carnegie Tech, and two languages for the UNIVAC computers, MATH-MATIC and UNICODE. The proliferation of languages made program sharing among users difficult. Furthermore, the new languages were all growing up around single architectures, some for UNIVAC computers and some for IBM 700-series machines. In response to this blossoming of machine-dependent languages, several major computer user groups in the United States, including SHARE (the IBM scientific user group) and USE (UNIVAC Scientific Exchange, the large-scale UNIVAC scientific user group), submitted a petition to the Association for Computing Machinery (ACM) on May 10, 1957, to form a committee to study and recommend action to create a machine-independent scientific programming language. Although Fortran might have been a candidate, it could not become a universal language, because at the time it was solely owned by IBM.

Previously, in 1955, GAMM (a German acronym for Society for Applied Mathematics and Mechanics) had formed a committee to design one universal, machine-independent algorithmic language. The desire for this new language was in part due to the Europeans' fear of being dominated by IBM. By late 1957, however, the appearance of several high-level languages in the United States convinced the GAMM subcommittee that their effort had to be widened to include the Americans, and a letter of invitation was sent to ACM. In April 1958, after Fritz Bauer of GAMM presented the formal proposal to ACM, the two groups officially agreed to a joint language design project.

2.5.2 Early Design Process

GAMM and ACM each sent four members to the first design meeting. The meeting, which was held in Zurich from May 27 to June 1, 1958, began with the following goals for the new language:

- The syntax of the language should be as close as possible to standard mathematical notation, and programs written in it should be readable with little further explanation.
- It should be possible to use the language for the description of algorithms in printed publications.
- Programs in the new language must be mechanically translatable into machine language.

The first goal indicated that the new language was to be used for scientific programming, which was the primary computer application area at that time. The second was something entirely new to the computing business. The last goal is an obvious necessity for any programming language.

The Zurich meeting succeeded in producing a language that met the stated goals, but the design process required innumerable compromises, both among individuals and between the two sides of the Atlantic. In some cases, the compromises were not so much over great issues as they were over spheres of influence. The question of whether to use a comma (the European method) or a period (the American method) for a decimal point is one example.

2.5.3 ALGOL 58 Overview

The language designed at the Zurich meeting was named the International Algorithmic Language (IAL). It was suggested during the design that the language be named ALGOL, for ALGOrithmic Language, but the name was rejected because it did not reflect the international scope of the committee. During the following year, however, the name was changed to ALGOL, and the language subsequently became known as ALGOL 58.

In many ways, ALGOL 58 was a descendant of Fortran, which is quite natural. It generalized many of Fortran's features and added several new constructs and concepts. Some of the generalizations had to do with the goal of not tying the language to any particular machine, and others were attempts to make the language more flexible and powerful. A rare combination of simplicity and elegance emerged from the effort.

ALGOL 58 formalized the concept of data type, although only variables that were not floating-point required explicit declaration. It added the idea of compound statements, which most subsequent languages incorporated. Some features of Fortran that were generalized were the following: Identifiers were allowed to have any length, as opposed to Fortran I's restriction to six or fewer characters; any number of array dimensions was allowed, unlike Fortran I's limitation to no more than three; the lower bound of arrays could be specified by the programmer, whereas in Fortran it was implicitly 1; nested selection statements were allowed, which was not the case in Fortran I.

ALGOL 58 acquired the assignment operator in a rather unusual way. Zuse used the form

expression => variable

for the assignment statement in Plankalkül. Although Plankalkül had not yet been published, some of the European members of the ALGOL 58 committee were familiar with the language. The committee dabbled with the Plankalkül assignment form but, because of arguments about character set limitations,[4] the greater-than symbol was changed to a colon. Then, largely at the insistence of the Americans, the whole statement was turned around to the Fortran form

variable := expression

The Europeans preferred the opposite form, but that would be the reverse of Fortran.

4. The card punches of that time did not include the greater-than symbol.

2.5.4 Reception of the ALGOL 58 Report

In December 1958, publication of the ALGOL 58 report (Perlis and Samelson, 1958) was greeted with enthusiasm. In the United States, the new language was viewed more as a collection of ideas for programming language design than as a universal standard language. Actually, the ALGOL 58 report was not meant to be a finished product but rather a preliminary document for international discussion. Nevertheless, three major design and implementation efforts used the report as their basis. At the University of Michigan, the MAD language was born (Arden et al., 1961). The U.S. Naval Electronics Group produced the NELIAC language (Huskey et al., 1963). At System Development Corporation, JOVIAL was designed and implemented (Shaw, 1963). JOVIAL, an acronym for Jules' Own Version of the International Algebraic Language, represents the only language based on ALGOL 58 to achieve widespread use (Jules was Jules I. Schwartz, one of JOVIAL's designers). JOVIAL became widely used because it was the official scientific language for the U.S. Air Force for a quarter century.

The rest of the U.S. computing community was not so kind to the new language. At first, both IBM and its major scientific user group, SHARE, seemed to embrace ALGOL 58. IBM began an implementation shortly after the report was published, and SHARE formed a subcommittee, SHARE IAL, to study the language. The subcommittee subsequently recommended that ACM standardize ALGOL 58 and that IBM implement it for all of the 700-series computers. The enthusiasm was short-lived, however. By the spring of 1959, both IBM and SHARE, through their Fortran experience, had had enough of the pain and expense of getting a new language started, both in terms of developing and using the first-generation compilers and in terms of training users in the new language and persuading them to use it. By the middle of 1959, both IBM and SHARE had developed such a vested interest in Fortran that they decided to retain it as *the* scientific language for the IBM 700-series machines, thereby abandoning ALGOL 58.

2.5.5 ALGOL 60 Design Process

During 1959, ALGOL 58 was furiously debated in both Europe and the United States. Large numbers of suggested modifications and additions were published in the European *ALGOL Bulletin* and in *Communications of the ACM*. One of the most important events of 1959 was the presentation of the work of the Zurich committee to the International Conference on Information Processing, for there Backus introduced his new notation for describing the syntax of programming languages, which later became known as BNF (Backus-Naur form). BNF is described in detail in Chapter 3.

In January 1960, the second ALGOL meeting was held, this time in Paris. The purpose of the meeting was to debate the 80 suggestions that had been formally submitted for consideration. Peter Naur of Denmark had become heavily involved in the development of ALGOL, even though he had not been

a member of the Zurich group. It was Naur who created and published the *ALGOL Bulletin*. He spent a good deal of time studying Backus's paper that introduced BNF and decided that BNF should be used to describe formally the results of the 1960 meeting. After making a few relatively minor changes to BNF, he wrote a description of the new proposed language in BNF and handed it out to the members of the 1960 group at the beginning of the meeting.

2.5.6 ALGOL 60 Overview

Although the 1960 meeting lasted only six days, the modifications made to ALGOL 58 were dramatic. Among the most important new developments were the following:

- The concept of block structure was introduced. This allowed the programmer to localize parts of programs by introducing new data environments, or scopes.
- Two different means of passing parameters to subprograms were allowed: pass by value and pass by name.
- Procedures were allowed to be recursive. The ALGOL 58 description was unclear on this issue. Note that although recursion was new for the imperative languages, Lisp had already provided recursive functions in 1959.
- Stack-dynamic arrays were allowed. A stack-dynamic array is one for which the subscript range or ranges are specified by variables, so that the size of the array is set at the time storage is allocated to the array, which happens when the declaration is reached during execution. Stack-dynamic arrays are described in detail in Chapter 6.

Several features that might have had a dramatic impact on the success or failure of the language were proposed and rejected. Most important among these were input and output statements with formatting, which were omitted because they were thought to be machine-dependent.

The ALGOL 60 report was published in May 1960 (Naur, 1960). A number of ambiguities still remained in the language description, and a third meeting was scheduled for April 1962 in Rome to address the problems. At this meeting the group dealt only with problems; no additions to the language were allowed. The results of this meeting were published under the title "Revised Report on the Algorithmic Language ALGOL 60" (Backus et al., 1963).

2.5.7 Evaluation

In some ways, ALGOL 60 was a great success; in other ways, it was a dismal failure. It succeeded in becoming, almost immediately, the only acceptable formal means of communicating algorithms in computing literature, and it remained that for more than 20 years. Every imperative programming language designed since 1960 owes something to ALGOL 60. In fact, most are direct

or indirect descendants; examples include PL/I, SIMULA 67, ALGOL 68, C, Pascal, Ada, C++, Java, and C#.

The ALGOL 58/ALGOL 60 design effort included a long list of firsts. It was the first time that an international group attempted to design a programming language. It was the first language that was designed to be machine independent. It was also the first language whose syntax was formally described. This successful use of the BNF formalism initiated several important fields of computer science: formal languages, parsing theory, and BNF-based compiler design. Finally, the structure of ALGOL 60 affected machine architecture. In the most striking example of this, an extension of the language was used as the systems language of a series of large-scale computers, the Burroughs B5000, B6000, and B7000 machines, which were designed with a hardware stack to implement efficiently the block structure and recursive subprograms of the language.

On the other side of the coin, ALGOL 60 never achieved widespread use in the United States. Even in Europe, where it was more popular than in the United States, it never became the dominant language. There are a number of reasons for its lack of acceptance. For one thing, some of the features of ALGOL 60 turned out to be too flexible; they made understanding difficult and implementation inefficient. The best example of this is the pass-by-name method of passing parameters to subprograms, which is explained in Chapter 9. The difficulties of implementing ALGOL 60 are evidenced by Rutishauser's statement in 1967 that few, if any, implementations included the full ALGOL 60 language (Rutishauser, 1967, p. 8).

The lack of input and output statements in the language was another major reason for its lack of acceptance. Implementation-dependent input/output made programs difficult to port to other computers.

Ironically, one of the most important contributions to computer science associated with ALGOL 60, BNF, was also a factor in its lack of acceptance. Although BNF is now considered a simple and elegant means of syntax description, in 1960 it seemed strange and complicated.

Finally, although there were many other problems, the entrenchment of Fortran among users and the lack of support by IBM were probably the most important factors in ALGOL 60's failure to gain widespread use.

The ALGOL 60 effort was never really complete, in the sense that ambiguities and obscurities were always a part of the language description (Knuth, 1967).

The following is an example of an ALGOL 60 program:

```
comment ALGOL 60 Example Program
 Input:  An integer, listlen, where listlen is less than
         100, followed by listlen-integer values
 Output: The number of input values that are greater than
         the average of all the input values ;
begin
  integer array intlist [1:99];
  integer listlen, counter, sum, average, result;
```

```
  sum := 0;
  result := 0;
  readint (listlen);
  if (listlen > 0) ∧ (listlen < 100) then
    begin
comment Read input into an array and compute the average;
    for counter := 1 step 1 until listlen do
      begin
      readint (intlist[counter]);
      sum := sum + intlist[counter]
      end;
comment Compute the average;
    average := sum / listlen;
comment Count the input values that are > average;
    for counter := 1 step 1 until listlen do
      if intlist[counter] > average
        then result := result + 1;
comment Print result;
    printstring("The number of values > average is:");
    printint (result)
    end
  else
    printstring ("Error-input list length is not legal";
end
```

2.6 Computerizing Business Records: COBOL

The story of COBOL is, in a sense, the opposite of that of ALGOL 60. Although it has been used for nearly 60 years, COBOL has had little effect on the design of subsequent languages, except for PL/I. It may still be the most widely used language,[5] although it is difficult to be sure one way or the other. Perhaps the most important reason why COBOL has had little influence is that few have attempted to design a new language for business applications since it appeared. That is due in part to how well COBOL's capabilities meet the needs of its application area. Another reason is that a great deal of growth in business computing over the past 30 years has occurred in small businesses. In these businesses, very little software development has taken place. Instead, most of the software used is purchased as off-the-shelf packages for various general business applications.

5. In the late 1990s, in a study associated with the Y2K problem, it was estimated that there were approximately 800 million lines of COBOL in regular use in the 22 square miles of Manhattan.

2.6.1 Historical Background

The beginning of COBOL is somewhat similar to that of ALGOL 60, in the sense that the language was designed by a committee of people meeting for relatively short periods of time. At the time, in 1959, the state of business computing was similar to the state of scientific computing several years earlier, when Fortran was being designed. One compiled language for business applications, FLOW-MATIC, had been implemented in 1957, but it belonged to one manufacturer, UNIVAC, and was designed for that company's computers. Another language, AIMACO, was being used by the U.S. Air Force, but it was only a minor variation of FLOW-MATIC. IBM had designed a programming language for business applications, COMTRAN (COMmercial TRANslator), but it had not yet been implemented. Several other language design projects were being planned.

2.6.2 FLOW-MATIC

The origins of FLOW-MATIC are worth at least a brief discussion, because it was the primary progenitor of COBOL. In December 1953, Grace Hopper at Remington-Rand UNIVAC wrote a proposal that was indeed prophetic. It suggested that "mathematical programs should be written in mathematical notation, data processing programs should be written in English statements" (Wexelblat, 1981, p. 16). Unfortunately, in 1953, it was impossible to convince nonprogrammers that a computer could be made to understand English words. It was not until 1955 that a similar proposal had some hope of being funded by UNIVAC management, and even then it took a prototype system to do the final convincing. Part of this selling process involved compiling and running a small program, first using English keywords, then using French keywords, and then using German keywords. This demonstration was considered remarkable by UNIVAC management and was instrumental in their acceptance of Hopper's proposal.

2.6.3 COBOL Design Process

The first formal meeting on the subject of a common language for business applications, which was sponsored by the Department of Defense, was held at the Pentagon on May 28 and 29, 1959 (exactly one year after the Zurich ALGOL meeting). The consensus of the group was that the language, then named CBL (Common Business Language), should have the following general characteristics: Most agreed that it should use English as much as possible, although a few argued for a more mathematical notation. The language must be easy to use, even at the expense of being less powerful, in order to broaden the base of those who could program computers. In addition to making the language easy to use, it was believed that the use of English would allow managers to read programs. Finally, the design should not be overly restricted by the problems of its implementation.

One of the overriding concerns at the meeting was that steps to create this universal language should be taken quickly, as a lot of work was already being done to create other business languages. In addition to the existing languages, RCA and Sylvania were working on their own business applications languages. It was clear that the longer it took to produce a universal language, the more difficult it would be for the language to become widely used. On this basis, it was decided that there should be a quick study of existing languages. For this task, the Short Range Committee was formed.

There were early decisions to separate the statements of the language into two categories—data description and executable operations—and to have statements in these two categories be in different parts of programs. One of the debates of the Short Range Committee was over the inclusion of subscripts. Many committee members argued that subscripts were too complex for the people in data processing, who were thought to be uncomfortable with mathematical notation. Similar arguments revolved around whether arithmetic expressions should be included. The final report of the Short Range Committee, which was completed in December 1959, described the language that was later named COBOL 60.

The language specification for COBOL 60, published by the Government Printing Office in April 1960 (Department of Defense, 1960), was described as "initial." Revised versions were published in 1961 and 1962 (Department of Defense, 1961, 1962). The language was standardized by the American National Standards Institute (ANSI) group in 1968. The next three revisions were standardized by ANSI in 1974, 1985, and 2002. The language continues to evolve today.

2.6.4 Evaluation

The COBOL language originated a number of novel concepts, some of which eventually appeared in other languages. For example, the `DEFINE` verb of COBOL 60 was the first high-level language construct for macros. More important, hierarchical data structures (records), which first appeared in Plankalkül, were first implemented in COBOL. They have been included in most of the imperative languages designed since then. COBOL was also the first language that allowed names to be truly connotative, because it allowed both long names (up to 30 characters) and word-connector characters (hyphens).

Overall, the data division is the strong part of COBOL's design, whereas the procedure division is relatively weak. Every variable is defined in detail in the data division, including the number of decimal digits and the location of the implied decimal point. File records are also described with this level of detail, as are lines to be output to a printer, which makes COBOL ideal for printing accounting reports. Perhaps the most important weakness of the original procedure division was in its lack of functions. Versions of COBOL prior to the 1974 standard also did not allow subprograms with parameters.

Our final comment on COBOL: It was the first programming language whose use was mandated by the Department of Defense (DoD). This mandate

came after its initial development, because COBOL was not designed specifically for the DoD. In spite of its merits, COBOL probably would not have survived without that mandate. The poor performance of the early compilers simply made the language too expensive to use. Eventually, of course, compilers became more efficient and computers became much faster and cheaper and had much larger memories. Together, these factors allowed COBOL to succeed, inside and outside DoD. Its appearance led to the electronic mechanization of accounting, an important development by any measure.

The following is an example of a COBOL program. This program reads a file named `BAL-FWD-File` that contains inventory information about a certain collection of items. Among other things, each item record includes the number currently on hand (`BAL-ON-HAND`) and the item's reorder point (`BAL-REORDER-POINT`). The reorder point is the threshold number of items on hand at which more must be ordered. The program produces a list of items that must be reordered as a file named `REORDER-LISTING`.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. PRODUCE-REORDER-LISTING.

ENVIRONMENT DIVISION.
CONFIGURATION SECTION.
SOURCE-COMPUTER. DEC-VAX.
OBJECT-COMPUTER. DEC-VAX.
INPUT-OUTPUT SECTION.
FILE-CONTROL.
    SELECT BAL-FWD-FILE ASSIGN TO READER.
    SELECT REORDER-LISTING ASSIGN TO LOCAL-PRINTER.

DATA DIVISION.
FILE SECTION.
FD  BAL-FWD-FILE
    LABEL RECORDS ARE STANDARD
    RECORD CONTAINS 80 CHARACTERS.

01  BAL-FWD-CARD.
    02 BAL-ITEM-NO          PICTURE IS 9(5).
    02 BAL-ITEM-DESC        PICTURE IS X(20).
    02 FILLER               PICTURE IS X(5).
    02 BAL-UNIT-PRICE       PICTURE IS 999V99.
    02 BAL-REORDER-POINT    PICTURE IS 9(5).
    02 BAL-ON-HAND          PICTURE IS 9(5).
    02 BAL-ON-ORDER         PICTURE IS 9(5).
    02 FILLER               PICTURE IS X(30).
FD  REORDER-LISTING
    LABEL RECORDS ARE STANDARD
    RECORD CONTAINS 132 CHARACTERS.

01  REORDER-LINE.
```

```
    02 RL-ITEM-NO           PICTURE IS Z(5).
    02 FILLER               PICTURE IS X(5).
    02 RL-ITEM-DESC         PICTURE IS X(20).
    02 FILLER               PICTURE IS X(5).
    02 RL-UNIT-PRICE        PICTURE IS ZZZ.99.
    02 FILLER               PICTURE IS X(5).
    02 RL-AVAILABLE-STOCK   PICTURE IS Z(5).
    02 FILLER               PICTURE IS X(5).
    02 RL-REORDER-POINT     PICTURE IS Z(5).
    02 FILLER               PICTURE IS X(71).

WORKING-STORAGE SECTION.
01  SWITCHES.
    02 CARD-EOF-SWITCH      PICTURE IS X.
01  WORK-FIELDS.
    02 AVAILABLE-STOCK      PICTURE IS 9(5).

PROCEDURE DIVISION.
000-PRODUCE-REORDER-LISTING.
    OPEN INPUT BAL-FWD-FILE.
    OPEN OUTPUT REORDER-LISTING.
    MOVE "N" TO CARD-EOF-SWITCH.
    PERFORM 100-PRODUCE-REORDER-LINE
        UNTIL CARD-EOF-SWITCH IS EQUAL TO "Y".
    CLOSE BAL-FWD-File.
    CLOSE REORDER-LISTING.
    STOP RUN.

100-PRODUCE-REORDER-LINE.
    PERFORM 110-READ-INVENTORY-RECORD.
    IF CARD-EOF-SWITCH IS NOT EQUAL TO "Y"]
        PERFORM 120-CALCULATE-AVAILABLE-STOCK
        IF AVAILABLE-STOCK IS LESS THAN BAL-REORDER-POINT
            PERFORM 130-PRINT-REORDER-LINE.

110-READ-INVENTORY-RECORD.
    READ BAL-FWD-FILE RECORD
         AT END
             MOVE "Y" TO CARD-EOF-SWITCH.

120-CALCULATE-AVAILABLE-STOCK.
ADD BAL-ON-HAND BAL-ON-ORDER
    GIVING AVAILABLE-STOCK.

130-PRINT-REORDER-LINE.
    MOVE SPACE              TO REORDER-LINE.
    MOVE BAL-ITEM-NO        TO RL-ITEM-NO.
    MOVE BAL-ITEM-DESC      TO RL-ITEM-DESC.
    MOVE BAL-UNIT-PRICE     TO RL-UNIT-PRICE.
```

```
MOVE AVAILABLE-STOCK     TO RL-AVAILABLE-STOCK.
MOVE BAL-REORDER-POINT   TO RL-REORDER-POINT.
WRITE REORDER-LINE.
```

2.7 The Beginnings of Timesharing: Basic

Basic (Mather and Waite, 1971) is another programming language that has enjoyed widespread use but has gotten little respect. Like COBOL, it has largely been ignored by computer scientists. Also, like COBOL, in its earliest versions it was inelegant and included only a meager set of control statements.

Basic was very popular on microcomputers in the late 1970s and early 1980s. This followed directly from two of the main characteristics of early versions of Basic. It was easy for beginners to learn, especially those who were not science oriented, and its smaller dialects could be implemented on computers with very small memories.[6] When the capabilities of microcomputers grew and other languages were implemented, the use of Basic waned. A strong resurgence in the use of Basic began with the appearance of Visual Basic (Microsoft, 1991) in the early 1990s.

2.7.1 Design Process

Basic (Beginner's All-purpose Symbolic Instruction Code) was originally designed at Dartmouth College (now Dartmouth University) in New Hampshire by two mathematicians, John Kemeny and Thomas Kurtz, who, in the early 1960s, developed compilers for a variety of dialects of Fortran and ALGOL 60. Their science students generally had little trouble learning or using those languages in their studies. However, Dartmouth was primarily a liberal arts institution, where science and engineering students made up only about 25 percent of the student body. It was decided in the spring of 1963 to design a new language especially for liberal arts students. This new language would use terminals as the method of computer access. The goals of the system were as follows:

1. It must be easy for nonscience students to learn and use.
2. It must be "pleasant and friendly."
3. It must provide fast turnaround for homework.
4. It must allow free and private access.
5. It must consider user time more important than computer time.

6. Some early microcomputers included Basic interpreters that resided in 4096 bytes of ROM.

The last goal was indeed a revolutionary concept. It was based at least partly on the belief that computers would become significantly cheaper as time went on, which they certainly did.

The combination of the second, third, and fourth goals led to the time-shared aspect of Basic. Only with individual access through terminals by numerous simultaneous users could these goals be met in the early 1960s.

In the summer of 1963, Kemeny began work on the compiler for the first version of Basic, using remote access to a GE 225 computer. Design and coding of the operating system for Basic began in the fall of 1963. At 4:00 A.M. on May 1, 1964, the first program using the timeshared Basic was typed in and run. In June, the number of terminals on the system grew to 11, and by the fall it had ballooned to 20.

2.7.2 Language Overview

The original version of Basic was very small and, oddly, was not interactive: There was no way for an executing program to get input data from the user. Programs were typed in, compiled, and run, in a sort of batch-oriented way. The original Basic had only 14 different statement types and a single data type—floating-point. Because it was believed that few of the targeted users would appreciate the difference between integer and floating-point types, the type was referred to as "numbers." Overall, it was a very limited language, though quite easy to learn.

2.7.3 Evaluation

The most important aspect of the original Basic was that it was the first widely used language that was used through terminals connected to a remote computer.[7] Terminals had just begun to be available at that time. Before then, most programs were entered into computers through either punched cards or paper tape.

Much of the design of Basic came from Fortran, with some minor influence from the syntax of ALGOL 60. Later, it grew in a variety of ways, with little or no effort made to standardize it. The American National Standards Institute issued a Minimal Basic standard (ANSI, 1978b), but this represented only the bare minimum of language features. In fact, the original Basic was very similar to Minimal Basic.

Although it may seem surprising, Digital Equipment Corporation used a rather elaborate version of Basic named Basic-PLUS to write significant

7. Lisp initially was used through terminals, but it was not widely used in the early 1960s.

portions of their largest operating system for the PDP-11 minicomputers, RSTS, in the 1970s.

Basic has been criticized for the poor structure of programs written in it, among other things. By the evaluation criteria discussed in Chapter 1, specifically readability and reliability, the language does indeed fare very poorly. Clearly, the early versions of the language were not meant for and should not have been used for serious programs of any significant size. Later versions are much better suited to such tasks.

The resurgence of Basic in the 1990s was driven by the appearance of Visual Basic (VB). VB became widely used in large part because it provided a simple way of building graphical user interfaces (GUIs), hence the name Visual Basic. When .NET appeared, a new version of VB came with it, VB.NET. Although it was a significant departure from earlier versions of VB, it quickly displaced the older language. Perhaps the most important difference between VB and the .NET version is that the later version fully supports object-oriented programming.

The following is an example of a Basic program:

```
REM  Basic Example Program
REM  Input:  An integer, listlen, where listlen is less
REM          than 100, followed by listlen-integer values
REM  Output: The number of input values that are greater
REM          than the average of all input values
  DIM intlist(99)
  result = 0
  sum = 0
  INPUT listlen
  IF listlen > 0 AND listlen < 100 THEN
REM  Read input into an array and compute the sum
   FOR counter = 1 TO listlen
     INPUT intlist(counter)
     sum = sum + intlist(counter)
   NEXT counter
REM  Compute the average
   average = sum / listlen
REM  Count the number of input values that are > average
   FOR counter = 1 TO listlen
     IF intlist(counter) > average
        THEN result = result + 1
   NEXT counter
REM  Print the result
    PRINT "The number of values that are > average is:";
          result
  ELSE
   PRINT "Error-input list length is not legal"
  END IF
END
```

User Design and Language Design

ALAN COOPER

Best-selling author of *About Face: The Essentials of User Interface Design*, Alan Cooper also had a large hand in designing what can be touted as the language with the most concern for user interface design, Visual Basic. For him, it all comes down to a vision for humanizing technology.

SOME INFORMATION ON THE BASICS

How did you get started in all of this? I'm a high school dropout with an associate degree in programming from a California community college. My first job was as a programmer for American President Lines (one of the United States' oldest ocean transportation companies) in San Francisco. Except for a few months here and there, I've remained self-employed.

What is your current job? Founder and chairman of Cooper, the company that humanizes technology (www.cooper.com).

What is or was your favorite job? Interaction design consultant.

You are very well known in the fields of language design and user interface design. Any thoughts on designing languages versus designing software, versus designing anything else? It's pretty much the same in the world of software: Know your user.

ABOUT THAT EARLY WINDOWS RELEASE

In the 1980s, you started using Windows and have talked about being lured by its plusses: the graphical user interface support and the dynamically linked library that let you create tools that configured themselves. What about the parts of Windows that you eventually helped shape? I was very impressed by Microsoft's inclusion of support for practical multitasking in Windows. This included dynamic relocation and interprocess communications. MSDOS.exe was the shell program for the first few releases of Windows. It was a terrible program, and I believed that it could be improved dramatically, and I was the guy to do it. In my spare time, I immediately began to write a better shell program than the one Windows came with. I called it Tripod. Microsoft's original shell, MSDOS.exe, was one of the main stumbling blocks to the initial success of Windows. Tripod attempted to solve the problem by being easier to use and to configure.

When was that "Aha!" moment? It wasn't until late in 1987, when I was interviewing a corporate client, that the key design strategy for Tripod popped into my head. As the IS manager explained to me his need to create and publish a wide range of shell solutions to his disparate user base, I realized the conundrum that there is no such thing as an ideal shell. Every user would need their own personal shell, configured to their own needs and skill levels. In an instant, I perceived the solution to the shell design problem: It would be a shell construction set; a tool where each user would be able to construct exactly the shell that he or she needed for a unique mix of applications and training.

What is so compelling about the idea of a shell that can be individualized? Instead of me telling the users what the ideal shell was, they could design their own, personalized ideal shell. With a customizable shell, a programmer would create a shell that was powerful and wide ranging but also somewhat dangerous, whereas an IT manager would create a shell that could be given to a desk clerk that exposed only those few application-specific tools that the clerk used.

> *"MSDOS.exe was the shell program for the first few releases of Windows. It was a terrible program, and I believed that it could be improved dramatically, and I was the guy to do it. In my spare time, I immediately began to write a better shell program than the one Windows came with."*

How did you get from writing a shell program to collaborating with Microsoft? Tripod and Ruby are the same thing. After I signed a deal with Bill Gates, I changed the name of the prototype from Tripod to Ruby. I then used the Ruby prototype as prototypes should be used: as a disposable model for constructing release-quality code. Which is what I did. MS took the release version of Ruby and added QuickBasic to it, creating VB. All of those original innovations were in Tripod/Ruby.

RUBY AS THE INCUBATOR FOR VISUAL BASIC

Let's revisit your interest in early Windows and that DLL feature. The DLL wasn't a thing, it was a facility in the OS. It allowed a programmer to build code objects that could be linked to at run time as opposed to only at compile time. This is what allowed me to invent the dynamically extensible parts of VB, where controls can be added by third-party vendors.

The Ruby product embodied many significant advances in software design, but two of them stand out as exceptionally successful. As I mentioned, the dynamic linking capability of Windows had always intrigued me, but having the tools and knowing what to do with them were two different things. With Ruby, I finally found two practical uses for dynamic linking, and the original program contained both. First, the language was both installable and could be extended dynamically. Second, the palette of gizmos could be added to dynamically.

Was your language in Ruby the first to have a dynamic linked library and to be linked to a visual front end? As far as I know, yes.

Using a simple example, what would this enable a programmer to do with his or her program? Purchase a control, such as a grid control, from a third-party vendor, install it on his or her computer, and have the grid control appear as an integral part of the language, including the visual programming front end.

Why do they call you "the father of Visual Basic"? Ruby came with a small language, one suited only for executing the dozen or so simple commands that a shell program needs. However, this language was implemented as a chain of DLLs, any number of which could be installed at run time. The internal parser would identify a verb and then pass it along the chain of DLLs until one of them acknowledged that it knew how to process the verb. If all of the DLLs passed, there was a syntax error. From our earliest discussions, both Microsoft and I had entertained the idea of growing the language, possibly even replacing it altogether with a "real" language. C was the candidate most frequently mentioned, but eventually, Microsoft took advantage of this dynamic interface to unplug our little shell language and replace it entirely with Quick-Basic. This new marriage of language to visual front end was static and permanent, and although the original dynamic interface made the coupling possible, it was lost in the process.

SOME FINAL COMMENTS ON NEW IDEAS

In the world of programming and programming tools, including languages and environments, what projects most interest you? I'm interested in creating programming tools that are designed to help users instead of programmers.

What's the most critical rule, famous quote, or design idea to keep in mind? Bridges are not built by engineers. They are built by ironworkers.

Similarly, software programs are not built by engineers. They are built by programmers.

2.8 Everything for Everybody: PL/I

PL/I represents the first large-scale attempt to design a language that could be used for a broad spectrum of application areas. All previous and most subsequent languages have focused on one particular application area, such as science, artificial intelligence, or business.

2.8.1 Historical Background

Like Fortran, PL/I was developed as an IBM product. By the early 1960s, the users of computers in industry had settled into two separate and quite different camps: scientific and business. From the IBM point of view, scientific programmers could use either the large-scale 7090 or the small-scale 1620 IBM computers. This group used floating-point data and arrays extensively. Fortran was the primary language, although some assembly language also was used. They had their own user group, SHARE, and had little contact with anyone who worked on business applications.

For business applications, people used the large 7080 or the small 1401 IBM computers. They needed the decimal and character string data types, as well as elaborate and efficient input and output facilities. They used COBOL, although in early 1963 when the PL/I story begins, the conversion from assembly language to COBOL was far from complete. This category of users also had its own user group, GUIDE, and seldom had contact with scientific users.

In early 1963, IBM planners perceived the beginnings of a change in this situation. The two widely separated computer user groups were moving toward each other in ways that were thought certain to create problems. Scientists began to gather large files of data to be processed. This data required more sophisticated and more efficient input and output facilities. Business applications people began to use regression analysis to build management information systems, which required floating-point data and arrays. It began to appear that computing installations would soon require two separate computers and technical staffs, supporting two very different programming languages.[8]

These perceptions naturally led to the concept of designing a single universal computer that would be capable of doing both floating-point and decimal arithmetic, and therefore both scientific and business applications. Thus was born the concept of the IBM System/360 line of computers. Along with this came the idea of a programming language that could be used for both business and scientific applications. For good measure, features to support systems programming and list processing were thrown in. Therefore, the new language was to replace Fortran, COBOL, Lisp, and the systems applications of assembly language.

8. At the time, large computer installations required both full-time hardware and full-time system software maintenance staff.

2.8.2 Design Process

The design effort began when IBM and SHARE formed the Advanced Language Development Committee of the SHARE Fortran Project in October 1963. This new committee quickly met and formed a subcommittee called the 3×3 Committee, so named because it had three members from IBM and three from SHARE. The 3×3 Committee met for three or four days every other week to design the language.

As with the Short Range Committee for COBOL, the initial design was scheduled for completion in a remarkably short time. Apparently, regardless of the scope of a language design effort, in the early 1960s the prevailing belief was that it could be done in three months. The first version of PL/I, which was then named Fortran VI, was supposed to be completed by December, less than three months after the committee was formed. The committee pleaded successfully on two different occasions for extensions, moving the due date back to January and then to late February 1964.

The initial design concept was that the new language would be an extension of Fortran IV, maintaining compatibility, but that goal was dropped quickly along with the name Fortran VI. Until 1965, the language was known as NPL (New Programming Language). The first published report on NPL was given at the SHARE meeting in March 1964. A more complete description followed in April, and the version that would actually be implemented was published in December 1964 (IBM, 1964) by the compiler group at the IBM Hursley Laboratory in England, which was chosen to do the implementation. In 1965, the name was changed to PL/I to avoid the confusion of the name NPL with the National Physical Laboratory in England. If the compiler had been developed outside the United Kingdom, the name might have remained NPL.

2.8.3 Language Overview

Perhaps the best single-sentence description of PL/I is that it included what were then considered the best parts of ALGOL 60 (recursion and block structure), Fortran IV (separate compilation with communication through global data), and COBOL 60 (data structures, input/output, and report-generating facilities), along with an extensive collection of new constructs, all somehow cobbled together. Because PL/I is no longer a popular language, we will not attempt, even briefly, to discuss all the features of the language, or even its most controversial constructs. Instead, we will mention some of the language's contributions to the pool of knowledge of programming languages.

PL/I was the first programming language to have the following facilities:

- Programs were allowed to create concurrently executing subprograms. Although this was a good idea, it was poorly developed in PL/I.
- It was possible to detect and handle 23 different types of exceptions, or run-time errors.

- Subprograms were allowed to be used recursively, but the capability could be disabled, allowing more efficient linkage for nonrecursive subprograms.
- Pointers were included as a data type.
- Cross-sections of arrays could be referenced. For example, the third row of a matrix could be referenced as if it were a single-dimensioned array.

2.8.4 Evaluation

Any evaluation of PL/I must begin by recognizing the ambitiousness of the design effort. In retrospect, it appears naive to think that so many constructs could have been combined successfully. However, that judgment must be tempered by acknowledging that there was little language design experience at the time. Overall, the design of PL/I was based on the premise that any construct that was useful and could be implemented should be included, with insufficient concern about how a programmer could understand and make effective use of such a collection of constructs and features. Edsger Dijkstra, in his Turing Award Lecture (Dijkstra, 1972), made one of the strongest criticisms of the complexity of PL/I: "I absolutely fail to see how we can keep our growing programs firmly within our intellectual grip when by its sheer baroqueness the programming language—our basic tool, mind you!—already escapes our intellectual control."

In addition to the problem with the complexity due to its large size, PL/I suffered from a number of what are now considered to be poorly designed constructs. Among these were pointers, exception handling, and concurrency, although we must point out that in all cases, these constructs had not appeared in any previous language.

In terms of usage, PL/I must be considered at least a partial success. In the 1970s, it enjoyed significant use in both business and scientific applications. It was also widely used during that time as an instructional vehicle in colleges, primarily in several subset forms, such as PL/C (Cornell, 1977) and PL/CS (Conway and Constable, 1976).

The following is an example of a PL/I program:

```
/* PL/I PROGRAM EXAMPLE
 INPUT:   AN INTEGER, LISTLEN, WHERE LISTLEN IS LESS THAN
          100, FOLLOWED BY LISTLEN-INTEGER VALUES
 OUTPUT:  THE NUMBER OF INPUT VALUES THAT ARE GREATER THAN
          THE AVERAGE OF ALL INPUT VALUES */
PLIEX: PROCEDURE OPTIONS (MAIN);
  DECLARE INTLIST (1:99) FIXED.
  DECLARE (LISTLEN, COUNTER, SUM, AVERAGE, RESULT) FIXED;
  SUM = 0;
  RESULT = 0;
  GET LIST (LISTLEN);
  IF (LISTLEN > 0) & (LISTLEN < 100) THEN
```

```
    DO;
/* READ INPUT DATA INTO AN ARRAY AND COMPUTE THE SUM */
    DO COUNTER = 1 TO LISTLEN;
      GET LIST (INTLIST (COUNTER));
      SUM = SUM + INTLIST (COUNTER);
    END;
/* COMPUTE THE AVERAGE */
    AVERAGE = SUM / LISTLEN;
/* COUNT THE NUMBER OF VALUES THAT ARE > AVERAGE */
    DO COUNTER = 1 TO LISTLEN;
      IF INTLIST (COUNTER) > AVERAGE THEN
        RESULT = RESULT + 1;
    END;
/* PRINT RESULT */
    PUT SKIP LIST ('THE NUMBER OF VALUES > AVERAGE IS:');
    PUT LIST (RESULT);
    END;
ELSE
    PUT SKIP LIST ('ERROR-INPUT LIST LENGTH IS ILLEGAL');
END PLIEX;
```

2.9 Two Early Dynamic Languages: APL and SNOBOL

The structure of this section is different from that of the other sections because the languages discussed here are very different. Neither APL nor SNOBOL had much influence on later mainstream languages.[9] Some of the interesting features of APL are discussed later in the book.

In appearance and in purpose, APL and SNOBOL are quite different. They share two fundamental characteristics, however: dynamic typing and dynamic storage allocation. Variables in both languages are essentially untyped. A variable acquires a type when it is assigned a value, at which time it assumes the type of the value assigned. Storage is allocated to a variable only when it is assigned a value, because before that there is no way to know the amount of storage that will be needed.

2.9.1 Origins and Characteristics of APL

APL (Brown et al., 1988) was designed around 1960 by Kenneth E. Iverson at IBM. It was not originally designed to be an implemented programming language but rather was intended to be a vehicle for describing computer architecture.

9. However, they have had some influence on some nonmainstream languages (J is based on APL, ICON is based on SNOBOL, and AWK is partially based on SNOBOL).

APL was first described in the book from which it gets its name, *A Programming Language* (Iverson, 1962). In the mid-1960s, the first implementation of APL was developed at IBM.

APL has a large number of powerful operators that are specified with a large number of symbols, which created a problem for implementors. Initially, APL was used through IBM printing terminals. These terminals had special optional print balls that provided the odd character set required by the language. One reason APL has so many operators is that it provides a large number of unit operations on arrays. For example, the transpose of any matrix is done with a single operator. The large collection of operators provides very high expressivity but also makes APL programs difficult to read. Therefore, people think of APL as a language that is best used for "throw-away" programming. Although programs can be written quickly, they should be discarded after use because they are difficult to maintain.

APL has been around for over 55 years and is still used today, although not widely. Furthermore, it has not changed a great deal over its lifetime.

2.9.2 Origins and Characteristics of SNOBOL

SNOBOL (pronounced "snowball"; Griswold et al., 1971) was designed in the early 1960s by three people at Bell Laboratories: D. J. Farber, R. E. Griswold, and I. P. Polonsky (Farber et al., 1964). It was designed specifically for text processing. The heart of SNOBOL is a collection of powerful operations for string pattern matching. One of the early applications of SNOBOL was for writing text editors. Because the dynamic nature of SNOBOL makes it slower than alternative languages, it is no longer used for such programs. However, SNOBOL is still a live and supported language that is used for a variety of text-processing tasks in several different application areas.

2.10 The Beginnings of Data Abstraction: SIMULA 67

Although SIMULA 67 never achieved widespread use and had little impact on the programmers and computing of its time, some of the constructs it introduced make it historically important.

2.10.1 Design Process

Two Norwegians, Kristen Nygaard and Ole-Johan Dahl, developed the language SIMULA I between 1962 and 1964 at the Norwegian Computing Center (NCC) in Oslo. They were primarily interested in using computers for simulation but also worked in operations research. SIMULA I was designed exclusively for system simulation and was first implemented in late 1964 on a UNIVAC 1107 computer.

As soon as the SIMULA I implementation was completed, Nygaard and Dahl began efforts to extend the language by adding new features and modifying some existing constructs in order to make the language useful for general-purpose applications. The result of this work was SIMULA 67, whose design was first presented publicly in March 1967 (Dahl and Nygaard, 1967). We will discuss only SIMULA 67, although some of the features of interest in SIMULA 67 are also in SIMULA I.

2.10.2 Language Overview

SIMULA 67 is an extension of ALGOL 60, taking both block structure and the control statements from that language. The primary deficiency of ALGOL 60 (and other languages at that time) for simulation applications was the design of its subprograms. Simulation requires subprograms that are allowed to restart at the position where they previously stopped. Subprograms with this kind of control are known as **coroutines** because the caller and called subprograms have a somewhat equal relationship with each other, rather than the rigid master/slave relationship they have in most imperative languages.

To provide support for coroutines in SIMULA 67, the class construct was developed. This was an important development because the concept of data abstraction began with it and data abstraction provides the foundation for object-oriented programming.

It is interesting to note that the important concept of data abstraction was not developed and attributed to the class construct until 1972, when Hoare (1972) recognized the connection.

2.11 Orthogonal Design: ALGOL 68

ALGOL 68 was the source of several new ideas in language design, some of which were subsequently adopted by other languages. We include it here for that reason, even though it never achieved widespread use in either Europe or the United States.

2.11.1 Design Process

The development of the ALGOL family did not end when the revised report on ALGOL 60 appeared in 1962, although it was six years until the next design iteration was published. The resulting language, ALGOL 68 (van Wijngaarden et al., 1969), was dramatically different from its predecessor.

One of the most interesting innovations of ALGOL 68 was one of its primary design criteria: orthogonality. Recall our discussion of orthogonality in Chapter 1. The use of orthogonality resulted in several innovative features of ALGOL 68, one of which is described in the following section.

2.11.2 Language Overview

One important result of orthogonality in ALGOL 68 was its inclusion of user-defined data types. Earlier languages, such as Fortran, included only a few basic data structures. PL/I included a larger number of data structures, which made it harder to learn and difficult to implement, but it still could not provide an appropriate data structure for every need.

The approach of ALGOL 68 to data structures was to provide a few primitive types and structures and allow the user to combine those primitives to define a large number of different structures. This provision for user-defined data types was carried over to some extent into all of the major imperative languages designed since then. User-defined data types are valuable because they allow the user to design data abstractions that fit particular problems very closely. All aspects of data types are discussed in Chapter 6.

As another first in the area of data types, ALGOL 68 introduced the kind of dynamic arrays that will be termed *implicit heap-dynamic* in Chapter 5. A dynamic array is one in which the declaration does not specify subscript bounds. Assignments to a dynamic array cause allocation of required storage. In ALGOL 68, dynamic arrays are called **`flex`** arrays.

2.11.3 Evaluation

ALGOL 68 includes a significant number of features that had not been previously used. Its use of orthogonality, which some may argue was overdone, was nevertheless revolutionary.

ALGOL 68 repeated one of the sins of ALGOL 60, however, and it was an important factor in its limited popularity. The language was described using an elegant and concise but also unknown metalanguage. Before one could read the language-describing document (van Wijngaarden et al., 1969), he or she had to learn the new metalanguage, called van Wijngaarden grammars, which were far more complex than BNF. To make matters worse, the designers invented a collection of words to explain the grammar and the language. For example, keywords were called *indicants*, substring extraction was called *trimming*, and the process of a subprogram execution was called a *coercion of deproceduring*, which might be *meek*, *firm*, or something else.

It is natural to contrast the design of PL/I with that of ALGOL 68, because they appeared only a few years apart. ALGOL 68 achieved writability by the principle of orthogonality: a few primitive concepts and the unrestricted use of a few combining mechanisms. PL/I achieved writability by including a large number of fixed constructs. ALGOL 68 extended the elegant simplicity of ALGOL 60, whereas PL/I simply threw together the features of several languages to attain its goals. Of course, it must be remembered that the goal of PL/I was to provide a unified tool for a broad class of problems, whereas ALGOL 68 was targeted to a single class: scientific applications.

PL/I achieved far greater acceptance than ALGOL 68, due largely to IBM's promotional efforts and the problems of understanding and implementing

ALGOL 68. Implementation was a difficult problem for both, but PL/I had the resources of IBM to apply to building a compiler. ALGOL 68 enjoyed no such benefactor.

2.12 Some Early Descendants of the ALGOLs

All imperative languages owe some of their design to ALGOL 60 and/or ALGOL 68. This section discusses some of the early descendants of these languages.

2.12.1 Simplicity by Design: Pascal

2.12.1.1 Historical Background

Niklaus Wirth (Wirth is pronounced "Virt") was a member of the International Federation of Information Processing (IFIP) Working Group 2.1, which was created to continue the development of ALGOL in the mid-1960s. In August 1965, Wirth and C. A. R. ("Tony") Hoare contributed to that effort by presenting to the group a somewhat modest proposal for additions and modifications to ALGOL 60 (Wirth and Hoare, 1966). The majority of the group rejected the proposal as being too small an improvement over ALGOL 60. Instead, a much more complex revision was developed, which eventually became ALGOL 68. Wirth, along with a few other group members, did not believe that the ALGOL 68 report should have been released, based on the complexity of both the language and the metalanguage used to describe it. This position later proved to have some validity because the ALGOL 68 documents, and therefore the language, were indeed found to be challenging by the computing community.

The Wirth and Hoare version of ALGOL 60 was named ALGOL-W. It was implemented at Stanford University and was used primarily as an instructional vehicle, but only at a few universities. The primary contributions of ALGOL-W were the value-result method of passing parameters and the **case** statement for multiple selection. The value-result method is an alternative to ALGOL 60's pass-by-name method. Both are discussed in Chapter 9. The **case** statement is discussed in Chapter 8.

Wirth's next major design effort, again based on ALGOL 60, was his most successful: Pascal.[10] The original published definition of Pascal appeared in 1971 (Wirth, 1971). This version was modified somewhat in the implementation process and is described in Wirth (1973). The features that are often ascribed to Pascal in fact came from earlier languages. For example, user-defined data types were introduced in ALGOL 68, the **case** statement in ALGOL-W, and Pascal's records are similar to those of COBOL and PL/I.

10. Pascal is named after Blaise Pascal, a seventeenth-century French philosopher and mathematician who invented the first mechanical adding machine in 1642 (among other things).

2.12.1.2 Evaluation

The largest impact of Pascal was on the teaching of programming. In 1970, most students of computer science, engineering, and science were introduced to programming with Fortran, although some universities used PL/I, languages based on PL/I, and ALGOL-W. By the mid-1970s, Pascal had become the most widely used language for this purpose. This was quite natural, because Pascal was designed specifically for teaching programming. It was not until the late 1990s that Pascal was no longer the most commonly used language for teaching programming in colleges and universities.

Because Pascal was designed as a teaching language, it lacks several features that are essential for many kinds of applications. The best example of this is the impossibility of writing a subprogram that takes as a parameter an array of variable length. Another example is the lack of any separate compilation capability. These deficiencies naturally led to many nonstandard dialects, such as Turbo Pascal.

Pascal's popularity, for both teaching programming and other applications, was based primarily on its remarkable combination of simplicity and expressivity. Although there are some insecurities in Pascal, it is still a relatively safe language, particularly when compared with Fortran or C. By the mid-1990s, the popularity of Pascal was on the decline, both in industry and in universities, primarily due to the rise of Modula-2, Ada, and C++, all of which included features not available in Pascal.

The following is an example of a Pascal program:

```
{Pascal Example Program
 Input:   An integer, listlen, where listlen is less than
          100, followed by listlen-integer values
 Output:  The number of input values that are greater than
          the average of all input values }
program pasex (input, output);
  type intlisttype = array [1..99] of integer;
  var
    intlist : intlisttype;
    listlen, counter, sum, average, result : integer;
  begin
  result := 0;
  sum := 0;
  readln (listlen);
  if ((listlen > 0) and (listlen < 100)) then
    begin
{ Read input into an array and compute the sum }
  for counter := 1 to listlen do
    begin
    readln (intlist[counter]);
    sum := sum + intlist[counter]
    end;
{ Compute the average }
    average := sum / listlen;
```

```
{ Count the number of input values that are > average }
      for counter := 1 to listlen do
        if (intlist[counter] > average) then
          result := result + 1;
{ Print the result }
    writeln ('The number of values > average is:',
             result)
    end  { of the then clause of if (( listlen > 0 ... }
  else
    writeln ('Error-input list length is not legal')
end.
```

2.12.2 A Portable Systems Language: C

Like Pascal, C contributed little to the previously known collection of language features, but it has been widely used over a long period of time. Although originally designed for systems programming, C is well suited for a wide variety of applications.

2.12.2.1 Historical Background

C's ancestors include CPL, BCPL, B, and ALGOL 68. CPL was developed at Cambridge University in the early 1960s. BCPL is a simple systems language, also developed at Cambridge, this time by Martin Richards in 1967 (Richards, 1969).

The first work on the UNIX operating system was done in the late 1960s by Ken Thompson at Bell Laboratories. The first version was written in assembly language. The first high-level language implemented under UNIX was B, which was based on BCPL. B was designed and implemented by Thompson in 1970.

Neither BCPL nor B is a typed language, which is an oddity among high-level languages, although both are much lower-level than a language such as Java. Being untyped means that all data are considered machine words, which, although simple, leads to many complications and insecurities. For example, there is the problem of specifying floating-point rather than integer arithmetic in an expression. In one implementation of BCPL, the variable operands of a floating-point operation were preceded by periods. Variable operands not preceded by periods were considered to be integers. An alternative to this would have been to use different symbols for the floating-point operators.

This problem, along with several others, led to the development of a new typed language based on B. Originally called NB but later named C, it was designed and implemented by Dennis Ritchie at Bell Laboratories in 1972 (Kernighan and Ritchie, 1978). In some cases through BCPL, and in other cases directly, C was influenced by ALGOL 68. This is seen in its **`for`** and **`switch`** statements, in its assigning operators, and in its treatment of pointers.

The only "standard" for C in its first decade and a half was the book by Kernighan and Ritchie (1978).[11] Over that time span, the language slowly evolved, with different implementors adding different features. In 1989, ANSI produced an official description of C (ANSI, 1989), which included many of the features that implementors had already incorporated into the language. This standard was updated in 1999 (ISO, 1999). This later version includes a few significant changes to the language. Among these are a complex data type, a Boolean data type, and C++-style comments (//). We will refer to the 1989 version, which has long been called ANSI C, as C89; we will refer to the 1999 version as C99.

2.12.2.2 Evaluation

C has adequate control statements and data-structuring facilities to allow its use in many application areas. It also has a rich set of operators that provide a high degree of expressiveness.

One of the most important reasons why C is both liked and disliked is its lack of complete type checking. For example, in versions before C99, functions could be written for which parameters were not type checked. Those who like C appreciate the flexibility; those who do not like it find it too insecure. A major reason for its great increase in popularity in the 1980s was that a compiler for it was part of the widely used UNIX operating system. This inclusion in UNIX provided an essentially free and quite good compiler that was available to programmers on many different kinds of computers.

The following is an example of a C program:

```
/* C Example Program
 Input:  An integer, listlen, where listlen is less than
         100, followed by listlen-integer values
 Output: The number of input values that are greater than
         the average of all input values */
int main (){
  int intlist[99], listlen, counter, sum, average, result;
  sum = 0;
  result = 0;
  scanf("%d", &listlen);
  if ((listlen > 0) && (listlen < 100)) {
/* Read input into an array and compute the sum */
    for (counter = 0; counter < listlen; counter++) {
     scanf("%d", &intlist[counter]);
     sum += intlist[counter];
    }
/* Compute the average */
    average = sum / listlen;
```

11. This language is often referred to as "K & R C."

```
/* Count the input values that are > average */
     for  (counter = 0; counter < listlen; counter++)
       if  (intlist[counter] > average) result++;
/* Print result */
    printf("Number of values > average is:%d\n", result);
  }
  else
    printf("Error-input list length is not legal\n");
}
```

2.13 Programming Based on Logic: Prolog

Simply put, logic programming is the use of a formal logic notation to communicate computational processes to a computer. Predicate calculus is the notation used in current logic programming languages.

Programming in logic programming languages is nonprocedural. Programs in such languages do not state exactly *how* a result is to be computed but rather describe the necessary form and/or characteristics of the result. What is needed to provide this capability in logic programming languages is a concise means of supplying the computer with both the relevant information and an inferencing process for computing desired results. Predicate calculus supplies the basic form of communication to the computer, and the proof method, named **resolution**, developed first by Robinson (1965), supplies the inferencing technique.

2.13.1 Design Process

During the early 1970s, Alain Colmerauer and Phillippe Roussel in the Artificial Intelligence Group at the University of Aix-Marseille, together with Robert Kowalski of the Department of Artificial Intelligence at the University of Edinburgh, developed the fundamental design of Prolog. The primary components of Prolog are a method for specifying predicate calculus propositions and an implementation of a restricted form of resolution. Both predicate calculus and resolution are described in Chapter 16. The first Prolog interpreter was developed at Marseille in 1972. The version of the language that was implemented is described in Roussel (1975). The name Prolog is from *pro*gramming *log*ic.

2.13.2 Language Overview

Prolog programs consist of collections of statements. Prolog has only a few kinds of statements, but they can be complex.

One common use of Prolog is as a kind of intelligent database. This application provides a simple framework for discussing the Prolog language.

The database of a Prolog program consists of two kinds of statements: facts and rules. The following are examples of fact statements:

```
mother(joanne, jake).
father(vern, joanne).
```

These state that `joanne` is the `mother` of `jake`, and `vern` is the `father` of `joanne`.

An example of a rule statement is

```
grandparent(X, Z) :- parent(X, Y), parent(Y, Z).
```

This states that it can be deduced that `X` is the `grandparent` of `Z` if it is true that `X` is the `parent` of `Y` and `Y` is the parent of `Z`, for some specific values for the variables `X`, `Y`, and `Z`.

The Prolog database can be interactively queried with goal statements, an example of which is

```
father(bob, darcie).
```

This asks if `bob` is the `father` of `darcie`. When such a query, or goal, is presented to the Prolog system, it uses its resolution process to attempt to determine the truth of the statement. If it can conclude that the goal is true, it displays "true." If it cannot prove it, it displays "false."

2.13.3 Evaluation

In the 1980s, there was a relatively small group of computer scientists who believed that logic programming provided the best hope for escaping from the complexity of imperative languages, and also from the enormous problem of producing the large amount of reliable software that was needed. So far, however, there are two major reasons why logic programming has not become more widely used. First, as with some other nonimperative approaches, programs written in logic languages thus far have proven to be highly inefficient relative to equivalent imperative programs. Second, it has been determined that it is an effective approach for only a few relatively small areas of application: certain kinds of database management systems and some areas of AI.

There is a dialect of Prolog that supports object-oriented programming: Prolog++ (Moss, 1994). Logic programming and Prolog are described in greater detail in Chapter 16.

2.14 History's Largest Design Effort: Ada

The Ada language is the result of the most extensive and expensive language design effort ever undertaken. The following paragraphs briefly describe the evolution of Ada.

2.14.1 Historical Background

The Ada language was developed for the Department of Defense (DoD), so the state of their computing environment was instrumental in determining its form. By 1974, over half of the applications of computers in DoD were embedded systems. An embedded system is one in which the computer hardware is embedded in the device it controls or for which it provides services. Software costs were rising rapidly, primarily because of the increasing complexity of systems. More than 450 different programming languages were in use for DoD projects, and none of them was standardized by DoD. Every defense contractor could define a new and different language for every contract.[12] Because of this language proliferation, application software was rarely reused. Furthermore, no software development tools were created (because they are usually language dependent). A great many languages were in use, but none was actually suitable for embedded systems applications. For these reasons, in 1974, the Army, Navy, and Air Force each independently proposed the development of a single high-level language for embedded systems.

2.14.2 Design Process

Noting this widespread interest, in January 1975, Malcolm Currie, director of Defense Research and Engineering, formed the High-Order Language Working Group (HOLWG), initially headed by Lt. Col. William Whitaker of the Air Force. The HOLWG had representatives from all of the military services and liaisons with Great Britain, France, and what was then West Germany. Its initial charter was to do the following:

- Identify the requirements for a new DoD high-level language.
- Evaluate existing languages to determine whether there was a viable candidate.
- Recommend adoption or implementation of a minimal set of programming languages.

In April 1975, the HOLWG produced the Strawman requirements document for the new language (Department of Defense, 1975a). This was distributed to military branches, federal agencies, selected industrial and university representatives, and interested parties in Europe.

12. This result was largely due to the widespread use of assembly language for embedded systems, along with the fact that most embedded systems used specialized processors.

The Strawman document was followed by Woodenman (Department of Defense, 1975b) in August 1975, Tinman (Department of Defense, 1976) in January 1976, Ironman (Department of Defense, 1977) in January 1977, and finally Steelman (Department of Defense, 1978) in June 1978.

After a tedious process, the many submitted proposals for the language were narrowed down to four finalists, all of which were based on Pascal. In May 1979, the Cii Honeywell/Bull language design proposal was chosen from the four finalists as the design that would be used. The Cii Honeywell/Bull design team in France, the only foreign competitor among the final four, was led by Jean Ichbiah.

In the spring of 1979, Jack Cooper of the Navy Materiel Command recommended the name for the new language, Ada, which was then adopted. The name commemorates Augusta Ada Byron (1815–1851), countess of Lovelace, mathematician, and daughter of poet Lord Byron. She is generally recognized as being the world's first programmer. She worked with Charles Babbage on his mechanical computers, the Difference and Analytical Engines, writing programs for several numerical processes.

The design and the rationale for Ada were published by ACM in its *SIGPLAN Notices* (ACM, 1979) and distributed to a readership of more than 10,000 people. A public test and evaluation conference was held in October 1979 in Boston, with representatives from over 100 organizations from the United States and Europe. By November, more than 500 language reports had been received from 15 different countries. Most of the reports suggested small modifications rather than drastic changes and outright rejections. Based on the language reports, the next version of the requirements specification, the Stoneman document (Department of Defense, 1980), was released in February 1980.

A revised version of the language design was completed in July 1980 and was accepted as MIL-STD 1815, the standard *Ada Language Reference Manual*. The number 1815 was chosen because it was the year of the birth of Augusta Ada Byron. Another revised version of the *Ada Language Reference Manual* was released in July 1982. In 1983, the American National Standards Institute standardized Ada. This "final" official version is described in Goos and Hartmanis (1983). The Ada language design was then frozen for a minimum of five years.

2.14.3 Language Overview

This subsection briefly describes four of the major contributions of the Ada language.

Packages in the Ada language provide the means for encapsulating data objects, specifications for data types, and procedures. This, in turn, provides the support for the use of data abstraction in program design.

The Ada language includes extensive facilities for exception handling, which allow the programmer to gain control after any one of a wide variety of exceptions, or run-time errors, has been detected.

Program units can be generic in Ada. For example, it is possible to write a sort procedure that uses an unspecified type for the data to be sorted. Such a generic procedure must be instantiated for a specified type before it can be used, which is done with a statement that causes the compiler to generate a version of the procedure with the given type. The availability of such generic units increases the range of program units that might be reused, rather than duplicated, by programmers.

The Ada language also provides for concurrent execution of special program units, named tasks, using the rendezvous mechanism. Rendezvous is the name of a method of intertask communication and synchronization.

2.14.4 Evaluation

Perhaps the most important aspects of the design of the Ada language to consider are the following:

- Because the design was competitive, there were no limits on participation.
- The Ada language embodies most of the concepts of software engineering and language design of the late 1970s. Although one can question the actual approaches used to incorporate these features, as well as the wisdom of including such a large number of features in a language, most agree that the features are valuable.
- Although most people did not anticipate it, the development of a compiler for the Ada language was a difficult task. Only in 1985, almost four years after the language design was completed, did truly usable Ada compilers begin to appear.

The most serious criticism of Ada in its first few years was that it was too large and too complex. In particular, Hoare (1981) stated that it should not be used for any application where reliability is critical, which is precisely the type of application for which it was designed. On the other hand, others have praised it as the epitome of language design for its time. In fact, even Hoare eventually softened his view of the language.

The following is an example of an Ada program:

```
-- Ada Example Program
-- Input:   An integer, List_Len, where List_Len is less
--          than 100, followed by List_Len-integer values
-- Output:  The number of input values that are greater
--          than the average of all input values
with Ada.Text_IO, Ada.Integer.Text_IO;
use Ada.Text_IO, Ada.Integer.Text_IO;
procedure Ada_Ex is
  type Int_List_Type is array (1..99) of Integer;
  Int_List : Int_List_Type;
  List_Len, Sum, Average, Result : Integer;
```

```
begin
  Result:= 0;
  Sum := 0;
  Get (List_Len);
  if (List_Len > 0) and (List_Len < 100) then
-- Read input data into an array and compute the sum
    for Counter := 1 .. List_Len loop
      Get (Int_List(Counter));
      Sum := Sum + Int_List(Counter);
    end loop;
-- Compute the average
    Average := Sum / List_Len;
-- Count the number of values that are > average
    for Counter := 1 .. List_Len loop
      if Int_List(Counter) > Average then
        Result:= Result+ 1;
      end if;
    end loop;
-- Print result
    Put ("The number of values > average is:");
    Put (Result);
    New_Line;
  else
    Put_Line ("Error-input list length is not legal");
  end if;
end Ada_Ex;
```

2.14.5 Ada 95 and Ada 2005

Two of the most important new features of Ada 95 are described briefly in the following paragraphs. In the remainder of the book, we will use the name Ada 83 for the original version and Ada 95 (its actual name) for the later version when it is important to distinguish between the two versions. In discussions of language features common to both versions, we will use the name Ada. The Ada 95 standard language is defined in ARM (1995).

The type derivation mechanism of Ada 83 is extended in Ada 95 to allow adding new components to those inherited from a base class. This provides for inheritance, a key ingredient in object-oriented programming languages. Dynamic binding of subprogram calls to subprogram definitions is accomplished through subprogram dispatching, which is based on the tag value of derived types through classwide types. This feature provides for polymorphism, another principal feature of object-oriented programming.

The rendezvous mechanism of Ada 83 provided only a cumbersome and inefficient means of sharing data among concurrent processes. It was necessary to introduce a new task to control access to the shared data. The protected objects of Ada 95 offer an attractive alternative to this. The shared data is encapsulated in a syntactic structure that controls all access to the data, either by rendezvous or by subprogram call.

It is widely believed that the popularity of Ada 95 suffered because the Department of Defense stopped requiring its use in military software systems. There were, of course, other factors that hindered its growth in popularity. Most important among these was the widespread acceptance of C++ for object-oriented programming, which occurred before Ada 95 was released.

There were several additions to Ada 95 to get Ada 2005. Among these were interfaces, similar to those of Java, more control of scheduling algorithms, and synchronized interfaces.

Ada is widely used in both commercial and defense avionics, air traffic control, and rail transportation, as well as in other areas.

2.15 Object-Oriented Programming: Smalltalk

Smalltalk was the first programming language that fully supported object-oriented programming. It is therefore an important part of any discussion of the evolution of programming languages.

2.15.1 Design Process

The concepts that led to the development of Smalltalk originated in the Ph.D. dissertation work of Alan Kay in the late 1960s at the University of Utah (Kay, 1969). Kay had the remarkable foresight to predict the future availability of powerful desktop computers. Recall that the first microcomputer systems were not marketed until the mid-1970s, and they were only remotely related to the machines envisioned by Kay, which were seen to execute a million or more instructions per second and contain several megabytes of memory. Such machines, in the form of workstations, became widely available only in the early 1980s.

Kay believed that desktop computers would be used by nonprogrammers and thus would need very powerful human-interfacing capabilities. The computers of the late 1960s were largely batch oriented and were used exclusively by professional programmers and scientists. For use by nonprogrammers, Kay determined, a computer would have to be highly interactive and use sophisticated graphics in its user interface. Some of the graphics concepts came from the LOGO experience of Seymour Papert, in which graphics were used to aid children in the use of computers (Papert, 1980).

Kay originally envisioned a system he called Dynabook, which was meant to be a general information processor. It was based in part on the Flex language, which he had helped design. Flex was based primarily on SIMULA 67. Dynabook used the paradigm of the typical desk, on which there are a number of papers, some partially covered. The top sheet is often the focus of attention, with the others temporarily out of focus. The display of Dynabook would model this scene, using screen windows to represent various sheets of paper on the desktop. The user would interact with such a display both through

keystrokes and by touching the screen with his or her fingers. After the preliminary design of Dynabook earned him a Ph.D., Kay's goal became to see such a machine constructed.

Kay found his way to the Xerox Palo Alto Research Center (Xerox PARC) and presented his ideas on Dynabook. This led to his employment there and the subsequent birth of the Learning Research Group at Xerox. The first charge of the group was to design a language to support Kay's programming paradigm and implement it on the best personal computer then available. These efforts resulted in an "Interim" Dynabook, consisting of a Xerox Alto workstation and Smalltalk-72 software. Together, they formed a research tool for further development. A number of research projects were conducted with this system, including several experiments to teach programming to children. Along with the experiments came further developments, leading to a sequence of languages that ended with Smalltalk-80. As the language grew, so did the power of the hardware on which it resided. By 1980, both the language and the Xerox hardware nearly matched the early vision of Alan Kay.

2.15.2 Language Overview

The Smalltalk world is populated by nothing but objects, from integer constants to large complex software systems. All computing in Smalltalk is done by the same uniform technique: sending a message to an object to invoke one of its methods. A reply to a message is an object, which either returns the requested information or simply notifies the sender that the requested processing has been completed. The fundamental difference between a message and a subprogram call is this: A message is sent to a data object, specifically to one of the methods defined for the object. The called method is then executed, often modifying the data of the object to which the message was sent; a subprogram call is a message to the code of a subprogram. Usually the data to be processed by the subprogram is sent to it as a parameter.[13]

In Smalltalk, object abstractions are classes, which are very similar to the classes of SIMULA 67. Instances of the class can be created and are then the objects of the program.

The syntax of Smalltalk is unlike that of most other programming language, in large part because of the use of messages, rather than arithmetic and logic expressions and conventional control statements. One of the Smalltalk control constructs is illustrated in the example in the next subsection.

2.15.3 Evaluation

Smalltalk has done a great deal to promote two separate aspects of computing: graphical user interfaces and object-oriented programming. The windowing systems that are now the dominant method of user interfaces

13. Of course, a method call can also pass data to be processed by the called method.

to software systems grew out of Smalltalk. Today, the most significant software design methodologies and programming languages are object oriented. Although the origin of some of the ideas of object-oriented languages came from SIMULA 67, they reached maturation in Smalltalk. It is clear that Smalltalk's impact on the computing world is extensive and will be long-lived.

The following is an example of a Smalltalk class definition:

```
"Smalltalk Example Program"
"The following is a class definition, instantiations of
which can draw equilateral polygons of any number of sides"
class name                    Polygon
superclass                    Object
instance variable names       ourPen
numSides
sideLength
"Class methods"
  "Create an instance"
  new
     ^ super new getPen

  "Get a pen for drawing polygons"
  getPen
     ourPen <- Pen new defaultNib: 2

  "Instance methods"
  "Draw a polygon"
  draw
     numSides timesRepeat: [ourPen go: sideLength;
                            turn: 360 // numSides]

  "Set length of sides"
  length: len
  sideLength <- len

  "Set number of sides"
  sides: num
     numSides <- num
```

2.16 Combining Imperative and Object-Oriented Features: C++

The origins of C were discussed in Section 2.12; the origins of Simula 67 were discussed in Section 2.10; the origins of Smalltalk were discussed in Section 2.15. C++ builds language facilities, borrowed from Simula 67, on top of C to support much of what Smalltalk pioneered. C++ has evolved from C through a sequence of modifications to improve its imperative features and to add constructs to support object-oriented programming.

2.16.1 Design Process

The first step from C toward C++ was made by Bjarne Stroustrup at Bell Laboratories in 1980. The initial modifications to C included the addition of function parameter type checking and conversion and, more significantly, classes, which are related to those of SIMULA 67 and Smalltalk. Also included were derived classes, public/private access control of inherited components, constructor and destructor methods, and friend classes. During 1981, inline functions, default parameters, and overloading of the assignment operator were added. The resulting language was called C with Classes and is described in Stroustrup (1983).

It is useful to consider some goals of C with Classes. The primary goal was to provide a language in which programs could be organized as they could be organized in SIMULA 67—that is, with classes and inheritance. A second important goal was that there should be little or no performance penalty relative to C. For example, array index range checking was not even considered because a significant performance disadvantage, relative to C, would result. A third goal of C with Classes was that it could be used for every application for which C could be used, so virtually none of the features of C would be removed, not even those considered to be unsafe.

By 1984, this language was extended by the inclusion of virtual methods, which provide dynamic binding of method calls to specific method definitions, method name and operator overloading, and reference types. This version of the language was called C++. It is described in Stroustrup (1984).

In 1985, the first available implementation appeared: a system named Cfront, which translated C++ programs into C programs. This version of Cfront and the version of C++ it implemented were named Release 1.0. It is described in Stroustrup (1986).

Between 1985 and 1989, C++ continued to evolve, based largely on user reactions to the first distributed implementation. This next version was named Release 2.0. Its Cfront implementation was released in June 1989. The most important features added to C++ Release 2.0 were support for multiple inheritance (classes with more than one parent class) and abstract classes, along with some other enhancements. Abstract classes are described in Chapter 12.

Release 3.0 of C++ evolved between 1989 and 1990. It added templates, which provide parameterized types, and exception handling. The current version of C++, which was standardized in 1998, is described in ISO (1998).

In 2002, Microsoft released its .NET computing platform, which included a new version of C++, named Managed C++, or MC++. MC++ extends C++ to provide access to the functionality of the .NET Framework. The additions include properties, delegates, interfaces, and a reference type for garbage-collected objects. Properties are discussed in Chapter 11. Delegates are briefly discussed in the introduction to C# in Section 2.19. Because .NET does not support multiple inheritance, neither does MC++.

2.16.2 Language Overview

Because C++ has both functions and methods, it supports both procedural and object-oriented programming.

Operators in C++ can be overloaded, meaning the user can create operators for existing operators on user-defined types. C++ methods can also be overloaded, meaning the user can define more than one method with the same name, provided either the numbers or types of their parameters are different.

Dynamic binding in C++ is provided by virtual methods. These methods define type-dependent operations, using overloaded methods, within a collection of classes that are related through inheritance. A pointer to an object of class A can also point to objects of classes that have class A as an ancestor. When this pointer points to an overloaded virtual method, the method of the current type is chosen dynamically.

Both methods and classes can be templated, which means that they can be parameterized. For example, a method can be written as a templated method to allow it to have versions for a variety of parameter types. Classes enjoy the same flexibility.

C++ supports multiple inheritance. The exception-handling constructs of C++ are discussed in Chapter 14.

2.16.3 Evaluation

C++ rapidly became and remains a widely used language. One factor in its popularity is the availability of good and inexpensive compilers. Another factor is that it is almost completely backward compatible with C (meaning that C programs, with few changes, can be compiled as C++ programs), and in most implementations it is possible to link C++ code with C code—and thus relatively easy for the many C programmers to learn C++. Finally, at the time C++ first appeared, when object-oriented programming began to receive widespread interest, C++ was the only available language that was suitable for large commercial software projects.

On the negative side, because C++ is a very large and complex language, it clearly suffers drawbacks similar to those of PL/I. It inherited most of the insecurities of C, which make it less safe than languages such as Ada and Java.

2.16.4 A Replacement for Objective-C, Swift

Beginning with MAC OS X in 2002, Apple systems software was written in Objective-C. Swift was developed by Apple as an improved replacement for Objective-C. Work on Swift began in 2010 by Chris Lattner. The language was introduced in 2014, with version 2 being introduced in 2015. The first version was proprietary, but the second is open source. It is currently implemented under all of Apple's operating systems, as well as on Linux.

Among the features of Swift are a tuple data type, an option type (variables of this type can have a special no-value value), protocols (similar to Java interfaces), two categories of types, class and struct, which support reference types and value types, as in C#, generic types, no pointers, a safer switch construct, in which the default is to not fall through to the next option, and all statement collections, including a single statement, must be enclosed in braces in all control constructs.

Statements need not be terminated with semicolons, unless there are two or more statements on the same line. Types of data need not be declared, as type inferencing is used. Unlike C and C++, assignment statements do not return a value, so using `x = 0` is not legal in a Boolean expression. This eliminates a common error in C and C++ programs, in which `x = 0` is typed instead of `x == 0`. Heap allocated objects are automatically reclaimed, using reference counters.

Swift programs can interact with Objective-C code and Swift uses the same libraries as that language. Swift is already the tenth most popular programming language, according to the TIOBE Community Report.

2.16.5 Another Related Language: Delphi

Delphi (Lischner, 2000) is a hybrid language, similar to C++ and Objective-C, in that it was created by adding object-oriented support, among other things, to an existing imperative language, in this case Pascal. Apple designed an object-oriented version of Pascal, named Object Pascal, but subsequently dropped the project. Borland, which had developed Turbo Pascal for Windows, also designed an object-oriented version of Pascal, based on Turbo Pascal, also named Object Pascal. For several reasons, Borland renamed Object Pascal as Delphi. The first product with that name, which included an integrated development environment (IDE), was released in 1995. Some consider the IDE to be Delphi and the underlying programming language to be Object Pascal. Other suppliers of similar products continue to refer to the language as Object Pascal.

Many of the differences between C++ and Delphi are a result of the predecessor language and the surrounding programming cultures from which they are derived. Because C is a powerful but potentially unsafe language, C++ also fits that description, at least in the areas of array subscript range checking, pointer arithmetic, and its numerous type coercions. Similarly, because Pascal is more elegant and safer than C, Delphi is more elegant and safer than C++. Delphi is also less complex than C++. For example, Delphi does not include user-defined operator overloading, generic subprograms, and parameterized classes, all of which are part of C++.

Delphi was designed by Anders Hejlsberg, who had previously developed the Turbo Pascal system. Hejlsberg, who moved to Microsoft in 1996, was the lead designer of C#.

2.17 An Imperative-Based Object-Oriented Language: Java

Java's designers started with C++, removed some constructs, changed some, and added a few others. The resulting language provides much of the power and flexibility of C++, but in a smaller, simpler, and safer language. Since that initial design, Java has grown considerably.

2.17.1 Design Process

Java, like many programming languages, was designed for an application for which there appeared to be no satisfactory existing language. In 1990, Sun Microsystems determined there was a need for a programming language for embedded consumer electronic devices, such as toasters, microwave ovens, and interactive TV systems. Reliability was one of the primary goals for such a language. It may not seem that reliability would be an important factor in the software for a microwave oven. If an oven had malfunctioning software, it probably would not pose a grave danger to anyone and most likely would not lead to large legal settlements. However, if the software in a particular model was found to be erroneous after a million units had been manufactured and sold, their recall would entail significant cost. Therefore, reliability *is* an important characteristic of the software in consumer electronic products.

After considering C and C++, it was decided that neither would be satisfactory for developing software for consumer electronic devices. Although C was relatively small, it did not provide support for object-oriented programming, which they deemed a necessity. C++ supported object-oriented programming, but it was judged to be too large and complex, in part because it also supported procedure-oriented programming. It was also believed that neither C nor C++ provided the necessary level of reliability. So, a new language, later named Java, was designed. Its design was guided by the fundamental goal of providing greater simplicity and reliability than C++ was believed to provide.

Although the initial impetus for Java was consumer electronics, none of the products with which it was used in its early years were ever marketed. Starting in 1993, when the World Wide Web became widely used, and largely because of the new graphical browsers, Java was found to be a useful tool for Web programming. In particular, Java applets, which are relatively small Java programs that are interpreted in Web browsers and whose output can be included in displayed Web documents, quickly became very popular in the middle to late 1990s. In the first few years of Java popularity, the Web was its most common application.

The Java design team was headed by James Gosling, who had previously designed the UNIX emacs editor and the NeWS windowing system.

2.17.2 Language Overview

As we stated previously, Java is based on C++ but it was specifically designed to be smaller, simpler, and more reliable. Like C++, Java has both classes and primitive types. Java arrays are instances of a predefined class, whereas in C++

they are not, although many C++ users build wrapper classes for arrays to add features like index range checking, which is implicit in Java.

Java does not have pointers, but its reference types provide some of the capabilities of pointers. These references are used to point to class instances. All objects are allocated on the heap. References are always implicitly dereferenced, when necessary. So they behave more like ordinary scalar variables.

Java has a primitive Boolean type named **`boolean`**, used mainly for the control expressions of its control statements (such as **`if`** and **`while`**). Unlike C and C++, arithmetic expressions cannot be used for control expressions.

One significant difference between Java and many of its predecessors that support object-oriented programming, including C++, is that it is not possible to write stand-alone subprograms in Java. All Java subprograms are methods and are defined in classes. Furthermore, methods can be called through a class or object only. One consequence of this is that while C++ supports both procedural and object-oriented programming, Java supports object-oriented programming only.

Another important difference between C++ and Java is that C++ supports multiple inheritance directly in its class definitions. Java supports only single inheritance of classes, although some of the benefits of multiple inheritance can be gained by using its interface construct.

Among the C++ constructs that were not copied into Java are structs and unions.

Java includes a relatively simple form of concurrency control through its **`synchronized`** modifier, which can appear on methods and blocks. In either case, it causes a lock to be attached. The lock ensures mutually exclusive access or execution. In Java, it is relatively easy to create concurrent processes, which in Java are called *threads*.

Java uses implicit storage deallocation for its objects, often called **garbage collection**. This frees the programmer from needing to delete objects explicitly when they are no longer needed. Programs written in languages that do not have garbage collection often suffer from what is sometimes called memory leakage, which means that storage is allocated but never deallocated. This can obviously lead to eventual depletion of all available storage. Object deallocation is discussed in detail in Chapter 6.

Unlike C and C++, Java includes assignment type coercions (implicit type conversions) only if they are widening (from a "smaller" type to a "larger" type). So **`int`** to **`float`** coercions are done across the assignment operator, but **`float`** to **`int`** coercions are not.

2.17.3 Evaluation

The designers of Java did well at trimming the excess and/or unsafe features of C++. For example, the elimination of half of the assignment coercions that are done in C++ was clearly a step toward higher reliability. Index range checking of array accesses also makes the language safer. The addition of

concurrency enhances the range of applications that can be written in the language, as do the class libraries for graphical user interfaces, database access, and networking.

Java's portability, at least in intermediate form, has often been attributed to the design of the language, but it is not. Any language can be translated to an intermediate form and "run" on any platform that has a virtual machine for that intermediate form. The price of this kind of portability is the cost of interpretation, which traditionally has been about an order of magnitude more than execution of machine code. The initial version of the Java interpreter, called the Java Virtual Machine (JVM), indeed was at least 10 times slower than equivalent compiled C programs. However, many Java programs are now translated to machine code before being executed, using Just-in-Time (JIT) compilers. This makes the efficiency of Java programs competitive with that of programs in conventionally compiled languages such as C++, at least when array index range checking is not considered.

The use of Java increased faster than that of any other programming language. Initially, this was due to its value in programming dynamic Web documents. Clearly, one of the reasons for Java's rapid rise to prominence is simply that programmers like its design. Some developers thought C++ was too large and complex to be practical and safe. Java offered them an alternative that has much of the power of C++, but in a simpler, safer language. Another reason is that the compiler/interpreter system for Java is free and easily obtained on the Web. Java is now widely used in a variety of different applications areas.

The most recent version of Java, Java SE8, appeared in 2014. Since the first version, significant features have been added to the language. Among these are an enumeration class, generics, a new iteration construct, lambda expressions, and numerous class libraries.

The following is an example of a Java program:

```
// Java Example Program
// Input:   An integer, listlen, where listlen is less
//          than 100, followed by length-integer values
// Output:  The number of input data that are greater than
//          the average of all input values
import java.io.*;
class IntSort {
public static void main(String args[]) throws IOException {
  DataInputStream in = new DataInputStream(System.in);
  int listlen,
      counter,
      sum = 0,
      average,
      result = 0;
  int[] intlist =  new int[99];
  listlen = Integer.parseInt(in.readLine());
  if ((listlen > 0) && (listlen < 100)) {
```

```
/* Read input into an array and compute the sum */
    for (counter = 0; counter < listlen; counter++) {
      intlist[counter] =
             Integer.valueOf(in.readLine()).intValue();
      sum += intlist[counter];
    }
/* Compute the average */
    average = sum / listlen;
/* Count the input values that are > average */
    for (counter = 0; counter < listlen; counter++)
      if (intlist[counter] > average) result++;
/* Print result */
      System.out.println(
          "\nNumber of values > average is:" + result);
    } //** end of then clause of if ((listlen > 0) ...
    else System.out.println(
             "Error-input list length is not legal\n");
  } //** end of method main
} //** end of class IntSort
```

2.18 Scripting Languages

Scripting languages have evolved over the past 35 years. Early scripting languages were used by putting a list of commands, called a **script**, in a file to be interpreted. The first of these languages, named `sh` (for shell), began as a small collection of commands that were interpreted as calls to system subprograms that performed utility functions, such as file management and simple file filtering. To this were added variables, control flow statements, functions, and various other capabilities, and the result is a complete programming language. One of the most powerful and widely known of these is `ksh` (Bolsky and Korn, 1995), which was developed by David Korn at Bell Laboratories.

Another scripting language is `awk`, developed by Al Aho, Brian Kernighan, and Peter Weinberger at Bell Laboratories (Aho et al., 1988). `awk` began as a report-generation language but later became a more general-purpose language.

2.18.1 Origins and Characteristics of Perl

The Perl language, developed by Larry Wall, was originally a combination of `sh` and `awk`. Perl has grown significantly since its beginnings and is now a powerful, although still somewhat primitive, programming language. Although it is still often called a scripting language, it is actually more similar to a typical imperative language, since it is always compiled, at least into an intermediate language, before it is executed. Furthermore, it has all the constructs to make it applicable to a wide variety of areas of computational problems.

Perl has a number of interesting features, only a few of which are mentioned in this chapter and discussed later in the book.

Variables in Perl are statically typed and implicitly declared. There are three distinctive namespaces for variables, denoted by the first character of the variables' names. All scalar variable names begin with dollar signs (`$`), all array names begin with at signs (`@`), and all hash names (hashes are briefly described below) begin with percent signs (`%`). This convention makes variable names in programs more readable than those of most other programming languages.

Perl includes a large number of implicit variables. Some of them are used to store Perl parameters, such as the particular form of newline character or characters that are used in the implementation. Implicit variables are commonly used as default parameters to built-in functions and default operands for some operators. The implicit variables have distinctive—although cryptic—names, such as `$!` and `@_`. The implicit variables' names, like the user-defined variable names, use the three namespaces, so `$!` is the name of a scalar variable.

Perl's arrays have two characteristics that set them apart from the arrays of the common imperative languages. First, they have dynamic length, meaning that they can grow and shrink as needed during execution. Second, arrays can be sparse, meaning that there can be gaps between the elements. These gaps do not take space in memory, and the iteration statement used for arrays, **`foreach`**, iterates over the missing elements.

Perl includes associative arrays, which are called **hashes**. These data structures are indexed by strings and are implicitly controlled hash tables. The Perl system supplies the hash function and increases the size of the structure when necessary.

Perl is a powerful, but somewhat dangerous, language. Its scalar type stores both strings and numbers, which are normally stored in double-precision floating-point form. Depending on the context, numbers may be coerced to strings and vice versa. If a string is used in numeric context and the string cannot be converted to a number, zero is used and there is no warning or error message provided for the user. This can lead to errors that are not detected by the compiler or run-time system. Array indexing cannot be checked, because there is no set subscript range for any array. References to nonexistent elements return **`undef`**, which is interpreted as zero in numeric context. So, there is also no error detection in array element access.

Perl's initial use was as a UNIX utility for processing text files. It was and still is widely used as a UNIX system administration tool. When the World Wide Web appeared, Perl achieved widespread use as a common gateway interface language for use with the Web, although it is now rarely used for that purpose. Perl is used as a general-purpose language for a variety of applications, such as computational biology and artificial intelligence.

The following is an example of a Perl program:

```
# Perl Example Program
# Input:   An integer, $listlen, where $listlen is less
#          than 100, followed by $listlen-integer values.
```

```
# Output:  The number of input values that are greater than
#          the average of all input values.
($sum, $result) = (0, 0);
$listlen = <STDIN>;
if (($listlen > 0) && ($listlen < 100)) {
# Read input into an array and compute the sum
  for ($counter = 0; $counter < $listlen; $counter++) {
    $intlist[$counter] = <STDIN>;
  } #- end of for (counter ...
# Compute the average
  $average = $sum / $listlen;
# Count the input values that are > average
  foreach  $num (@intlist) {
    if ($num > $average) { $result++; }
  } #- end of foreach $num ...
# Print result
  print "Number of values > average is: $result \n";
} #- end of if (($listlen ...
else {
  print "Error--input list length is not legal \n";
}
```

2.18.2 Origins and Characteristics of JavaScript

Use of the Web exploded in the mid-1990s after the first graphical browsers appeared. The need for computation associated with HTML documents, which by themselves are completely static, quickly became critical. Computation on the server side was made possible with the common gateway interface (CGI), which allowed HTML documents to request the execution of programs on the server, with the results of such computations returned to the browser in the form of HTML documents. Computation on the browser end became available with the advent of Java applets. Both of these approaches have now been replaced for the most part by newer technologies, primarily scripting languages.

JavaScript was originally developed by Brendan Eich at Netscape. Its original name was Mocha. It was later renamed LiveScript. In late 1995, LiveScript became a joint venture of Netscape and Sun Microsystems and its name was changed to JavaScript. JavaScript has gone through extensive evolution, moving from version 1.0 to version 1.5 by adding many new features and capabilities. A language standard for JavaScript was developed in the late 1990s by the European Computer Manufacturers Association (ECMA) as ECMA-262. This standard has also been approved by the International Standards Organization (ISO) as ISO-16262. Microsoft's version of JavaScript is named JScript .NET.

Although a JavaScript interpreter could be embedded in many different applications, its most common use is embedded in Web browsers. JavaScript

code is embedded in HTML documents and interpreted by the browser when the documents are displayed. The primary uses of JavaScript in Web programming are to validate form input data and create dynamic HTML documents.

In spite of its name, JavaScript is related to Java only through the use of similar syntax. Java is strongly typed, but JavaScript is dynamically typed (see Chapter 5). JavaScript's character strings and its arrays have dynamic length. Because of this, array indices are not checked for validity, although this is required in Java. Java fully supports object-oriented programming, but JavaScript supports neither inheritance nor dynamic binding of method calls to methods.

One of the most important uses of JavaScript is for dynamically creating and modifying HTML documents. JavaScript defines an object hierarchy that matches a hierarchical model of an HTML document, which is defined by the Document Object Model. Elements of an HTML document are accessed through these objects, providing the basis for dynamic control of the elements of documents.

Following is a JavaScript script for the problem previously solved in several languages in this chapter. Note that it is assumed that this script will be called from an HTML document and interpreted by a Web browser.

```
// example.js
//   Input:  An integer, listLen, where listLen is less
//           than 100, followed by listLen-numeric values
// Output:   The number of input values that are greater
//           than the average of all input values

var intList = new Array(99);
var listLen, counter, sum = 0, result = 0;

listLen = prompt (
        "Please type the length of the input list", "");
if ((listLen > 0) && (listLen < 100)) {

// Get the input and compute its sum
   for (counter = 0; counter < listLen; counter++) {
        intList[counter] = prompt (
                        "Please type the next number", "");
        sum += parseInt(intList[counter]);
   }

// Compute the average
   average = sum / listLen;

// Count the input values that are > average
   for (counter = 0; counter < listLen; counter++)
       if (intList[counter] > average) result++;
```

```
// Display the results
   document.write("Number of values > average is: ",
                  result, "<br />");
}  else
    document.write(
       "Error - input list length is not legal <br />");
```

2.18.3 Origins and Characteristics of PHP

PHP (Tatroe et al., 2013) was developed by Rasmus Lerdorf, an employee of the Apache Group, in 1994. His initial motivation was to provide a tool to help track visitors to his personal Web site. In 1995, he developed a package called Personal Home Page Tools, which became the first publicly distributed version of PHP. Originally, PHP was an abbreviation for Personal Home Page. Later, its user community began using the recursive name PHP: Hypertext Preprocessor, which subsequently forced the original name into obscurity. PHP is now developed, distributed, and supported as an open-source product. PHP processors are resident on most Web servers.

PHP is an HTML-embedded server-side scripting language specifically designed for Web applications. PHP code is interpreted on the Web server when an HTML document in which it is embedded has been requested by a browser. PHP code usually produces HTML code as output, which replaces the PHP code in the HTML document. Therefore, a Web browser never sees PHP code.

PHP is similar to JavaScript in its syntactic appearance, the dynamic nature of its strings and arrays, and its use of dynamic typing. PHP's arrays are a combination of JavaScript's arrays and Perl's hashes.

The original version of PHP did not support object-oriented programming. Abstract classes, interfaces, destructors, and access controls for class members have since been added to the language.

PHP allows simple access to HTML form data, so form processing is easy with PHP. PHP provides support for many different database management systems. This makes it a useful language for building programs that need Web access to databases.

The current version of PHP is 7, released in 2015.

2.18.4 Origins and Characteristics of Python

Python (Lutz, 2013) is an object-oriented interpreted scripting language. Its initial design was by Guido van Rossum at Stichting Mathematisch Centrum in the Netherlands in the early 1990s. Its development is being continued by the Python Software Foundation. Python is being used for the same kinds of applications as Perl: system administration and other relatively small computing tasks. Python is an open-source system that is available for most common

computing platforms. The Python implementation is available at `www.python.org`, which also has extensive information regarding Python.

Python's syntax is not based directly on any commonly used language. It is type checked, but dynamically typed. Instead of arrays, Python includes three kinds of data structures: lists; immutable lists, which are called **tuples**; and hashes, which are called **dictionaries**. There is a collection of list methods, such as `append`, `insert`, `remove`, and `sort`, as well as a collection of methods for dictionaries, such as `keys`, `values`, `copy`, and `has_key`. Python also supports list comprehensions, which originated with the Haskell language. List comprehensions are discussed in Section 15.8.

Python is object oriented, includes the pattern-matching capabilities of Perl, and has exception handling. Garbage collection is used to reclaim objects when they are no longer needed.

Support for form processing is provided by the `cgi` module. Modules that support cookies, networking, and database access are also available.

Python includes support for concurrency with its threads, as well as support for network programming with its sockets. It also has more support for functional programming than other nonfunctional programming languages.

One of the more interesting features of Python is that it can be easily extended by any user. The modules that support the extensions can be written in any compiled language. Extensions can add functions, variables, and object types. These extensions are implemented as additions to the Python interpreter.

2.18.5 Origins and Characteristics of Ruby

Ruby (Thomas et al., 2005) was designed by Yukihiro Matsumoto (aka Matz) in the early 1990s and released in 1996. Since then it has continually evolved. The motivation for Ruby was dissatisfaction of its designer with Perl and Python. Although both Perl and Python support object-oriented programming,[14] neither is a pure object-oriented language, at least in the sense that each has primitive (nonobject) types and each supports functions.

The primary characterizing feature of Ruby is that it is a pure object-oriented language, just as is Smalltalk. Every data value is an object and all operations are via method calls. The operators in Ruby are only syntactic mechanisms to specify method calls for the corresponding operations. Because they are methods, they can be redefined. All classes, predefined or user defined, can be subclassed.

Both classes and objects in Ruby are dynamic in the sense that methods can be dynamically added to either. This means that both classes and objects can have different sets of methods at different times during execution. So, different instantiations of the same class can behave differently. Collections of methods, data, and constants can be included in the definition of a class.

14. Actually, Python's support for object-oriented programming is partial.

The syntax of Ruby is related to that of Eiffel and Ada. There is no need to declare variables, because dynamic typing is used. The scope of a variable is specified in its name: A variable whose name begins with a letter has local scope; one that begins with `@` is an instance variable; one that begins with `$` has global scope. A number of features of Perl are present in Ruby, including implicit variables with silly names, such as `$_`.

As is the case with Python, any user can extend and/or modify Ruby. Ruby is culturally interesting because it is the first programming language designed in Japan that has achieved relatively widespread use in the United States.

2.19 The Flagship .NET Language: C#

C#, along with the development platform .NET,[15] was announced by Microsoft in 2000. In January 2002, production versions of both were released.

2.19.1 Design Process

C# is based on C++ and Java but includes some ideas from Delphi and Visual Basic. Its lead designer, Anders Hejlsberg, also designed Turbo Pascal and Delphi, which explains the Delphi parts of the heritage of C#.

The purpose of C# is to provide a language for component-based software development, specifically for such development in the .NET Framework. In this environment, components from a variety of languages easily can be combined to form systems. All of the .NET languages, which include C#, VB.NET, Managed C++, F#, and JScript .NET,[16] use the common type system (CTS). The CTS provides a common class library. All types in all five .NET languages inherit from a single class root, `System.Object`. Compilers that conform to the CTS specification create objects that can be combined into software systems. All .NET languages are compiled into the same intermediate form, Intermediate Language (IL).[17] Unlike Java, however, the IL is never interpreted. A Just-in-Time compiler is used to translate IL into machine code before it is executed.

2.19.2 Language Overview

Many believe that one of Java's most important advances over C++ lies in the fact that it excludes some of C++'s features. For example, C++ supports multiple inheritance, pointers, structs, **`enum`** types, operator overloading, and a goto statement, but Java includes none of these. The designers of C# obviously

15. The .NET development system is briefly discussed in Chapter 1.
16. Many other languages have been modified to be .NET languages.
17. Initially, IL was called MSIL (Microsoft Intermediate Language), but apparently many people thought that name was too long.

disagreed with this wholesale removal of features, because all of these except multiple inheritance have been included in C#.

To the credit of C#'s designers, however, in several cases, the C# version of a C++ feature has been improved. For example, the **enum** types of C# are safer than those of C++, because they are never implicitly converted to integers. This allows them to be more type safe. The **struct** type was changed significantly, resulting in a truly useful construct, whereas in C++ it serves virtually no purpose. C#'s structs are discussed in Chapter 12. C# takes a stab at improving the **switch** statement that is used in C, C++, and Java. C#'s switch is discussed in Chapter 8.

Although C++ includes function pointers, they share the lack of safety that is inherent in C++'s pointers to variables. C# includes a new type, delegates, which are both object-oriented and type-safe references to subprograms. Delegates are used for implementing event handlers, controlling the execution of threads, and callbacks.[18] Callbacks are implemented in Java with interfaces; in C++, method pointers are used.

In C#, methods can take a variable number of parameters, as long as they are all the same type. This is specified by the use of a formal parameter of array type, preceded by the **`params`** reserved word.

Both C++ and Java use two distinct typing systems: one for primitives and one for objects. In addition to being confusing, this leads to a frequent need to convert values between the two systems—for example, to put a primitive value into a collection that stores objects. C# makes the conversion between values of the two typing systems partially implicit through the implicit boxing and unboxing operations, which are discussed in detail in Chapter 12.[19]

Among the other features of C# are rectangular arrays, which are not supported in most programming languages, and a **`foreach`** statement, which is an iterator for arrays and collection objects. A similar **`foreach`** statement is found in Perl, PHP, and Java 5.0. Also, C# includes properties, which are an alternative to public data members. Properties are specified as data members with get and set methods, which are implicitly called when references and assignments are made to the associated data members.

C# has evolved continuously and quickly from its initial release in 2002. The most recent version is C# 7.0. New in C# 7.0 are tuples and a form of pattern matching.

2.19.3 Evaluation

C# was meant to be an improvement over both C++ and Java as a general-purpose programming language. Although it can be argued that some of its features are a step backward, C# includes some constructs that move it beyond its predecessors. Some of its features will surely be adopted by other programming languages of the future.

---

18. When an object calls a method of another object and needs to be notified when that method has completed its task, the called method calls its caller back. This is known as a callback.
19. This feature was added to Java in Java 5.0.

The following is an example of a C# program:

```
// C# Example Program
// Input:  An integer, listlen, where listlen is less than
//         100, followed by listlen-integer values.
// Output: The number of input values that are greater
//         than the average of all input values.
using System;
public class Ch2example {
  static void Main() {
    int[] intlist;
    int listlen,
        counter,
        sum = 0,
        average,
        result = 0;
    intList = new int[99];
    listlen = Int32.Parse(Console.readLine());
    if ((listlen > 0) && (listlen < 100)) {
// Read input into an array and compute the sum
      for (counter = 0; counter < listlen; counter++) {
        intList[counter] =
                        Int32.Parse(Console.readLine());
        sum += intList[counter];
      } //- end of for (counter ...
// Compute the average
      average = sum / listlen;
// Count the input values that are > average
      foreach (int num in intList)
        if (num > average) result++;
// Print result
      Console.WriteLine(
         "Number of values > average is:" + result);
    } //- end of if ((listlen ...
    else
      Console.WriteLine(
         "Error--input list length is not legal");
  } //- end of method Main
} //- end of class Ch2example
```

2.20 Markup-Programming Hybrid Languages

A markup-programming hybrid language is a markup language in which some of the elements can specify programming actions, such as control flow and computation. The following subsections introduce two such hybrid languages, XSLT and JSP.

2.20.1 XSLT

eXtensible markup language (XML) is a metamarkup language. Such a language is used to define markup languages. XML-derived markup languages are used to define XML data documents. Although XML documents are human readable, they are processed by computers. This processing sometimes consists only of transformations to alternative forms that can be effectively displayed or printed. In many cases, such transformations are to HTML, which can be displayed by a Web browser. In other cases, the data in the document is processed, just as with other forms of data files.

The transformation of XML documents to HTML documents is specified in another markup language, eXtensible stylesheet language transformations (XSLT) (`www.w3.org/TR/XSLT`). XSLT can specify programming-like operations. Therefore, XSLT is a markup-programming hybrid language. XSLT was defined by the World Wide Web Consortium (W3C) in the late 1990s.

An XSLT processor is a program that takes as input an XML data document and an XSLT document (which is also in the form of an XML document). In this processing, the XML data document is transformed to another XML document,[20] using the transformations described in the XSLT document. The XSLT document specifies transformations by defining templates, which are data patterns that could be found by the XSLT processor in the XML input file. Associated with each template in the XSLT document are its transformation instructions, which specify how the matching data is to be transformed before being put in the output document. So, the templates (and their associated processing) act as subprograms, which are "executed" when the XSLT processor finds a pattern match in the data of the XML document.

XSLT also has programming constructs at a lower level. For example, a looping construct is included, which allows repeated parts of the XML document to be selected. There is also a sort process. These lower-level constructs are specified with XSLT tags, such as `<for-each>`.

2.20.2 JSP

The "core" part of the Java Server Pages Standard Tag Library (JSTL) is another markup-programming hybrid language, although its form and purpose are different from those of XSLT. Before discussing JSTL, it is necessary to introduce the ideas of servlets and Java Server Pages (JSP). A **servlet** is an instance of a Java class that resides on and is executed on a Web server system. The execution of a servlet is requested by a markup document being displayed by a Web browser. The servlet's output, which is in the form of an HTML document, is returned to the requesting browser. A program that runs in the Web server process, called a **servlet container**, controls the execution of servlets. Servlets are commonly used for form processing and for database access.

---

20. The output document of the XSLT processor could also be in HTML or plain text.

JSP is a collection of technologies designed to support dynamic Web documents and provide other processing needs of Web documents. When a JSP document, which is often a mixture of HTML and Java, is requested by a browser, the JSP processor program, which resides on a Web server system, converts the document to a servlet. The document's embedded Java code is copied to the servlet. The plain HTML is copied into Java print statements that output it as is. The JSTL markup in the JSP document is processed, as discussed in the following paragraph. The servlet produced by the JSP processor is run by the servlet container.

The JSTL defines a collection of XML action elements that control the processing of the JSP document on the Web server. These elements have the same form as other elements of HTML and XML. One of the most commonly used JSTL control action elements is `if`, which specifies a Boolean expression as an attribute.[21] The content of the `if` element (the text between the opening tag (`<if>`) and its closing tag (`</if>`)) is HTML code that will be included in the output document only if the Boolean expression evaluates to true. The `if` element is related to the C/C++ `#if` preprocessor command. The JSP container processes the JSTL parts of JSP documents in a way that is similar to how the C/C++ preprocessor processes C and C++ programs. The preprocessor commands are instructions for the preprocessor to specify how the output file is to be constructed from the input file. Similarly, JSTL control action elements are instructions for the JSP processor to specify how to build the XML output file from the XML input file.

One common use of the `if` element is for the validation of form data submitted by a browser user. Form data is accessible by the JSP processor and can be tested with the `if` element to ensure that it is sensible data. If not, the `if` element can insert an error message for the user in the output document.

For multiple selection control, JSTL has `choose`, `when`, and `otherwise` elements. JSTL also includes a `forEach` element, which iterates over collections, which typically are form values from a client. The `forEach` element can include `begin`, `end`, and `step` attributes to control its iterations.

SUMMARY

We have investigated the development of a number of programming languages. This chapter gives the reader a good perspective on current issues in language design. We have set the stage for an in-depth discussion of the important features of contemporary languages.

BIBLIOGRAPHIC NOTES

Perhaps the most important source of historical information about the development of early programming languages is *History of Programming Languages*, edited by Richard Wexelblat (1981). It contains the developmental background

21. An attribute in HTML, which is embedded in the opening tag of an element, provides further information about that element.

and environment of 13 important programming languages, as told by the designers themselves. A similar work resulted from a second "history" conference, published as a special issue of *ACM SIGPLAN Notices* (ACM, 1993a). In this work, the history and evolution of 13 more programming languages are discussed.

The paper "Early Development of Programming Languages" (Knuth and Pardo, 1977), which is part of the *Encyclopedia of Computer Science and Technology*, is an excellent 85-page work that details the development of languages up to and including Fortran. The paper includes example programs to demonstrate the features of many of those languages.

Another book of great interest is *Programming Languages: History and Fundamentals*, by Jean Sammet (1969). It is a 785-page work filled with details of 80 programming languages of the 1950s and 1960s. Sammet has also published several updates to her book, such as *Roster of Programming Languages for 1974–75* (1976).

REVIEW QUESTIONS

1. In what year was Plankalkül designed? In what year was that design published?
2. What two common data structures were included in Plankalkül?
3. How were the pseudocodes of the early 1950s implemented?
4. Speedcoding was invented to overcome two significant shortcomings of the computer hardware of the early 1950s. What were they?
5. Why was the slowness of interpretation of programs acceptable in the early 1950s?
6. What hardware capability that first appeared in the IBM 704 computer strongly affected the evolution of programming languages? Explain why.
7. In what year was the Fortran design project begun?
8. What was the primary application area of computers at the time Fortran was designed?
9. What was the source of all of the control flow statements of Fortran I?
10. What was the most significant feature added to Fortran I to get Fortran II?
11. What control flow statements were added to Fortran IV to get Fortran 77?
12. Which version of Fortran was the first to have any sort of dynamic variables?
13. Which version of Fortran was the first to have character string handling?
14. Why were linguists interested in artificial intelligence in the late 1950s?
15. Where was Lisp developed? By whom?
16. In what way are Scheme and Common Lisp opposites of each other?
17. What dialect of Lisp is used for introductory programming courses at some universities?

18. What two professional organizations together designed ALGOL 60?
19. In what version of ALGOL did block structure appear?
20. What missing language element of ALGOL 60 damaged its chances for widespread use?
21. What language was designed to describe the syntax of ALGOL 60?
22. On what programming language was COBOL based?
23. In what year did the COBOL design process begin?
24. What data structure that appeared in COBOL originated with Plankalkül?
25. What organization was most responsible for the early success of COBOL (in terms of extent of use)?
26. What user group was the target of the first version of Basic?
27. Why was Basic an important language in the early 1980s?
28. PL/I was designed to replace what two languages?
29. For what new line of computers was PL/I designed?
30. What features of SIMULA 67 are now important parts of some object-oriented languages?
31. What innovation of data structuring was introduced in ALGOL 68 but is often credited to Pascal?
32. What design criterion was used extensively in ALGOL 68?
33. What language introduced the **`case`** statement?
34. What operators in C were modeled on similar operators in ALGOL 68?
35. What are two characteristics of C that make it less safe than Pascal?
36. What is a nonprocedural language?
37. What are the two kinds of statements that populate a Prolog database?
38. What is the primary application area for which Ada was designed?
39. What are the concurrent program units of Ada called?
40. What Ada construct provides support for abstract data types?
41. What populates the Smalltalk world?
42. What three concepts are the basis for object-oriented programming?
43. Why does C++ include the features of C that are known to be unsafe?
44. What language was Swift designed to replace?
45. What do the Ada and COBOL languages have in common?
46. What was the first application for Java?
47. What characteristic of Java is most evident in JavaScript?
48. How does the typing system of PHP and JavaScript differ from that of Java?

49. What array structure is included in C# but not in C, C++, or Java?
50. What two languages was the original version of Perl meant to replace?
51. For what application area is JavaScript most widely used?
52. What is the relationship between JavaScript and PHP, in terms of their use?
53. PHP's primary data structure is a combination of what two data structures from other languages?
54. What data structure does Python use in place of arrays?
55. What characteristic does Ruby share with Smalltalk?
56. What characteristic of Ruby's arithmetic operators makes them unique among those of other languages?
57. What deficiency of the **`switch`** statement of C is addressed with the changes made by C# to that statement?
58. What is the primary platform on which C# is used?
59. What are the inputs to an XSLT processor?
60. What is the output of an XSLT processor?
61. What element of the JSTL is related to a subprogram?
62. To what is a JSP document converted by a JSP processor?
63. Where are servlets executed?

PROBLEM SET

1. What features of Plankalkül do you think would have had the greatest influence on Fortran 0 if the Fortran designers had been familiar with Plankalkül?
2. Determine the capabilities of Backus's 701 Speedcoding system, and compare them with those of a contemporary programmable hand calculator.
3. Write a short history of the A-0, A-1, and A-2 systems designed by Grace Hopper and her associates.
4. Compare the facilities of Fortran 0 with those of the Laning and Zierler system.
5. Which of the three original goals of the ALGOL design committee, in your opinion, was most difficult to achieve at that time?
6. Make an educated guess as to the most common syntax error in Lisp programs.
7. Lisp began as a pure functional language but gradually acquired more and more imperative features. Why?

8. Describe in detail the three most important reasons, in your opinion, why ALGOL 60 did not become a very widely used language.
9. Why, in your opinion, did COBOL allow long identifiers when Fortran and ALGOL did not?
10. Outline the major motivation of IBM in developing PL/I.
11. Was IBM's assumption, on which it based its decision to develop PL/I, correct, given the history of computers and language developments since 1964?
12. Describe, in your own words, the concept of orthogonality in programming language design.
13. What is the primary reason why PL/I became more widely used than ALGOL 68?
14. What are the arguments both for and against the idea of a typeless language?
15. Are there any logic programming languages other than Prolog?
16. What is your opinion of the argument that languages that are too complex are too dangerous to use, and we should therefore keep all languages small and simple?
17. Do you think language design by committee is a good idea? Support your opinion.
18. Languages continually evolve. What sort of restrictions do you think are appropriate for changes in programming languages? Compare your answers with the evolution of Fortran.
19. Build a table identifying all of the major language developments, together with when they occurred, in what language they first appeared, and the identities of the developers.
20. There have been some public interchanges between Microsoft and Sun concerning the design of Microsoft's J++ and C# and Sun's Java. Read some of these documents, which are available on their respective Web sites, and write an analysis of the disagreements concerning the delegates.
21. In recent years data structures have evolved within scripting languages to replace traditional arrays. Explain the chronological sequence of these developments.
22. Explain two reasons why pure interpretation is an acceptable implementation method for several recent scripting languages.
23. Why, in your opinion, do new scripting languages appear more frequently than new compiled languages?
24. Give a brief general description of a markup-programming hybrid language.

PROGRAMMING EXERCISES

1. To understand the value of records in a programming language, write a small program in a C-based language that uses an array of structs that store student information, including name, age, GPA as a float, and grade level as a string (e.g., "freshmen," etc.). Also, write the same program in the same language without using structs.
2. To understand the value of recursion in a programming language, write a program that implements quicksort, first using recursion and then without recursion.
3. To understand the value of counting loops, write a program that implements matrix multiplication using counting loop constructs. Then write the same program using only logical loops—for example, `while` loops.

3

Describing Syntax and Semantics

3.1 Introduction
3.2 The General Problem of Describing Syntax
3.3 Formal Methods of Describing Syntax
3.4 Attribute Grammars
3.5 Describing the Meanings of Programs: Dynamic Semantics

This chapter begins by defining the terms *syntax* and *semantics*. Then, a detailed discussion of the most common method of describing syntax, context-free grammars (also known as Backus-Naur Form), is presented. Included in this discussion are derivations, parse trees, ambiguity, descriptions of operator precedence and associativity, and extended Backus-Naur Form. Attribute grammars, which can be used to describe both the syntax and static semantics of programming languages, are discussed next. In the last section, three formal methods of describing semantics—operational, axiomatic, and denotational semantics—are introduced. Because of the inherent complexity of the semantics description methods, our discussion of them is brief. One could easily write an entire book on just one of the three (as several authors have).

3.1 Introduction

The task of providing a concise yet understandable description of a programming language is difficult but essential to the language's success. ALGOL 60 and ALGOL 68 were first presented using concise formal descriptions; in both cases, however, the descriptions were not easily understandable, partly because each used a new notation. The levels of acceptance of both languages suffered as a result. On the other hand, some languages have suffered the problem of having many slightly different dialects, a result of a simple but informal and imprecise definition.

One of the problems in describing a language is the diversity of the people who must understand the description. Among these are initial evaluators, implementors, and users. Most new programming languages are subjected to a period of scrutiny by potential users, often people within the organization that employs the language's designer, before their designs are completed. These are the initial evaluators. The success of this feedback cycle depends heavily on the clarity of the description.

Programming language implementors obviously must be able to determine how the expressions, statements, and program units of a language are formed, and also their intended effect when executed. The difficulty of the implementors' job is, in part, determined by the completeness and precision of the language description.

Finally, language users must be able to determine how to encode software solutions by referring to a language reference manual. Textbooks and courses enter into this process, but language manuals are usually the only authoritative printed information source about a language.

The study of programming languages, like the study of natural languages, can be divided into examinations of syntax and semantics. The **syntax** of a programming language is the form of its expressions, statements, and program units. Its **semantics** is the meaning of those expressions, statements, and program units. For example, the syntax of a Java `while` statement is

`while` (boolean_expr) statement

The semantics of this statement form is that when the current value of the Boolean expression is true, the embedded statement is executed. Then control implicitly returns to the Boolean expression to repeat the process. If the Boolean expression is false, control transfers to the statement following the **`while`** construct.

Although they are often separated for discussion purposes, syntax and semantics are closely related. In a well-designed programming language, semantics should follow directly from syntax; that is, the appearance of a statement should strongly suggest what the statement is meant to accomplish.

Describing syntax is easier than describing semantics, partly because a concise and universally accepted notation is available for syntax description, but none has yet been developed for semantics.

3.2 The General Problem of Describing Syntax

A language, whether natural (such as English) or artificial (such as Java), is a set of strings of characters from some alphabet. The strings of a language are called **sentences** or statements. The syntax rules of a language specify which strings of characters from the language's alphabet are in the language. English, for example, has a large and complex collection of rules for specifying the syntax of its sentences. By comparison, even the largest and most complex programming languages are syntactically very simple.

Formal descriptions of the syntax of programming languages, for simplicity's sake, often do not include descriptions of the lowest-level syntactic units. These small units are called **lexemes**. The description of lexemes can be given by a lexical specification, which is usually separate from the syntactic description of the language. The lexemes of a programming language include its numeric literals, operators, and special words, among others. One can think of programs as strings of lexemes rather than of characters.

Lexemes are partitioned into groups—for example, the names of variables, methods, classes, and so forth in a programming language form a group called *identifiers*. Each lexeme group is represented by a name, or token. So, a **token** of a language is a category of its lexemes. For example, an identifier is a token that can have lexemes, or instances, such as `sum` and `total`. In some cases, a token has only a single possible lexeme. For example, the token for the arithmetic operator symbol + has just one possible lexeme. Consider the following Java statement:

```
index = 2 * count + 17;
```

The lexemes and tokens of this statement are

Lexemes	*Tokens*
`index`	identifier
`=`	equal_sign
`2`	int_literal

`*`	mult_op
`count`	identifier
`+`	plus_op
`17`	int_literal
`;`	semicolon

The example language descriptions in this chapter are very simple, and most include lexeme descriptions.

3.2.1 Language Recognizers

In general, languages can be formally defined in two distinct ways: by **recognition** and by **generation** (although neither provides a definition that is practical by itself for people trying to learn or use a programming language). Suppose we have a language L that uses an alphabet Σ of characters. To define L formally using the recognition method, we would need to construct a mechanism R, called a recognition device, capable of reading strings of characters from the alphabet Σ. R would indicate whether a given input string was or was not in L. In effect, R would either accept or reject the given string. Such devices are like filters, separating legal sentences from those that are incorrectly formed. If R, when fed any string of characters over Σ, accepts it only if it is in L, then R is a description of L. Because most useful languages are, for all practical purposes, infinite, this might seem like a lengthy and ineffective process. Recognition devices, however, are not used to enumerate all of the sentences of a language—they have a different purpose.

The syntax analysis part of a compiler is a recognizer for the language the compiler translates. In this role, the recognizer need not test all possible strings of characters from some set to determine whether each is in the language. Rather, it need only determine whether given programs are in the language. In effect then, the syntax analyzer determines whether the given programs are syntactically correct. The structure of syntax analyzers, also known as parsers, is discussed in Chapter 4.

3.2.2 Language Generators

A language generator is a device that can be used to generate the sentences of a language. We can think of the generator as having a button that produces a sentence of the language every time it is pushed. Because the particular sentence that is produced by a generator when its button is pushed is unpredictable, a generator seems to be a device of limited usefulness as a language descriptor. However, people prefer certain forms of generators over recognizers because they can more easily read and understand them. By contrast, the syntax-checking portion of a compiler (a language recognizer) is not as useful a language description for a programmer because it can be used only in trial-and-error mode. For example, to determine the correct syntax of a particular statement using a compiler, the programmer can only submit a speculated version and

note whether the compiler accepts it. On the other hand, it is often possible to determine whether the syntax of a particular statement is correct by comparing it with the structure of the generator.

There is a close connection between formal generation and recognition devices for the same language. This was one of the seminal discoveries in computer science, and it led to much of what is now known about formal languages and compiler design theory. We return to the relationship of generators and recognizers in the next section.

3.3 Formal Methods of Describing Syntax

This section discusses the formal language-generation mechanisms, usually called **grammars**, that are commonly used to describe the syntax of programming languages.

3.3.1 Backus-Naur Form and Context-Free Grammars

In the middle to late 1950s, two men, Noam Chomsky and John Backus, in unrelated research efforts, developed the same syntax description formalism, which subsequently became the most widely used method for programming language syntax.

3.3.1.1 Context-Free Grammars

In the mid-1950s, Noam Chomsky, a noted linguist (among other things), described four classes of generative devices or grammars that define four classes of languages (Chomsky, 1956, 1959). Two of these grammar classes, named *context-free* and *regular*, turned out to be useful for describing the syntax of programming languages. The forms of the tokens of programming languages can be described by regular grammars. The syntax of whole programming languages, with minor exceptions, can be described by context-free grammars. Because Chomsky was a linguist, his primary interest was the theoretical nature of natural languages. He had no interest at the time in the artificial languages used to communicate with computers. So it was not until later that his work was applied to programming languages.

3.3.1.2 Origins of Backus-Naur Form

Shortly after Chomsky's work on language classes, the ACM-GAMM group began designing ALGOL 58. A landmark paper describing ALGOL 58 was presented by John Backus, a prominent member of the ACM-GAMM group, at an international conference in 1959 (Backus, 1959). This paper introduced a new formal notation for specifying programming language syntax. The new notation was later modified slightly by Peter Naur for the description of

ALGOL 60 (Naur, 1960). This revised method of syntax description became known as **Backus-Naur Form**, or simply **BNF**.

BNF is a natural notation for describing syntax. In fact, something similar to BNF was used by Panini to describe the syntax of Sanskrit several hundred years before Christ (Ingerman, 1967).

Although the use of BNF in the ALGOL 60 report was not immediately accepted by computer users, it soon became and is still the most popular method of concisely describing programming language syntax.

It is remarkable that BNF is nearly identical to Chomsky's generative devices for context-free languages, called **context-free grammars**. In the remainder of the chapter, we refer to context-free grammars simply as grammars. Furthermore, the terms BNF and grammar are used interchangeably.

3.3.1.3 Fundamentals

A **metalanguage** is a language that is used to describe another language. BNF is a metalanguage for programming languages.

BNF uses abstractions for syntactic structures. A simple Java assignment statement, for example, might be represented by the abstraction <assign> (pointed brackets are often used to delimit names of abstractions). The actual definition of <assign> can be given by

<assign> → <var> = <expression>

The text on the left side of the arrow, which is aptly called the **left-hand side** (LHS), is the abstraction being defined. The text to the right of the arrow is the definition of the LHS. It is called the **right-hand side** (RHS) and consists of some mixture of tokens, lexemes, and references to other abstractions. (Actually, tokens are also abstractions.) Altogether, the definition is called a **rule**, or **production**. In the example rule just given, the abstractions <var> and <expression> obviously must be defined for the <assign> definition to be useful.

This particular rule specifies that the abstraction <assign> is defined as an instance of the abstraction <var>, followed by the lexeme =, followed by an instance of the abstraction <expression>. One example sentence whose syntactic structure is described by the rule is

```
total = subtotal1 + subtotal2
```

The abstractions in a BNF description, or grammar, are often called **nonterminal symbols**, or simply **nonterminals**, and the lexemes and tokens of the rules are called **terminal symbols**, or simply **terminals**. A BNF description, or **grammar**, is a collection of rules.

Nonterminal symbols can have two or more distinct definitions, representing two or more possible syntactic forms in the language. Multiple definitions can be written as a single rule, with the different definitions separated

by the symbol |, meaning logical OR. For example, a Java **if** statement can be described with the rules

<if_stmt> → **if** (<logic_expr>) <stmt>
<if_stmt> → **if** (<logic_expr>) <stmt> **else** <stmt>

or with the rule

<if_stmt> → **if** (<logic_expr>) <stmt>
| **if** (<logic_expr>) <stmt> **else** <stmt>

In these rules, <stmt> represents either a single statement or a compound statement.

Although BNF is simple, it is sufficiently powerful to describe nearly all of the syntax of programming languages. In particular, it can describe lists of similar constructs, the order in which different constructs must appear, and nested structures to any depth, and even imply operator precedence and operator associativity.

3.3.1.4 Describing Lists

Variable-length lists in mathematics are often written using an ellipsis (. . .); 1, 2, . . . is an example. BNF does not include the ellipsis, so an alternative method is required for describing lists of syntactic elements in programming languages (for example, a list of identifiers appearing on a data declaration statement). For BNF, the alternative is recursion. A rule is **recursive** if its LHS appears in its RHS. The following rules illustrate how recursion is used to describe lists:

<ident_list> → identifier
| identifier, <ident_list>

This defines <ident_list> as either a single token (identifier) or an identifier followed by a comma and another instance of <ident_list>. Recursion is used to describe lists in many of the example grammars in the remainder of this chapter.

3.3.1.5 Grammars and Derivations

A grammar is a generative device for defining languages. The sentences of the language are generated through a sequence of applications of the rules, beginning with a special nonterminal of the grammar called the **start symbol**. This sequence of rule applications is called a **derivation**. In a grammar for a complete programming language, the start symbol represents a complete program and is often named <program>. The simple grammar shown in Example 3.1 is used to illustrate derivations.

EXAMPLE 3.1 A Grammar for a Small Language

```
<program> → begin <stmt_list> end

<stmt_list> → <stmt>
            | <stmt> ; <stmt_list>
<stmt> → <var> = <expression>
<var> → A | B | C
<expression> → <var> + <var>
             | <var> - <var>
             | <var>
```

The language described by the grammar of Example 3.1 has only one statement form: assignment. A program consists of the special word **begin**, followed by a list of statements separated by semicolons, followed by the special word **end**. An expression is either a single variable or two variables separated by either a + or - operator. The only variable names in this language are A, B, and C.

A derivation of a program in this language follows:

```
<program> => begin <stmt_list> end
          => begin <stmt> ; <stmt_list> end
          => begin <var> = <expression> ; <stmt_list> end
          => begin A = <expression> ; <stmt_list> end
          => begin A = <var> + <var> ; <stmt_list> end
          => begin A = B + <var> ; <stmt_list> end
          => begin A = B + C ; <stmt_list> end
          => begin A = B + ; <stmt> end
          => begin A = B + C ; <var> = <expression> end
          => begin A = B + C ; B = <expression> end
          => begin A = B + C ; B = <var> end
          => begin A = B + C ; B = C end
```

This derivation, like all derivations, begins with the start symbol, in this case <program>. The symbol => is read "derives." Each successive string in the sequence is derived from the previous string by replacing one of the nonterminals with one of that nonterminal's definitions. Each of the strings in the derivation, including <program>, is called a **sentential form**.

In this derivation, the replaced nonterminal is always the leftmost nonterminal in the previous sentential form. Derivations that use this order of replacement are called **leftmost derivations**. The derivation continues until the sentential form contains no nonterminals. That sentential form, consisting of only terminals, or lexemes, is the generated sentence.

In addition to leftmost, a derivation may be rightmost or in an order that is neither leftmost nor rightmost. Derivation order has no effect on the language generated by a grammar.

By choosing alternative RHSs of rules with which to replace nonterminals in the derivation, different sentences in the language can be generated. By exhaustively choosing all combinations of choices, the entire language can be generated. This language, like most others, is infinite, so one cannot generate *all* the sentences in the language in finite time.

Example 3.2 is another example of a grammar for part of a typical programming language.

EXAMPLE 3.2 A Grammar for Simple Assignment Statements

<assign> → <id> = <expr>
<id> → A| B | C
<expr> → <id> + <expr>
| <id> * <expr>
| (<expr>)
| <id>

The grammar of Example 3.2 describes assignment statements whose right sides are arithmetic expressions with multiplication and addition operators and parentheses. For example, the statement

```
A = B * ( A + C )
```

is generated by the leftmost derivation:

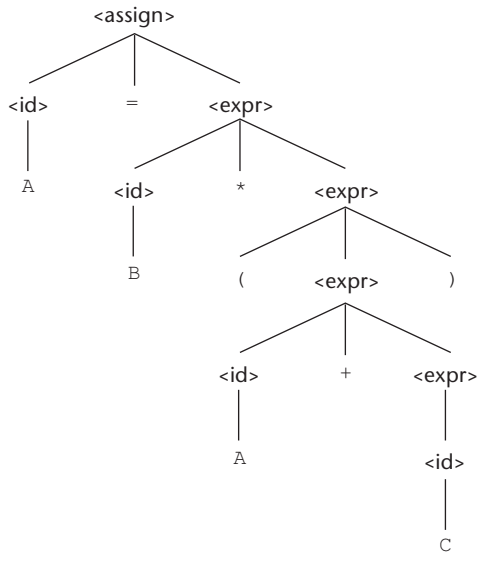
<assign> => <id> = <expr>
=> A = <expr>
=> A = <id> * <expr>
=> A = B * <expr>
=> A = B * (<expr>)
=> A = B * (<id> + <expr>)
=> A = B * (A + <expr>)
=> A = B * (A + <id>)
=> A = B * (A + C)

3.3.1.6 Parse Trees

One of the most attractive features of grammars is that they naturally describe the hierarchical syntactic structure of the sentences of the languages they define. These hierarchical structures are called **parse trees**. For example, the parse tree in Figure 3.1 shows the structure of the assignment statement derived previously.

Figure 3.1

A parse tree for the simple statement `A = B * (A + C)`



Every internal node of a parse tree is labeled with a nonterminal symbol; every leaf is labeled with a terminal symbol. Every subtree of a parse tree describes one instance of an abstraction in the sentence.

3.3.1.7 Ambiguity

A grammar that generates a sentential form for which there are two or more distinct parse trees is said to be **ambiguous**. Consider the grammar shown in Example 3.3, which is a minor variation of the grammar shown in Example 3.2.

EXAMPLE 3.3 An Ambiguous Grammar for Simple Assignment Statements

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <expr>
         | <expr> * <expr>
         | ( <expr> )
         | <id>
```

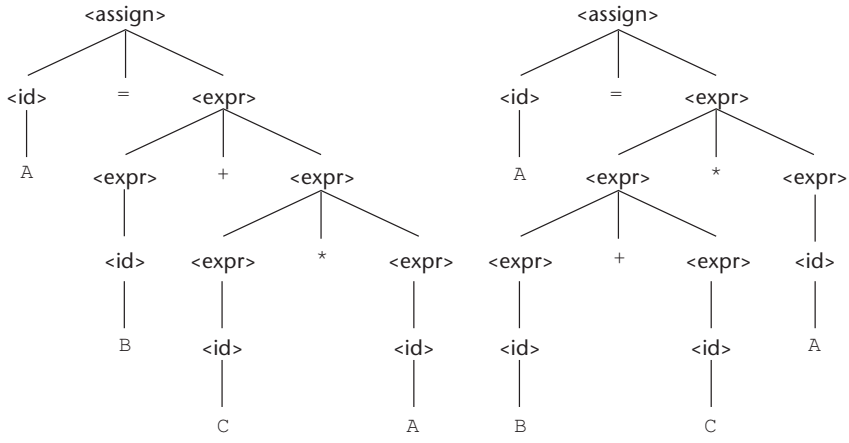
The grammar of Example 3.3 is ambiguous because the sentence

```
A = B + C * A
```

has two distinct parse trees, as shown in Figure 3.2. The ambiguity occurs because the grammar specifies slightly less syntactic structure than does the grammar of

Figure 3.2

Two distinct parse trees for the same sentence, `A = B + C * A`



Example 3.2. Rather than allowing the parse tree of an expression to grow only on the right, this grammar allows growth on both the left and the right.

Syntactic ambiguity of language structures is a problem because compilers often base the semantics of those structures on their syntactic form. Specifically, the compiler chooses the code to be generated for a statement by examining its parse tree. If a language structure has more than one parse tree, then the meaning of the structure cannot be determined uniquely. This problem is discussed in two specific examples in the following subsections.

There are several other characteristics of a grammar that are sometimes useful in determining whether a grammar is ambiguous.[1] They include the following: (1) if the grammar generates a sentence with more than one leftmost derivation and (2) if the grammar generates a sentence with more than one rightmost derivation.

Some parsing algorithms can be based on ambiguous grammars. When such a parser encounters an ambiguous construct, it uses nongrammatical information provided by the designer to construct the correct parse tree. In many cases, an ambiguous grammar can be rewritten to be unambiguous but still generate the desired language.

3.3.1.8 Operator Precedence

When an expression includes two different operators, for example, `x + y * z`, one obvious semantic issue is the order of evaluation of the two operators (for example, in this expression is it add and then multiply, or vice versa?). This semantic question can be answered by assigning different precedence levels to operators. For example, if `*` has been assigned higher precedence than `+` (by the language designer), multiplication will be done first, regardless of the order of appearance of the two operators in the expression.

1. Note that it is mathematically impossible to determine whether an arbitrary grammar is ambiguous.

As stated previously, a grammar can describe a certain syntactic structure so that part of the meaning of the structure can be determined from its parse tree. In particular, the fact that an operator in an arithmetic expression is generated lower in the parse tree (and therefore must be evaluated first) can be used to indicate that it has precedence over an operator produced higher up in the tree. In the first parse tree of Figure 3.2, for example, the multiplication operator is generated lower in the tree, which could indicate that it has precedence over the addition operator in the expression. The second parse tree, however, indicates just the opposite. It appears, therefore, that the two parse trees indicate conflicting precedence information.

Notice that although the grammar of Example 3.2 is not ambiguous, the precedence order of its operators is not the usual one. In this grammar, a parse tree of a sentence with multiple operators, regardless of the particular operators involved, has the rightmost operator in the expression at the lowest point in the parse tree, with the other operators in the tree moving progressively higher as one moves to the left in the expression. For example, in the expression `A + B * C`, `*` is the lowest in the tree, indicating it is to be done first. However, in the expression `A * B + C`, `+` is the lowest, indicating it is to be done first.

A grammar can be written for the simple expressions we have been discussing that is both unambiguous and specifies a consistent precedence of the `+` and `*` operators, regardless of the order in which the operators appear in an expression. The correct ordering is specified by using separate nonterminal symbols to represent the operands of the operators that have different precedence. This requires additional nonterminals and some new rules. Instead of using <expr> for both operands of both `+` and `*`, we could use three nonterminals to represent operands, which allows the grammar to force different operators to different levels in the parse tree. If <expr> is the root symbol for expressions, `+` can be forced to the top of the parse tree by having <expr> directly generate only `+` operators, using the new nonterminal, <term>, as the right operand of `+`. Next, we can define <term> to generate `*` operators, using <term> as the left operand and a new nonterminal, <factor>, as its right operand. Now, `*` will always be lower in the parse tree, simply because it is farther from the start symbol than `+` in every derivation. The grammar of Example 3.4 is such a grammar.

EXAMPLE 3.4 An Unambiguous Grammar for Expressions

<assign> → <id> = <expr>
<id> → `A | B | C`
<expr> → <expr> + <term>
| <term>
<term> → <term> * <factor>
| <factor>
<factor> → (<expr>)
| <id>

The grammar in Example 3.4 generates the same language as the grammars of Examples 3.2 and 3.3, but it is unambiguous and it specifies the usual precedence order of multiplication and addition operators. The following derivation of the sentence `A = B + C * A` uses the grammar of Example 3.4:

<assign> => <id> = <expr>
=> A = <expr>
=> A = <expr> + <term>
=> A = <term> + <term>
=> A = <factor> + <term>
=> A = <id> + <term>
=> A = B + <term>
=> A = B + <term> * <factor>
=> A = B + <factor> * <factor>
=> A = B + <id> * <factor>
=> A = B + C * <factor>
=> A = B + C * <id>
=> A = B + C * A

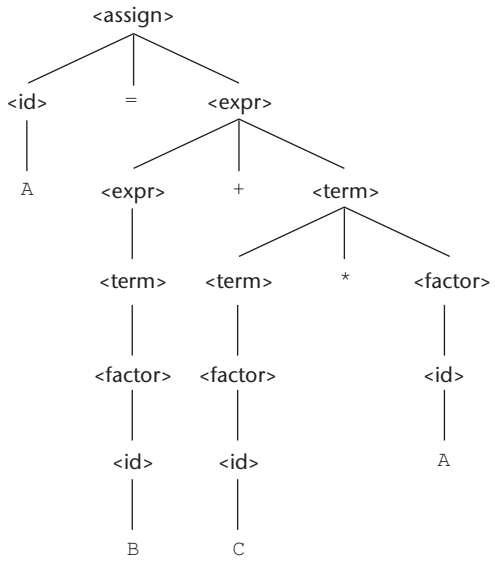
The unique parse tree for this sentence, using the grammar of Example 3.4, is shown in Figure 3.3.

The connection between parse trees and derivations is very close: Either can easily be constructed from the other. Every derivation with an unambiguous grammar has a unique parse tree, although that tree can be represented by different derivations. For example, the following derivation of the sentence `A = B + C * A` is different from the derivation of the same sentence given previously. This is a rightmost derivation, whereas the previous one is leftmost. Both of these derivations, however, are represented by the same parse tree.

<assign> => <id> = <expr>
=> <id> = <expr> + <term>
=> <id> = <expr> + <term> * <factor>
=> <id> = <expr> + <term> * <id>
=> <id> = <expr> + <term> * A
=> <id> = <expr> + <factor> * A
=> <id> = <expr> + <id> * A
=> <id> = <expr> + C * A
=> <id> = <term> + C * A
=> <id> = <factor> + C * A
=> <id> = <id> + C * A
=> <id> = B + C * A
=> A = B + C * A

Figure 3.3

The unique parse tree for `A = B + C * A` using an unambiguous grammar



3.3.1.9 Associativity of Operators

When an expression includes two operators that have the same precedence (as `*` and `/` usually have)—for example, `A / B * C`—a semantic rule is required to specify which should have precedence.[2] This rule is named *associativity*.

As was the case with precedence, a grammar for expressions may correctly imply operator associativity. Consider the following example of an assignment statement:

```
A = B + C + A
```

The parse tree for this sentence, as defined with the grammar of Example 3.4, is shown in Figure 3.4.

The parse tree of Figure 3.4 shows the left addition operator lower than the right addition operator. This is the correct order if addition is meant to be left associative, which is typical. In most cases, the associativity of addition in a computer is irrelevant. In mathematics, addition is associative, which means that left and right associative orders of evaluation mean the same thing. That is, `(A + B) + C = A + (B + C)`. Floating-point addition in a computer, however, is not necessarily associative. For example, suppose floating-point values store seven digits of accuracy. Consider the problem of adding 11 numbers together, where one of the numbers is 10^7 and the other ten are 1. If the small numbers (the 1's) are each added to the large number, one at a time, there is no effect on that number, because the small numbers occur in the eighth digit of the large number. However, if the small numbers are first added together and the result

2. An expression with two occurrences of the same operator has the same issue; for example, `A / B / C`.

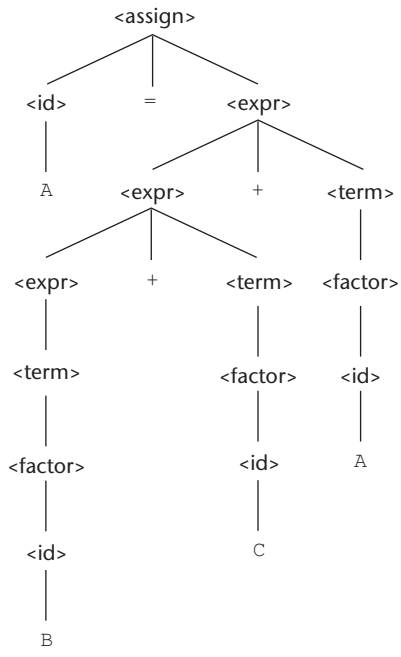


Figure 3.4

A parse tree for A = B + C + A illustrating the associativity of addition

is added to the large number, the result in seven-digit accuracy is $1.000001 * 10^7$. Subtraction and division are not associative, whether in mathematics or in a computer. Therefore, correct associativity may be essential for an expression that contains either of them.

When a grammar rule has its LHS also appearing at the beginning of its RHS, the rule is said to be **left recursive**. This left recursion specifies left associativity. For example, the left recursion of the rules of the grammar of Example 3.4 causes it to make both addition and multiplication left associative. Unfortunately, left recursion disallows the use of some important syntax analysis algorithms. When one of these algorithms is to be used, the grammar must be modified to remove the left recursion. This, in turn, disallows the grammar from precisely specifying that certain operators are left associative. Fortunately, left associativity can be enforced by the compiler, even though the grammar does not dictate it.

In most languages that provide it, the exponentiation operator is right associative. To indicate right associativity, right recursion can be used. A grammar rule is **right recursive** if the LHS appears at the right end of the RHS. Rules such as

<factor> → <exp> ** <factor>
| <exp>
<exp> → (<expr>)
| id

could be used to describe exponentiation as a right-associative operator.

3.3.1.10 An Unambiguous Grammar for `if-else`

The BNF rules for a Java **`if-else`** statement are as follows:

<if_stmt> → **`if`** (<logic_expr>) <stmt>
`if` (<logic_expr>) <stmt> **`else`** <stmt>

If we also have <stmt> → <if_stmt>, this grammar is ambiguous. The simplest sentential form that illustrates this ambiguity is

`if` (<logic_expr>) **`if`** (<logic_expr>) <stmt> **`else`** <stmt>

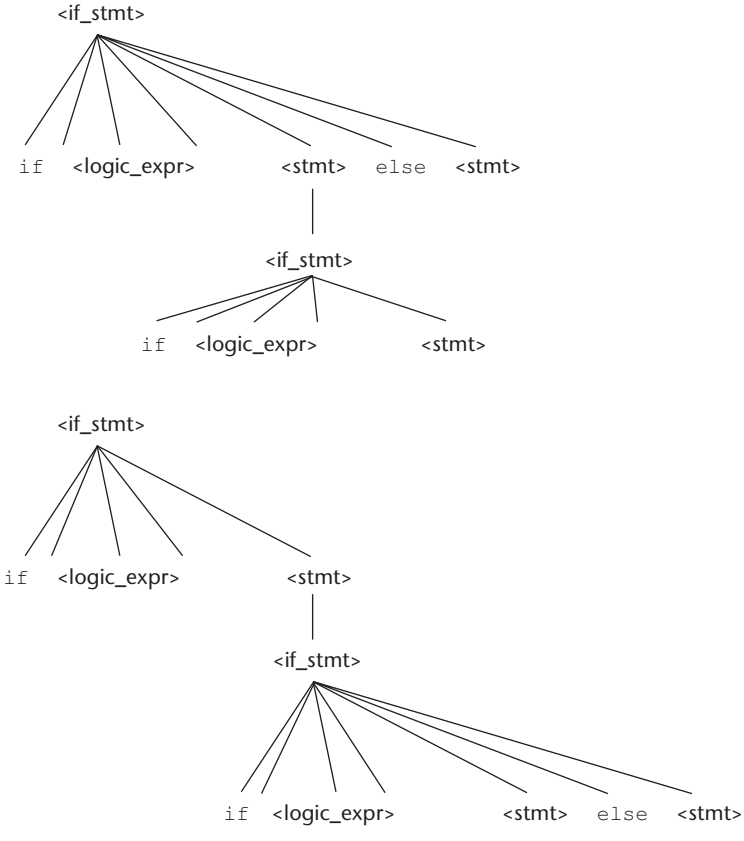
The two parse trees in Figure 3.5 show the ambiguity of this sentential form. Consider the following example of this construct:

```
if (done == true)
if (denom == 0)
    quotient = 0;
    else quotient = num / denom;
```

The problem is that if the upper parse tree in Figure 3.5 is used as the basis for translation, the else clause would be executed when `done` is not true, which

Figure 3.5

Two distinct parse trees for the same sentential form



probably is not what was intended by the author of the construct. We will examine the practical problems associated with this else-association problem in Chapter 8.

We will now develop an unambiguous grammar that describes this **if** statement. The rule for **if** constructs in many languages is that an else clause, when present, is matched with the nearest previous unmatched then clause. Therefore, there cannot be an **if** statement without an **else** between a then clause and its matching **else**. So, for this situation, statements must be distinguished between those that are matched and those that are unmatched, where unmatched statements are **else**-less **if**s and all other statements are matched. The problem with the earlier grammar is that it treats all statements as if they had equal syntactic significance—that is, as if they were all matched.

To reflect the different categories of statements, different abstractions, or nonterminals, must be used. The unambiguous grammar based on these ideas follows:

<stmt> → <matched> | <unmatched>

<matched> → **if** (<logic_expr>) <matched> **else** <matched>
| any non-if statement

<unmatched> → **if** (<logic_expr>) <stmt>
| **if** (<logic_expr>) <matched> **else** <unmatched>

There is just one possible parse tree, using this grammar, for the following sentential form:

if (<logic_expr>) **if** (<logic_expr>) <stmt> **else** <stmt>

3.3.2 Extended BNF

Because of a few minor inconveniences in BNF, it has been extended in several ways. Most extended versions are called Extended BNF, or simply EBNF, even though they are not all exactly the same. The extensions do not enhance the descriptive power of BNF; they only increase its readability and writability.

Three extensions are commonly included in the various versions of EBNF. The first of these denotes an optional part of an RHS, which is delimited by brackets. For example, a C **if-else** statement can be described as

<if_stmt> → **if** (<expression>) <statement> [**else** <statement>]

Without the use of the brackets, the syntactic description of this statement would require the following two rules:

<if_stmt> → **if** (<expression>) <statement>
| **if** (<expression>) <statement> **else** <statement>

The second extension is the use of braces in a RHS to indicate that the enclosed part can be repeated indefinitely or left out altogether. This extension allows lists to be built with a single rule, instead of using recursion and two rules.

For example, lists of identifiers separated by commas can be described by the following rule:

<ident_list> → <identifier> {, <identifier>}

This is a replacement of the recursion by a form of implied iteration; the part enclosed within braces can be iterated any number of times.

The third common extension deals with multiple-choice options. When a single element must be chosen from a group, the options are placed in parentheses and separated by the OR operator, |. For example,

<term> → <term> (* | / | %) <factor>

In BNF, a description of this <term> would require the following three rules:

<term> → <term> * <factor>
| <term> / <factor>
| <term> % <factor>

The brackets, braces, and parentheses in the EBNF extensions are **metasymbols**, which means they are notational tools and not terminal symbols in the syntactic entities they help describe. In cases where these metasymbols are also terminal symbols in the language being described, the instances that are terminal symbols can be underlined or quoted. Example 3.5 illustrates the use of braces and multiple choices in an EBNF grammar.

EXAMPLE 3.5 BNF and EBNF Versions of an Expression Grammar

BNF:

<expr> → <expr> + <term>
| <expr> - <term>
| <term>
<term> → <term> * <factor>
| <term> / <factor>
| <factor>
<factor> → <exp> ** <factor>
<exp>
<exp> → (<expr>)
| id

EBNF:

<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
<factor> → <exp> { ** <exp>}
<exp> → (<expr>)
| id

The BNF rule

<expr> → <expr> + <term>

clearly specifies—in fact forces—the + operator to be left associative. However, the EBNF version,

<expr> → <term> {+ <term>}

does not imply the direction of associativity. This problem is overcome in a syntax analyzer based on an EBNF grammar for expressions by designing the syntax analysis process to enforce the correct associativity. This is discussed further in Chapter 4.

Some versions of EBNF allow a numeric superscript to be attached to the right brace to indicate an upper limit to the number of times the enclosed part can be repeated. Also, some versions use a plus (+) superscript to indicate one or more repetitions. For example,

<compound> → **begin** <stmt> {<stmt>} **end**

and

<compound> → **begin** {<stmt>}$^{+}$ **end**

are equivalent.

In recent years, some variations on BNF and EBNF have appeared. Among these are the following:

- In place of the arrow, a colon is used and the RHS is placed on the next line.
- Instead of a vertical bar to separate alternative RHSs, they are simply placed on separate lines.
- In place of square brackets to indicate something being optional, the subscript opt is used. For example,
- Constructor Declarator → SimpleName (FormalParameterList$_{opt}$)
- Rather than using the | symbol in a parenthesized list of elements to indicate a choice, the words "one of" are used. For example,

AssignmentOperator → one of = *= /= %= += -=
<<= >>= &= ^= |=

There is a standard for EBNF, ISO/IEC 14977:1996 (1996), but it is rarely used. The standard uses the equal sign (=) instead of an arrow in rules, terminates each RHS with a semicolon, and requires quotes on all terminal symbols. It also specifies a host of other notational rules.

3.3.3 Grammars and Recognizers

Earlier in this chapter, we suggested that there is a close relationship between generation and recognition devices for a given language. In fact, given a context-free grammar, a recognizer for the language generated by the grammar

can be algorithmically constructed. A number of software systems have been developed that perform this construction. Such systems allow the quick creation of the syntax analysis part of a compiler for a new language and are therefore quite valuable. One of the first of these syntax analyzer generators is named yacc (*y*et *a*nother *c*ompiler *c*ompiler) (Johnson, 1975). There are now many such systems available.

3.4 Attribute Grammars

An **attribute grammar** is a device used to describe more of the structure of a programming language than can be described with a context-free grammar. An attribute grammar is an extension to a context-free grammar. The extension allows certain language rules to be conveniently described, such as type compatibility. Before we formally define the form of attribute grammars, we must clarify the concept of static semantics.

history note

Attribute grammars have been used in a wide variety of applications. They have been used to provide complete descriptions of the syntax and static semantics of programming languages (Watt, 1979); they have been used as the formal definition of a language that can be input to a compiler generation system (Farrow, 1982); and they have been used as the basis of several syntax-directed editing systems (Teitelbaum and Reps, 1981; Fischer et al., 1984). In addition, attribute grammars have been used in natural-language processing systems (Correa, 1992).

3.4.1 Static Semantics

There are some characteristics of programming languages that are difficult to describe with BNF, and some that are impossible. As an example of a syntax rule that is difficult to specify with BNF, consider type compatibility rules. In Java, for example, a floating-point value cannot be assigned to an integer type variable, although the opposite is legal. Although this restriction can be specified in BNF, it requires additional nonterminal symbols and rules. If all of the typing rules of Java were specified in BNF, the grammar would become too large to be useful, because the size of the grammar determines the size of the syntax analyzer.

As an example of a syntax rule that cannot be specified in BNF, consider the common rule that all variables must be declared before they are referenced. It has been proven that this rule cannot be specified in BNF.

These problems exemplify the categories of language rules called static semantics rules. The **static semantics** of a language is only indirectly related to the meaning of programs during execution; rather, it has to do with the legal forms of programs (syntax rather than semantics). Many static semantic rules of a language state its type constraints. Static semantics is so named because the analysis required to check these specifications can be done at compile time.

Because of the problems of describing static semantics with BNF, a variety of more powerful mechanisms has been devised for that task. One such mechanism, attribute grammars, was designed by Knuth (1968) to describe both the syntax and the static semantics of programs.

Attribute grammars are a formal approach both to describing and checking the correctness of the static semantics rules of a program. Although they are not

always used in a formal way in compiler design, the basic concepts of attribute grammars are at least informally used in every compiler (see Aho et al., 1988).

Dynamic semantics, which is the meaning of expressions, statements, and program units, is discussed in Section 3.5.

3.4.2 Basic Concepts

Attribute grammars are context-free grammars to which have been added attributes, attribute computation functions, and predicate functions. **Attributes**, which are associated with grammar symbols (the terminal and nonterminal symbols), are similar to variables in the sense that they can have values assigned to them. **Attribute computation functions**, sometimes called semantic functions, are associated with grammar rules. They are used to specify how attribute values are computed. **Predicate functions**, which state the static semantic rules of the language, are associated with grammar rules.

These concepts will become clearer after we formally define attribute grammars and provide an example.

3.4.3 Attribute Grammars Defined

An attribute grammar is a grammar with the following additional features:

- Associated with each grammar symbol X is a set of attributes A(X). The set A(X) consists of two disjoint sets S(X) and I(X), called synthesized and inherited attributes, respectively. **Synthesized attributes** are used to pass semantic information up a parse tree, while **inherited attributes** pass semantic information down and across a tree.
- Associated with each grammar rule is a set of semantic functions and a possibly empty set of predicate functions over the attributes of the symbols in the grammar rule. For a rule the $X_0 \rightarrow X_1 \ldots X_n$, synthesized attributes of X_0 are computed with semantic functions of the form $S(X_0) = f(A(X_1), \ldots, A(X_n))$. So the value of a synthesized attribute on a parse tree node depends only on the values of the attributes on that node's children nodes. Inherited attributes of symbols X_j, $1 \leq j \leq n$ (in the rule above), are computed with a semantic function of the form $I(X_j = f(A(X_0), \ldots, A(X_n))$. So the value of an inherited attribute on a parse tree node depends on the attribute values of that node's parent node and those of its sibling nodes. Note that, to avoid circularity, inherited attributes are often restricted to functions of the form $I(X_j) = f(A(X_0), \ldots, A(X_{(j-1)}))$. This form prevents an inherited attribute from depending on itself or on attributes to the right in the parse tree.
- A predicate function has the form of a Boolean expression on the union of the attribute set $\{A(X_0), \ldots, A(X_n)\}$ and a set of literal attribute values. The only derivations allowed with an attribute grammar are those in which every predicate associated with every nonterminal is true. A false predicate function value indicates a violation of the syntax or static semantics rules of the language.

A parse tree of an attribute grammar is the parse tree based on its underlying BNF grammar, with a possibly empty set of attribute values attached to each node. If all the attribute values in a parse tree have been computed, the tree is said to be **fully attributed**. Although in practice it is not always done this way, it is convenient to think of attribute values as being computed after the complete unattributed parse tree has been constructed by the compiler.

3.4.4 Intrinsic Attributes

Intrinsic attributes are synthesized attributes of leaf nodes whose values are determined outside the parse tree. For example, the type of an instance of a variable in a program could come from the symbol table, which is used to store variable names and their types. The contents of the symbol table are set based on earlier declaration statements. Initially, assuming that an unattributed parse tree has been constructed and that attribute values are needed, the only attributes with values are the intrinsic attributes of the leaf nodes. Given the intrinsic attribute values on a parse tree, the semantic functions can be used to compute the remaining attribute values.

3.4.5 Examples of Attribute Grammars

As a very simple example of how attribute grammars can be used to describe static semantics, consider the following fragment of an attribute grammar that describes the rule that the name on the **`end`** of an Ada procedure must match the procedure's name. (This rule cannot be stated in BNF.) The string attribute of <proc_name>, denoted by <proc_name>.string, is the actual string of characters that were found immediately following the reserved word **`procedure`** by the compiler. Notice that when there is more than one occurrence of a nonterminal in a syntax rule in an attribute grammar, the nonterminals are subscripted with brackets to distinguish them. Neither the subscripts nor the brackets are part of the described language.

Syntax rule: <proc_def> → **`procedure`** <proc_name>[1]
<proc_body> **`end`** <proc_name>[2];

Predicate: <proc_name>[1]string == <proc_name>[2].string

In this example, the predicate rule states that the name string attribute of the <proc_name> nonterminal in the subprogram header must match the name string attribute of the <proc_name> nonterminal following the end of the subprogram.

Next, we consider a larger example of an attribute grammar. In this case, the example illustrates how an attribute grammar can be used to check the type rules of a simple assignment statement. The syntax and static semantics of this assignment statement are as follows: The only variable names are `A`, `B`, and `C`. The right side of the assignments can be either a variable or an expression in the form of a variable added to another variable. The variables can be one of two types: int or real. When there are two variables on the right side of an assignment, they need not be the same type. The type of the expression when

the operand types are not the same is always real. When they are the same, the expression type is that of the operands. The type of the left side of the assignment must match the type of the right side. So the types of operands in the right side can be mixed, but the assignment is valid only if the target and the value resulting from evaluating the right side have the same type. The attribute grammar specifies these static semantic rules.

The syntax portion of our example attribute grammar is

<assign> → <var> = <expr>
<expr> → <var> + <var>
| <var>
<var> → A | B | C

The attributes for the nonterminals in the example attribute grammar are described in the following paragraphs:

- *actual_type*—A synthesized attribute associated with the nonterminals <var> and <expr>. It is used to store the actual type, int or real, of a variable or expression. In the case of a variable, the actual type is intrinsic. In the case of an expression, it is determined from the actual types of the child node or children nodes of the <expr> nonterminal.
- *expected_type*—An inherited attribute associated with the nonterminal <expr>. It is used to store the type, either int or real, that is expected for the expression, as determined by the type of the variable on the left side of the assignment statement.

The complete attribute grammar follows in Example 3.6.

EXAMPLE 3.6 An Attribute Grammar for Simple Assignment Statements

1. Syntax rule: <assign> → <var> = <expr>
   Semantic rule: <expr>.expected_type ← <var>.actual_type
2. Syntax rule: <expr> → <var>[2] + <var>[3]
   Semantic rule: <expr>.actual_type ←
   if (<var>[2].actual_type = int) and
   (<var>[3].actual_type = int)
   then int
   else real
   end if
   Predicate: <expr>.actual_type == <expr>.expected_type
3. Syntax rule: <expr> → <var>
   Semantic rule: <expr>.actual_type ← <var>.actual_type
   Predicate: <expr>.actual_type == <expr>.expected_type
4. Syntax rule: <var> → A | B | C
   Semantic rule: <var>.actual_type ← look-up (<var>.string)

The look-up function looks up a given variable name in the symbol table and returns the variable's type.

A parse tree of the sentence `A = A + B` generated by the grammar in Example 3.6 is shown in Figure 3.6. As in the grammar, bracketed numbers are added after the repeated node labels in the tree so they can be referenced unambiguously.

3.4.6 Computing Attribute Values

Now, consider the process of computing the attribute values of a parse tree, which is sometimes called **decorating** the parse tree. If all attributes were inherited, this could proceed in a completely top-down order, from the root to the leaves. Alternatively, it could proceed in a completely bottom-up order, from the leaves to the root, if all the attributes were synthesized. Because our grammar has both synthesized and inherited attributes, the evaluation process cannot be in any single direction. The following is an evaluation of the attributes, in an order in which it is possible to compute them:

1. <var>.actual_type ← look-up(`A`) (Rule 4)
2. <expr>.expected_type ← <var>.actual_type (Rule 1)
3. <var>[2].actual_type ← look-up(`A`) (Rule 4)
   <var>[3].actual_type ← look-up(`B`) (Rule 4)
4. <expr>.actual_type ← either int or real (Rule 2)
5. <expr>.expected_type == <expr>.actual_type is either
   TRUE or FALSE (Rule 2)

The tree in Figure 3.7 shows the flow of attribute values in the example of Figure 3.6. Solid lines show the parse tree; dashed lines show attribute flow in the tree.

The tree in Figure 3.8 shows the final attribute values on the nodes. In this example, `A` is defined as a real and `B` is defined as an int.

Determining attribute evaluation order for the general case of an attribute grammar is a complex problem, requiring the construction of a dependency graph to show all attribute dependencies.

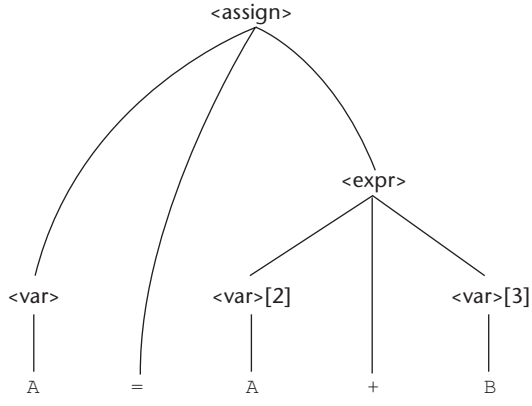


Figure 3.6
A parse tree for
`A = A + B`

Figure 3.7

The flow of attributes in the tree

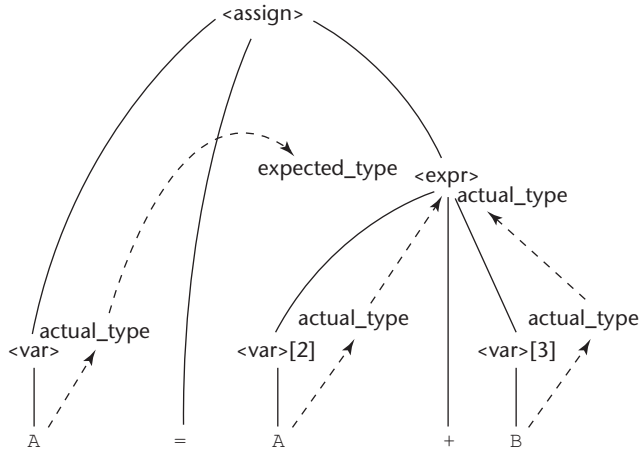
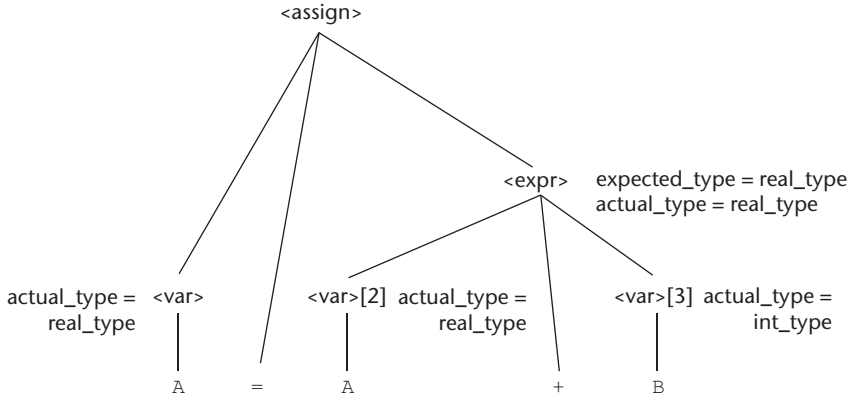


Figure 3.8

A fully attributed parse tree



3.4.7 Evaluation

Checking the static semantic rules of a language is an essential part of all compilers. Even if a compiler writer has never heard of an attribute grammar, he or she would need to use the fundamental ideas of attribute grammars to design the checks of static semantics rules for his or her compiler.

One of the main difficulties in using an attribute grammar to describe all of the syntax and static semantics of a real contemporary programming language is the size and complexity of the attribute grammar. The large number of attributes and semantic rules required for a complete programming language make such grammars difficult to write and read. Furthermore, the attribute values on a large parse tree are costly to evaluate. On the other hand, less formal attribute

grammars are a powerful and commonly used tool for compiler writers, who are more interested in the process of producing a compiler than they are in formalism.

3.5 Describing the Meanings of Programs: Dynamic Semantics

We now turn to the difficult task of describing the **dynamic semantics**, or meaning, of the expressions, statements, and program units of a programming language. Because of the power and naturalness of the available notation, describing syntax is a relatively simple matter. On the other hand, no universally accepted notation or approach has been devised for dynamic semantics. In this section, we briefly describe several of the methods that have been developed. For the remainder of this section, when we use the term *semantics*, we mean dynamic semantics.

There are several different reasons underlying the need for a methodology and notation for describing semantics. Programmers obviously need to know precisely what the statements of a language do before they can use them effectively in their programs. Compiler writers must know exactly what language constructs mean to design implementations for them correctly. If there were a precise semantics specification of a programming language, programs written in the language potentially could be proven correct without testing. Also, compilers could be shown to produce programs that exhibited exactly the behavior given in the language definition; that is, their correctness could be verified. A complete specification of the syntax and semantics of a programming language could be used by a tool to generate a compiler for the language automatically. Finally, language designers, who would develop the semantic descriptions of their languages, could in the process discover ambiguities and inconsistencies in their designs.

Software developers and compiler designers typically determine the semantics of programming languages by reading English explanations in language manuals. Because such explanations are often imprecise and incomplete, this approach is clearly unsatisfactory. Due to the lack of complete semantics specifications of programming languages, programs are rarely proven correct without testing, and commercial compilers are never generated automatically from language descriptions.

Scheme, a functional language described in Chapter 15, is one of only a few programming languages whose definition includes a formal semantics description. However, the method used is not one described in this chapter, as this chapter is focused on approaches that are suitable for imperative languages.

3.5.1 Operational Semantics

The idea behind **operational semantics** is to describe the meaning of a statement or program by specifying the effects of running it on a machine. The effects on the machine are viewed as the sequence of changes in its state, where

the machine's state is the collection of the values in its storage. An obvious operational semantics description, then, is given by executing a compiled version of the program on a computer. Most programmers have, on at least one occasion, written a small test program to determine the meaning of some programming language construct, often while learning the language. Essentially, what such a programmer is doing is using operational semantics to determine the meaning of the construct.

There are several problems with using this approach for complete formal semantics descriptions. First, the individual steps in the execution of machine language and the resulting changes to the state of the machine are too small and too numerous. Second, the storage of a real computer is too large and complex. There are usually several levels of memory devices, as well as connections to enumerable other computers and memory devices through networks. Therefore, machine languages and real computers are not used for formal operational semantics. Rather, intermediate-level languages and interpreters for idealized computers are designed specifically for the process.

There are different levels of uses of operational semantics. At the highest level, the interest is in the final result of the execution of a complete program. This is sometimes called **natural operational semantics**. At the lowest level, operational semantics can be used to determine the precise meaning of a program through an examination of the complete sequence of state changes that occur when the program is executed. This use is sometimes called **structural operational semantics**.

3.5.1.1 The Basic Process

The first step in creating an operational semantics description of a language is to design an appropriate intermediate language, where the primary desired characteristic of the language is clarity. Every construct of the intermediate language must have an obvious and unambiguous meaning. This language is at the intermediate level, because machine language is too low-level to be easily understood and another high-level language is obviously not suitable. If the semantics description is to be used for natural operational semantics, a virtual machine (an interpreter) must be constructed for the intermediate language. The virtual machine can be used to execute either single statements, code segments, or whole programs. The semantics description can be used without a virtual machine if the meaning of a single statement is all that is required. In this use, which is structural operational semantics, the intermediate code can be visually inspected.

The basic process of operational semantics is not unusual. In fact, the concept is frequently used in programming textbooks and programming language reference manuals. For example, the semantics of the C `for` construct can be described in terms of simpler statements, as in

C Statement	*Meaning*

`for (expr1; expr2; expr3) {`
`  ...`
`}`
	`expr1;`
	`loop: if expr2 == 0 goto out`
	`...`
	`expr3;`
	`goto loop`
	`out: ...`

The human reader of such a description is the virtual computer and is assumed to be able to "execute" the instructions in the definition correctly and recognize the effects of the "execution."

The intermediate language and its associated virtual machine used for formal operational semantics descriptions are often highly abstract. The intermediate language is meant to be convenient for the virtual machine, rather than for human readers. For our purposes, however, a more human-oriented intermediate language could be used. As such an example, consider the following list of statements, which would be adequate for describing the semantics of the simple control statements of a typical programming language:

ident = var
ident = ident + 1
ident = ident – 1
goto label
if var relop var **goto** label

In these statements, relop is one of the relational operators from the set {=, <>, >, <, <=, <=}, ident is an identifier, and var is either an identifier or a constant. These statements are all simple and therefore easy to understand and implement.

A slight generalization of these three assignment statements allows more general arithmetic expressions and assignment statements to be described. The new statements are

ident = var bin_op var
ident = un_op var

where bin_op is a binary arithmetic operator and un_op is a unary operator. Multiple arithmetic data types and automatic type conversions, of course, complicate this generalization. Adding just a few more relatively simple instructions would allow the semantics of arrays, records, pointers, and subprograms to be described.

In Chapter 8, the semantics of various control statements are described using this intermediate language.

3.5.1.2 Evaluation

The first and most significant use of formal operational semantics was to describe the semantics of PL/I (Wegner, 1972). That particular abstract machine and the translation rules for PL/I were together named the Vienna Definition Language (VDL), after the city where IBM designed it.

Operational semantics provides an effective means of describing semantics for language users and language implementors, as long as the descriptions are kept simple and informal. The VDL description of PL/I, unfortunately, is so complex that it serves no practical purpose.

Operational semantics depends on programming languages of lower levels, not mathematics. The statements of one programming language are described in terms of the statements of a lower-level programming language. This approach can lead to circularities, in which concepts are indirectly defined in terms of themselves. The methods described in the following two sections are much more formal, in the sense that they are based on mathematics and logic, not programming languages.

3.5.2 Denotational Semantics

Denotational semantics is the most rigorous and most widely known formal method for describing the meaning of programs. It is solidly based on recursive function theory. A thorough discussion of the use of denotational semantics to describe the semantics of programming languages is necessarily long and complex. It is our intent to provide the reader with an introduction to the central concepts of denotational semantics, along with a few simple examples that are relevant to programming language specifications.

The process of constructing a denotational semantics specification for a programming language requires one to define for each language entity both a mathematical object and a function that maps instances of that language entity onto instances of the mathematical object. Because the objects are rigorously defined, they model the exact meaning of their corresponding entities. The idea is based on the fact that there are rigorous ways of manipulating mathematical objects but not programming language constructs. The difficulty with this method lies in creating the objects and the mapping functions. The method is named *denotational* because the mathematical objects denote the meaning of their corresponding syntactic entities.

The mapping functions of a denotational semantics programming language specification, like all functions in mathematics, have a domain and a range. The domain is the collection of values that are legitimate parameters to the function; the range is the collection of objects to which the parameters are mapped. In denotational semantics, the domain is called the **syntactic domain**, because it is syntactic structures that are mapped. The range is called the **semantic domain**.

Denotational semantics is related to operational semantics. In operational semantics, programming language constructs are translated into simpler programming language constructs, which become the basis of the meaning of the construct. In denotational semantics, programming language constructs are mapped to mathematical objects, either sets or, more often, functions. However, unlike operational semantics, denotational semantics does not model the step-by-step computational processing of programs.

3.5.2.1 Two Simple Examples

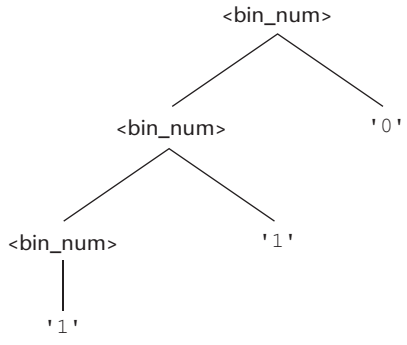
We use a very simple language construct, character string representations of binary numbers, to introduce the denotational method. The syntax of such binary numbers can be described by the following grammar rules:

```
<bin_num> → '0'
          | '1'
          | <bin_num> '0'
          | <bin_num> '1'
```

A parse tree for the example binary number, `110`, is shown in Figure 3.9. Notice that we put apostrophes around the syntactic digits to show they are not mathematical digits. This is similar to the relationship between ASCII coded digits and mathematical digits. When a program reads a number as a string, it must be converted to a mathematical number before it can be used as a value in the program.

Figure 3.9

A parse tree of the binary number `110`



The syntactic domain of the mapping function for binary numbers is the set of all character string representations of binary numbers. The semantic domain is the set of nonnegative decimal numbers, symbolized by N.

To describe the meaning of binary numbers using denotational semantics, we associate the actual meaning (a decimal number) with each rule that has a single terminal symbol as its RHS.

In our example, decimal numbers must be associated with the first two grammar rules. The other two grammar rules are, in a sense, computational rules, because they combine a terminal symbol, to which an object can be associated, with a nonterminal, which can be expected to represent some construct. Presuming an evaluation that progresses upward in the parse tree, the nonterminal in the right side would already have its meaning attached. So, a syntax rule with a nonterminal as its RHS would require a function that computed the meaning of the LHS, which represents the meaning of the complete RHS.

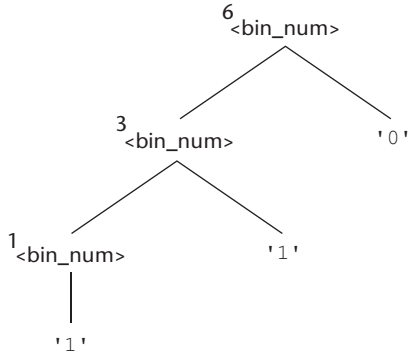
The semantic function, named M_{bin}, maps the syntactic objects, as described in the previous grammar rules, to the objects in N, the set of non-negative decimal numbers. The function M_{bin} is defined as follows:

M_{bin} ('0') = 0
M_{bin} ('1') = 1
M_{bin} (<bin_num> '0') = 2 * M_{bin} (<bin_num>)
M_{bin} (<bin_num>'1') = 2 * M_{bin} (<bin_num>) + 1

The meanings, or denoted objects (which in this case are decimal numbers), can be attached to the nodes of the parse tree shown on the previous page, yielding the tree in Figure 3.10. This is syntax-directed semantics. Syntactic entities are mapped to mathematical objects with concrete meaning.

Figure 3.10

A parse tree with denoted objects for 110



In part because we need it later, we now show a similar example for describing the meaning of syntactic decimal literals. In this case, the syntactic domain is the set of character string representations of decimal numbers. The semantic domain is once again the set N.

<dec_num> → '0'|'1'|'2'|'3'|'4'|'5'|'6'|'7''8'|'9'
|<dec_num> ('0'|'1'|'2'|'3'|'4'|'5'|'6'|'7'|'8'|'9')

The denotational mappings for these syntax rules are

M_{dec} ('0') = 0, M_{dec} ('1') = 1, M_{dec} ('2') = 2, . . ., M_{dec} ('9') = 9
M_{dec} (<dec_num> '0') = 10 * M_{dec} (<dec_num>)
M_{dec} (<dec_num> '1') = 10 * M_{dec} (<dec_num>) + 1
. . .
M_{dec} (<dec_num> '9') = 10 * M_{dec} (<dec_num>) + 9

In the following sections, we present the denotational semantics descriptions of a few simple constructs. The most important simplifying assumption made here is that both the syntax and static semantics of the constructs are correct. In addition, we assume that only two scalar types are included: integer and Boolean.

3.5.2.2 The State of a Program

The denotational semantics of a program could be defined in terms of state changes in an ideal computer. Operational semantics are defined in this way, and denotational semantics are defined in nearly the same way. In a further simplification, however, denotational semantics is defined in terms of only the values of all of the program's variables. So, denotational semantics uses the state of the program to describe meaning, whereas operational semantics uses the state of a machine. The key difference between operational semantics and denotational semantics is that state changes in operational semantics are defined by coded algorithms, written in some programming language, whereas in denotational semantics, state changes are defined by mathematical functions.

Let the state s of a program be represented as a set of ordered pairs, as follows:

$s = \{<i_1, v_1>, <i_2, v_2>, \ldots, <i_n, v_n>\}$

Each i is the name of a variable, and the associated v's are the current values of those variables. Any of the v's can have the special value **undef**, which indicates that its associated variable is currently undefined. Let VARMAP be a function of two parameters: a variable name and the program state. The value of VARMAP (i_j, s) is v_j (the value paired with i_j in state s). Most semantics mapping functions for programs and program constructs map states to states. These state changes are used to define the meanings of programs and program constructs. Some language constructs—for example, expressions—are mapped to values, not states.

3.5.2.3 Expressions

Expressions are fundamental to most programming languages. We assume here that expressions have no side effects. Furthermore, we deal with only very simple expressions: The only operators are + and *, and an expression can have at most one operator; the only operands are scalar integer variables and integer literals; there are no parentheses; and the value of an expression is an integer. Following is the BNF description of these expressions:

<expr> → <dec_num> | <var> | <binary_expr>
<binary_expr> → <left_expr> <operator> <right_expr>
<left_expr> → <dec_num> | <var>
<right_expr> → <dec_num> | <var>
<operator> → + | *

The only error we consider in expressions is a variable having an undefined value. Obviously, other errors can occur, but most of them are machine-dependent. Let Z be the set of integers, and let **error** be the error value. Then $Z \cup$ {**error**} is the semantic domain for the denotational specification for our expressions.

The mapping function for a given expression E and state s follows. To distinguish between mathematical function definitions and the assignment statements of programming languages, we use the symbol Δ = to define mathematical functions. The implication symbol, =>, used in this definition connects the form of an operand with its associated case (or switch) construct. Dot notation is used to refer to the child nodes of a node. For example, <binary_expr>.<left_expr> refers to the left child node of <binary_expr>.

M_e (<expr>, s) Δ= case <expr> of
<dec_num>=> M_{dec} (<dec_num>, s)
<var> =>if VARMAP (<var>, s) == **undef**
then **error**
else VARMAP (<var>, s)
<binary_expr> =>
if(M_e (<binary_expr>.<left_expr>,s) == **undef** OR
M_e (<binary_expr>.<right_expr>, s) == **undef**)
then **error**
else if (<binary_expr>.<operator> == '+')
then M_e (<binary_expr>.<left_expr>, s) +
M_e (<binary_expr>.<right_expr>, s)
else M_e (<binary_expr>.<left_expr>, s) *
M_e (<binary_expr>.<right_expr>, s)

3.5.2.4 Assignment Statements

An assignment statement is an expression evaluation plus the setting of the target variable to the expression's value. In this case, the meaning function maps a state to a state. This function can be described with the following:

M_a (x = E, s) Δ= if M_e (E, s) == **error**
then **error**
else s' = {<i_1, v_1' >, <i_2, v_2'>, . . . , <i_n, v_n'>}, where
for j = 1, 2, . . . , n
if i_j == x
then v_j' = M_e (E, s)
else v_j' = VARMAP (i_j,, s)

Note that the comparison in the third last line above, i_j == x, is of names, not values.

3.5.2.5 Logical Pretest Loops

The denotational semantics of a logical pretest loop is deceptively simple. To expedite the discussion, we assume that there are two other existing mapping functions, M_{sl} and M_b, that map statement lists and states to states and Boolean expressions to Boolean values (or **error**), respectively. The function is

$$
\begin{aligned}
M_l(\texttt{while}\ B\ \texttt{do}\ L, s) \Delta= & \text{if } M_b(B, s) == \textbf{undef} \\
& \text{then } \textbf{error} \\
& \text{else if } M_b(B, s) == \text{false} \\
& \quad \text{then } s \\
& \quad \text{else if } M_{sl}(L, s) == \textbf{error} \\
& \quad\quad \text{then } \textbf{error} \\
& \quad\quad \text{else } M_l(\texttt{while}\ B\ \texttt{do}\ L, M_{sl}(L, s))
\end{aligned}
$$

The meaning of the loop is simply the value of the program variables after the statements in the loop have been executed the prescribed number of times, assuming there have been no errors. In essence, the loop has been converted from iteration to recursion, where the recursion control is mathematically defined by other recursive state mapping functions. Recursion is easier to describe with mathematical rigor than iteration.

One significant observation at this point is that this definition, like actual program loops, may compute nothing because of nontermination.

3.5.2.6 Evaluation

Objects and functions, such as those used in the earlier constructs, can be defined for the other syntactic entities of programming languages. When a complete system has been defined for a given language, it can be used to determine the meaning of complete programs in that language. This provides a framework for thinking about programming in a highly rigorous way.

history note

A significant amount of work has been done on the possibility of using denotational language descriptions to generate compilers automatically (Jones, 1980; Milos et al., 1984; Bodwin et al., 1982). These efforts have shown that the method is feasible, but the work has never progressed to the point where it can be used to generate useful compilers.

As stated previously, denotational semantics can be used as an aid to language design. For example, statements for which the denotational semantic description is complex and difficult may indicate to the designer that such statements may also be difficult for language users to understand and that an alternative design may be in order.

Because of the complexity of denotational descriptions, they are of little use to language users. On the other hand, they provide an excellent way to describe a language concisely.

Although the use of denotational semantics is normally attributed to Scott and Strachey (1971), the general denotational approach to language description can be traced to the nineteenth century (Frege, 1892).

3.5.3 Axiomatic Semantics

Axiomatic semantics, thus named because it is based on mathematical logic, is the most abstract approach to semantics specification discussed in this chapter. Rather than directly specifying the meaning of a program, axiomatic semantics specifies what can be proven about the program. Recall that one of the possible uses of semantic specifications is to prove the correctness of programs.

In axiomatic semantics, there is no model of the state of a machine or program or model of state changes that take place when the program is executed. The meaning of a program is based on relationships among program variables and constants, which are the same for every execution of the program.

Axiomatic semantics has two distinct applications: program verification and program semantics specification. This section focuses on program verification in its description of axiomatic semantics.

Axiomatic semantics was defined in conjunction with the development of an approach to proving the correctness of programs. Such correctness proofs, when they can be constructed, show that a program performs the computation described by its specification. In a proof, each statement of a program is both preceded and followed by a logical expression that specifies constraints on program variables. These, rather than the entire state of an abstract machine (as with operational semantics), are used to specify the meaning of the statement. The notation used to describe constraints—indeed, the language of axiomatic semantics—is predicate calculus. Although simple Boolean expressions are often adequate to express constraints, in some cases they are not.

When axiomatic semantics is used to specify formally the meaning of a statement, the meaning is defined by the statement's effect on assertions about the data affected by the statement.

3.5.3.1 Assertions

The logical expressions used in axiomatic semantics are called predicates, or **assertions**. An assertion immediately preceding a program statement describes the constraints on the program variables at that point in the program. An assertion immediately following a statement describes the new constraints on those variables (and possibly others) after execution of the statement. These assertions are called the **precondition** and **postcondition**, respectively, of the statement. For two adjacent statements, the postcondition of the first serves as the precondition of the second. Developing an axiomatic description or proof of a given program requires that every statement in the program has both a precondition and a postcondition.

In the following sections, we examine assertions from the point of view that preconditions for statements are computed from given postconditions, although it is possible to consider these in the opposite sense. We assume all variables are integer type. As a simple example, consider the following assignment statement and postcondition:

```
sum = 2 * x + 1 {sum > 1}
```

Precondition and postcondition assertions are presented in braces to distinguish them from parts of program statements. One possible precondition for this statement is {`x > 10`}.

In axiomatic semantics, the meaning of a specific statement is defined by its precondition and its postcondition. In effect, the two assertions specify precisely the effect of executing the statement.

In the following subsections, we focus on correctness proofs of statements and programs, which is a common use of axiomatic semantics. The more general concept of axiomatic semantics is to state precisely the meaning of statements and programs in terms of logic expressions. Program verification is one application of axiomatic descriptions of languages.

3.5.3.2 Weakest Preconditions

The **weakest precondition** is the least restrictive precondition that will guarantee the validity of the associated postcondition. For example, in the statement and postcondition given in Section 3.5.3.1, {`x > 10`}, {`x > 50`}, and {`x > 1000`} are all valid preconditions. The weakest of all preconditions in this case is {`x > 0`}.

If the weakest precondition can be computed from the most general postcondition for each of the statement types of a language, then the processes used to compute these preconditions provide a concise description of the semantics of that language. Furthermore, correctness proofs can be constructed for programs in that language. A program proof is begun by using the characteristics of the results of the program's execution as the postcondition of the last statement of the program. This postcondition, along with the last statement, is used to compute the weakest precondition for the last statement. This precondition is then used as the postcondition for the second last statement. This process continues until the beginning of the program is reached. At that point, the precondition of the first statement states the conditions under which the program will compute the desired results. If these conditions are implied by the input specification of the program, the program has been verified to be correct.

An **inference rule** is a method of inferring the truth of one assertion on the basis of the values of other assertions. The general form of an inference rule is as follows:

$$\frac{\text{S1, S2, } \ldots \text{, S}n}{\text{S}}$$

This rule states that if S1, S2, . . . , and S*n* are true, then the truth of S can be inferred. The top part of an inference rule is called its **antecedent**; the bottom part is called its **consequent**.

An **axiom** is a logical statement that is assumed to be true. Therefore, an axiom is an inference rule without an antecedent.

For some program statements, the computation of a weakest precondition from the statement and a postcondition is simple and can be specified by an axiom. In most cases, however, the weakest precondition can be specified only by an inference rule.

To use axiomatic semantics with a given programming language, whether for correctness proofs or for formal semantics specifications, either an axiom or an inference rule must exist for each kind of statement in the language. In the following subsections, we present an axiom for assignment

statements and inference rules for statement sequences, selection statements, and logical pretest loop statements. Note that we assume that neither arithmetic nor Boolean expressions have side effects.

3.5.3.3 Assignment Statements

The precondition and postcondition of an assignment statement together define its meaning. To define the meaning of an assignment statement there must be a way to compute its precondition from its postcondition.

Let x = E be a general assignment statement and Q be its postcondition. Then, its weakest precondition, P, is defined by the axiom

$$P = Q_{x \rightarrow E}$$

which means that P is computed as Q with all instances of x replaced by E. For example, if we have the assignment statement and postcondition

```
a = b / 2 - 1 {a < 10}
```

the weakest precondition is computed by substituting `b / 2 - 1` for `a` in the postcondition {`a < 10`}, as follows:

```
b / 2 - 1 < 10
b < 22
```

Thus, the weakest precondition for the given assignment statement and postcondition is {`b < 22`}. Remember that the assignment axiom is guaranteed to be correct only in the absence of side effects. An assignment statement has a side effect if it changes some variable other than its target.

The usual notation for specifying the axiomatic semantics of a given statement form is

$$\{P\}\ S\ \{Q\}$$

where P is the precondition, Q is the postcondition, and S is the statement form. In the case of the assignment statement, the notation is

$$\{Q_{x \rightarrow E}\}\ x = E \{Q\}$$

As another example of computing a precondition for an assignment statement, consider the following:

```
x = 2 * y - 3 {x > 25}
```

The precondition is computed as follows:

```
2 * y - 3 > 25
y > 14
```

So {y > 14} is the weakest precondition for this assignment statement and postcondition.

Note that the appearance of the left side of the assignment statement in its right side does not affect the process of computing the weakest precondition. For example, for

```
x = x + y - 3 {x > 10}
```

the weakest precondition is

```
x + y - 3 > 10
y > 13 - x
```

Recall that axiomatic semantics was developed to prove the correctness of programs. In light of that, it is natural at this point to wonder how the axiom for assignment statements can be used to prove anything. Here is how: A given assignment statement with both a precondition and a postcondition can be considered a logical statement, or theorem. If the assignment axiom, when applied to the postcondition and the assignment statement, produces the given precondition, the theorem is proved. For example, consider the following logical statement:

```
{x > 3} x = x - 3 {x > 0}
```

Using the assignment axiom on the statement and its postcondition produces {x > 3}, which is the given precondition. Therefore, we have proven the example logical statement.

Next, consider the following logical statement:

```
{x > 5} x = x - 3 {x > 0}
```

In this case, the given precondition, {x > 5}, is not the same as the assertion produced by the axiom. However, it is obvious that {x > 5} implies {x > 3}. To use this in a proof, an inference rule named the **rule of consequence** is needed. The form of the rule of consequence is

$$\frac{\{P\}\ S\ \{Q\},\ P' \Rightarrow P,\ Q \Rightarrow Q'}{\{P'\}\ S\ \{Q'\}}$$

The => symbol means "implies," and S can be any program statement. The rule can be stated as follows: If the logical statement {P} S {Q} is true, the assertion P′ implies the assertion P, and the assertion Q implies the assertion Q′, then it can be inferred that {P′} S {Q′}. In other words, the rule of consequence says that a postcondition can always be weakened and a precondition can always be strengthened. This is quite useful in program proofs. For example, it allows the completion of the proof of the last logical statement example above. If we let P be {x > 3}, Q and Q′ be {x > 0}, and P′ be {x > 5}, we have

$$\frac{\{x > 3\}x = x - 3\{x > 0\}, (x > 5) => \{x > 3\}, (x > 0) => (x > 0)}{\{x > 5\}x = x - 3\{x > 0\}}$$

The first term of the antecedent (`{x > 3} x = x - 3 {x > 0}`) was proven with the assignment axiom. The second and third terms are obvious. Therefore, by the rule of consequence, the consequent is true.

3.5.3.4 Sequences

The weakest precondition for a sequence of statements cannot be described by an axiom, because the precondition depends on the particular kinds of statements in the sequence. In this case, the precondition can only be described with an inference rule. Let S1 and S2 be adjacent program statements. If S1 and S2 have the following pre- and postconditions

{P1} S1 {P2}
{P2} S2 {P3}

the inference rule for such a two-statement sequence is

$$\frac{\{P1\}\ S1\ \{P2\}, \{P2\}\ S2\ \{P3\}}{\{P1\}\ S1, S2\ \{P3\}}$$

So, for our example, {P1} S1; S2 {P3} describes the axiomatic semantics of the sequence S1; S2. The inference rule states that to get the sequence precondition, the precondition of the second statement is computed. This new assertion is then used as the postcondition of the first statement, which can then be used to compute the precondition of the first statement, which is also the precondition of the whole sequence. If S1 and S2 are the assignment statements

x1 = E1

and

x2 = E2

then we have

$\{P3_{x2 \rightarrow E2}\}$ x2 = E2 {P3}
$\{(P3_{x2 \rightarrow E2})_{x1 \rightarrow E_1}\}$ x1 = E1 $\{P3_{x2 \rightarrow E2}\}$

Therefore, the weakest precondition for the sequence x1 = E1; x2 = E2 with postcondition P3 is $\{(P3_{x2 \rightarrow E2})_{x1 \rightarrow E1}\}$.

For example, consider the following sequence and postcondition:

```
y = 3 * x + 1;
x = y + 3;
{x < 10}
```

The precondition for the second assignment statement is

```
y < 7
```

which is used as the postcondition for the first statement. The precondition for the first assignment statement can now be computed:

```
3 * x + 1 < 7
x < 2
```

So, `{x < 2}` is the precondition of both the first statement and the two-statement sequence.

3.5.3.5 Selection

We next consider the inference rule for selection statements, the general form of which is

if B **then** S1 **else** S2

We consider only selections that include **else** clauses. The inference rule is

$$\frac{\{B \text{ and } P\}\ S1\ \{Q\},\ \{(\text{not } B) \text{ and } P\}\ S2\ \{Q\}}{\{P\}\ \textbf{if}\ B\ \textbf{then}\ S1\ \textbf{else}\ S2\ \{Q\}}$$

This rule specifies that selection statements must be proven both when the Boolean control expression is true and when it is false. The first logical statement above the line represents the **then** clause; the second represents the **else** clause. According to the inference rule, we need a precondition P that can be used in the precondition of both the **then** and **else** clauses.

Consider the following example of the computation of the precondition using the selection inference rule. The example selection statement is

```
if x > 0 then
  y = y - 1
else
  y = y + 1
```

Suppose the postcondition, Q, for this selection statement is `{y > 0}`. We can use the axiom for assignment on the **then** clause

```
y = y - 1 {y > 0}
```

This produces `{y - 1 > 0}` or `{y > 1}`. It can be used as the P part of the precondition for the **then** clause. Now we apply the same axiom to the **else** clause

```
y = y + 1 {y > 0}
```

which produces the precondition {y + 1 > 0} or {y > -1}. Because {y > 1} => {y > -1}, the rule of consequence allows us to use {y > 1} for the precondition of the whole selection statement.

3.5.3.6 Logical Pretest Loops

Another essential construct of imperative programming languages is the logical pretest, or **while** loop. Computing the weakest precondition for a **while** loop is inherently more difficult than for a sequence, because the number of iterations cannot always be predetermined. In a case where the number of iterations is known, the loop can be unrolled and treated as a sequence.

The problem of computing the weakest precondition for loops is similar to the problem of proving a theorem about all positive integers. In the latter case, induction is normally used, and the same inductive method can be used for some loops. The principal step in induction is finding an inductive hypothesis. The corresponding step in the axiomatic semantics of a **while** loop is finding an assertion called a **loop invariant**, which is crucial to finding the weakest precondition.

The inference rule for computing the precondition for a **while** loop is as follows:

$$\frac{\{I \text{ and } B\}\ S\ \{I\}}{\{I\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{end}\ \{I \text{ and (not } B)\}}$$

In this rule, I is the loop invariant. This seems simple, but it is not. The complexity lies in finding an appropriate loop invariant.

The axiomatic description of a **while** loop is written as

{P} **while** B **do** S **end** {Q}

The loop invariant must satisfy a number of requirements to be useful. First, the weakest precondition for the **while** loop must guarantee the truth of the loop invariant. In turn, the loop invariant must guarantee the truth of the postcondition upon loop termination. These constraints move us from the inference rule to the axiomatic description. During execution of the loop, the truth of the loop invariant must be unaffected by the evaluation of the loop-controlling Boolean expression and the loop body statements. Hence, the name *invariant*.

Another complicating factor for **while** loops is the question of loop termination. A loop that does not terminate cannot be correct, and in fact computes nothing. If Q is the postcondition that holds immediately after loop exit, then a precondition P for the loop is one that guarantees Q at loop exit and also guarantees that the loop terminates.

The complete axiomatic description of a **while** construct requires all of the following to be true, in which I is the loop invariant:

P => I
{I and B} S { I }
(I and (not B)) => Q
the loop terminates

If a loop computes a sequence of numeric values, it may be possible to find a loop invariant using an approach that is used for determining the inductive hypothesis when mathematical induction is used to prove a statement about a mathematical sequence. The relationship between the number of iterations and the precondition for the loop body is computed for a few cases, with the hope that a pattern emerges that will apply to the general case. It is helpful to treat the process of producing a weakest precondition as a function, wp. In general

wp(statement, postcondition) = precondition

A wp function is often called a **predicate transformer**, because it takes a predicate, or assertion, as a parameter and returns another predicate.

To find I, the loop postcondition Q is used to compute preconditions for several different numbers of iterations of the loop body, starting with none. If the loop body contains a single assignment statement, the axiom for assignment statements can be used to compute these cases. Consider the example loop:

```
while y <> x do y = y + 1 end {y = x}
```

Remember that the equal sign is being used for two different purposes here. In assertions, it means mathematical equality; outside assertions, it means the assignment operator.

For zero iterations, the weakest precondition is, obviously,

```
{y = x}
```

For one iteration, it is

wp`(y = y + 1, {y = x}) = {y + 1 = x}`, or `{y = x - 1}`

For two iterations, it is

wp`(y = y + 1, {y = x - 1})={y + 1 = x - 1}`, or `{y = x - 2}`

For three iterations, it is

wp`(y = y + 1, {y = x - 2})={y + 1 = x - 2}`, or `{y = x - 3}`

It is now obvious that `{y < x}` will suffice for cases of one or more iterations. Combining this with `{y = x}` for the zero iterations case, we get `{y <= x}`, which can be used for the loop invariant. A precondition for the **`while`** statement can be determined from the loop invariant. In fact, I can be used as the precondition, P.

We must ensure that our choice satisfies the four criteria for I for our example loop. First, because P = I, P => I. The second requirement is that the following must be true:

{I and B} S {I}

In our example, we have

```
{y <= x and y <> x} y = y + 1 {y <= x}
```

Applying the assignment axiom to

```
y = y + 1 {y <= x}
```

we get {y + 1 <= x}, which is equivalent to {y < x}, which is implied by {y <= x and y <> x}. So, the earlier statement is proven.

Next, we must have

{I and (not B)} => Q

In our example, we have

```
{(y <= x) and not (y <> x)} => {y = x}
{(y <= x) and (y = x)} => {y = x}
{y = x} => {y = x}
```

So, this is obviously true. Next, loop termination must be considered. In this example, the question is whether the loop

```
{y <= x} while y <> x do y = y + 1 end {y = x}
```

terminates. Recalling that x and y are assumed to be integer variables, it is easy to see that this loop does terminate. The precondition guarantees that y initially is not larger than x. The loop body increments y with each iteration, until y is equal to x. No matter how much smaller y is than x initially, it will eventually become equal to x. So the loop will terminate. Because our choice of I satisfies all four criteria, it is a satisfactory loop invariant and loop precondition.

The previous process used to compute the invariant for a loop does not always produce an assertion that is the weakest precondition (although it does in the example).

As another example of finding a loop invariant using the approach used in mathematical induction, consider the following loop statement:

```
while  s > 1  do  s = s / 2  end  {s = 1}
```

As before, we use the assignment axiom to try to find a loop invariant and a precondition for the loop. For zero iterations, the weakest precondition is {s = 1}. For one iteration, it is

wp(s = s / 2, {s = 1}) = {s / 2 = 1}, or {s = 2}

For two iterations, it is

wp(s = s / 2, {s = 2}) = {s / 2 = 2}, or {s = 4}

For three iterations, it is

wp(s = s / 2, {s = 4}) = {s / 2 = 4}, or {s = 8}

From these cases, we can see clearly that the invariant is

{s is a nonnegative power of 2}

Once again, the computed I can serve as P, and I passes the four requirements. Unlike our earlier example of finding a loop precondition, this one clearly is not a weakest precondition. Consider using the precondition {s > 1}. The logical statement

```
{s > 1} while  s > 1 do  s = s / 2 end {s = 1}
```

can easily be proven, and this precondition is significantly broader than the one computed earlier. The loop and precondition are satisfied for any positive value for s, not just powers of 2, as the process indicates. Because of the rule of consequence, using a precondition that is stronger than the weakest precondition does not invalidate a proof.

Finding loop invariants is not always easy. It is helpful to understand the nature of these invariants. First, a loop invariant is a weakened version of the loop postcondition and also a precondition for the loop. So, I must be weak enough to be satisfied prior to the beginning of loop execution, but when combined with the loop exit condition, it must be strong enough to force the truth of the postcondition.

Because of the difficulty of proving loop termination, that requirement is often ignored. If loop termination can be shown, the axiomatic description of the loop is called **total correctness**. If the other conditions can be met but termination is not guaranteed, it is called **partial correctness**.

In more complex loops, finding a suitable loop invariant, even for partial correctness, requires a good deal of ingenuity. Because computing the precondition for a **while** loop depends on finding a loop invariant, proving the correctness of programs with **while** loops using axiomatic semantics can be difficult.

3.5.3.7 Program Proofs

This section provides validations for two simple programs. The first example of a correctness proof is for a very short program, consisting of a sequence of three assignment statements that interchange the values of two variables.

```
{x = A AND y = B}
t = x;
x = y;
y = t;
{x = B AND y = A}
```

Because the program consists entirely of assignment statements in a sequence, the assignment axiom and the inference rule for sequences can be used to prove its correctness. The first step is to use the assignment axiom on the last statement and the postcondition for the whole program. This yields the precondition

```
{x = B AND t = A}
```

Next, we use this new precondition as a postcondition on the middle statement and compute its precondition, which is

```
{y = B AND t = A}
```

Next, we use this new assertion as the postcondition on the first statement and apply the assignment axiom, which yields

```
{y = B AND x = A}
```

which is the same as the precondition on the program, except for the order of operands on the `AND` operator. Because `AND` is a symmetric operator, our proof is complete.

The following example is a proof of correctness of a pseudocode program that computes the factorial function.

```
{n >= 0}
count = n;
fact = 1;
while count <> 0 do
    fact = fact * count;
    count = count - 1;
end
{fact = n!}
```

The method described earlier for finding the loop invariant does not work for the loop in this example. Some ingenuity is required here, which can be aided by a brief study of the code. The loop computes the factorial function in order of the last multiplication first; that is, `(n - 1) * n` is done first, assuming `n` is greater than `1`. So, part of the invariant can be the following:

```
fact = (count + 1) * (count + 2) * . . . * (n - 1) * n
```

But we also must ensure that `count` is always nonnegative, which we can do by adding that to the assertion above, to get the following:

```
I = (fact = (count + 1) * . . . * n) AND (count >= 0)
```

Next, we must confirm that this I meets the requirements for invariants. Once again we let I also be used for P, so P clearly implies I. The next question is

{I and B} S {I}

I and B is the following:

```
((fact = (count + 1) * . . . * n) AND (count >= 0)) AND
    (count <> 0)
```

This reduces to

```
(fact = (count + 1) * . . . * n) AND (count > 0)
```

In our case, we must compute the precondition of the body of the loop, using the invariant for the postcondition. For

{P} `count = count - 1` {I}

we compute P to be

```
{(fact = count * (count + 1) * . . . * n) AND
    (count >= 1)}
```

Using this as the postcondition for the first assignment in the loop body,

```
{P} fact = fact * count {(fact = count * (count + 1)
                          * . . . * n) AND (count >= 1)}
```

In this case, P is

```
{(fact = (count + 1) * . . . * n) AND (count >= 1)}
```

It is clear that I and B implies this P, so by the rule of consequence,

{I `AND` B} S {`I`}

is true. Finally, the last test of I is

I `AND` (`NOT` B) => Q

For our example, this is

```
((fact = (count + 1) * . . . * n) AND (count >= 0)) AND
(count = 0)) => fact = n!
```

This is clearly true, for when `count = 0`, the first part is precisely the definition of factorial. So, our choice of I meets the requirements for a loop invariant. Now we can use our P (which is the same as I) from the **`while`** as the postcondition on the second assignment of the program

```
{P} fact = 1 {(fact = (count + 1) * . . . * n) AND
    (count >= 0)}
```

which yields for P

```
(1 = (count + 1) * . . . * n) AND (count >= 0))
```

Using this as the postcondition for the first assignment in the code

```
{P} count = n {(1 = (count + 1) * . . . * n) AND
    (count >= 0))}
```

produces for P

```
{(n + 1) * . . . * n = 1) AND (n >= 0)}
```

The left operand of the `AND` operator is true (because `1 = 1`) and the right operand is exactly the precondition of the whole code segment, {`n >= 0`}. Therefore, the program has been proven to be correct.

3.5.3.8 Evaluation

As stated previously, to define the semantics of a complete programming language using the axiomatic method, there must be an axiom or an inference rule for each statement type in the language. Defining axioms or inference rules for some of the statements of programming languages has proven to be a difficult task. An obvious solution to this problem is to design the language with the axiomatic method in mind, so that only statements for which axioms or inference rules can be written are included. Unfortunately, such a language would necessarily leave out some useful and powerful statements.

Axiomatic semantics is a powerful tool for research into program correctness proofs, and it provides an excellent framework in which to reason about programs, both during their construction and later. Its usefulness in describing the meaning of programming languages to language users and compiler writers is, however, highly limited.

SUMMARY

Backus-Naur Form and context-free grammars are equivalent metalanguages that are well suited for the task of describing the syntax of programming languages. Not only are they concise descriptive tools, but also the parse trees that can be associated with their generative actions give graphical evidence of the underlying syntactic structures. Furthermore, they are naturally related to recognition devices for the languages they generate, which leads to the relatively easy construction of syntax analyzers for compilers for these languages.

An attribute grammar is a descriptive formalism that can describe both the syntax and static semantics of a language. Attribute grammars are extensions to context-free grammars. An attribute grammar consists of a grammar, a set of attributes, a set of attribute computation functions, and a set of predicates that describe static semantics rules.

This chapter provides brief introductions to three methods of semantic description: operational, denotational, and axiomatic. Operational semantics is a method of describing the meaning of language constructs in terms of their effects on an ideal machine. In denotational semantics, mathematical objects are used to represent the meanings of language constructs. Language entities are converted to these mathematical objects with recursive functions. Axiomatic semantics, which is based on formal logic, was devised as a tool for proving the correctness of programs.

BIBLIOGRAPHIC NOTES

Syntax description using context-free grammars and BNF are thoroughly discussed in Cleaveland and Uzgalis (1976).

Research in axiomatic semantics was begun by Floyd (1967) and further developed by Hoare (1969). The semantics of a large part of Pascal was described by Hoare and Wirth (1973) using this method. The parts they did not complete involved functional side effects and goto statements. These were found to be the most difficult to describe.

The technique of using preconditions and postconditions during the development of programs is described (and advocated) by Dijkstra (1976) and also discussed in detail in Gries (1981).

Good introductions to denotational semantics can be found in Gordon (1979) and Stoy (1977). Introductions to all of the semantics description methods discussed in this chapter can be found in Marcotty et al. (1976). Another good reference for much of the chapter material is Pagan (1981). The form of the denotational semantic functions in this chapter is similar to that found in Meyer (1990).

REVIEW QUESTIONS

1. Define *syntax* and *semantics*.
2. Who are language descriptions for?
3. Describe the operation of a general language generator.
4. Describe the operation of a general language recognizer.
5. What is the difference between a sentence and a sentential form?
6. Define a left-recursive grammar rule.
7. What three extensions are common to most EBNFs?
8. Distinguish between static and dynamic semantics.
9. What purpose do predicates serve in an attribute grammar?
10. What is the difference between a synthesized and an inherited attribute?
11. How is the order of evaluation of attributes determined for the trees of a given attribute grammar?
12. What is the primary use of attribute grammars?
13. Explain the primary uses of a methodology and notation for describing the semantics of programming languages.
14. Why can machine languages not be used to define statements in operational semantics?
15. Describe the two levels of uses of operational semantics.
16. In denotational semantics, what are the syntactic and semantic domains?
17. What is stored in the state of a program for denotational semantics?
18. Which semantics approach is most widely known?

19. What two things must be defined for each language entity in order to construct a denotational description of the language?
20. Which part of an inference rule is the antecedent?
21. What is a predicate transformer function?
22. What does partial correctness mean for a loop construct?
23. On what branch of mathematics is axiomatic semantics based?
24. On what branch of mathematics is denotational semantics based?
25. What is the problem with using a software pure interpreter for operational semantics?
26. Explain what the preconditions and postconditions of a given statement mean in axiomatic semantics.
27. Describe the approach of using axiomatic semantics to prove the correctness of a given program.
28. Describe the basic concept of denotational semantics.
29. In what fundamental way do operational semantics and denotational semantics differ?

PROBLEM SET

1. The two mathematical models of language description are generation and recognition. Describe how each can define the syntax of a programming language.
2. Write EBNF descriptions for the following:
   a. A Java class definition header statement
   b. A Java method call statement
   c. A C **`switch`** statement
   d. A C **`union`** definition
   e. C **`float`** literals
3. Rewrite the BNF of Example 3.4 to give + precedence over * and force + to be right associative.
4. Rewrite the BNF of Example 3.4 to add the ++ and -- unary operators of Java.
5. Write a BNF description of the Boolean expressions of Java, including the three operators `&&`, `||`, and `!` and the relational expressions.
6. Using the grammar in Example 3.2, show a parse tree and a leftmost derivation for each of the following statements:
   a. `A = A * (B + (C * A))`
   b. `B = C * (A * C + B)`
   c. `A = A * (B + (C))`

7. Using the grammar in Example 3.4, show a parse tree and a leftmost derivation for each of the following statements:

   a. `A = (A + B) * C`

   b. `A = B + C + A`

   c. `A = A * (B + C)`

   d. `A = B * (C * (A + B))`

8. Prove that the following grammar is ambiguous:

   <S> → <A>

   <A> → <A> + <A> | <id>

   <id> → a | b | c

9. Modify the grammar of Example 3.4 to add a unary minus operator that has higher precedence than either + or *.

10. Describe, in English, the language defined by the following grammar:

    <S> → <A> <B> <C>

    <A> → a <A> | a

    <B> → b <B> | b

    <C> → c <C> | c

11. Consider the following grammar:

    <S> → <A> a <B> b

    <A> → <A> b | b

    <B> → a <B> | a

    Which of the following sentences are in the language generated by this grammar?

    a. baab

    b. bbbab

    c. bbaaaaa`S`

    d. bbaab

12. Consider the following grammar:

    <S> → a <S> c <B> | <A> | b

    <A> → c <A> | c

    <B> → d | <A>

    Which of the following sentences are in the language generated by this grammar?

    a. abcd

    b. acccbd

    c. acccbcc

    d. acd

    e. accc

13. Write a grammar for the language consisting of strings that have *n* copies of the letter a followed by the same number of copies of the letter b, where $n > 0$. For example, the strings ab, aaaabbbb, and aaaaaaaabbbbbbbb are in the language but a, abb, ba, and aaabb are not.
14. Draw parse trees for the sentences aabb and aaaabbbb, as derived from the grammar of Problem 13.
15. Convert the BNF of Example 3.1 to EBNF.
16. Convert the BNF of Example 3.3 to EBNF.
17. Convert the following EBNF to BNF:

    $S \rightarrow A\{bA\}$

    $A \rightarrow a[b]A$
18. What is the difference between an intrinsic attribute and a nonintrinsic synthesized attribute?
19. Write an attribute grammar whose BNF basis is that of Example 3.6 in Section 3.4.5 but whose language rules are as follows: Data types cannot be mixed in expressions, but assignment statements need not have the same types on both sides of the assignment operator.
20. Write an attribute grammar whose base BNF is that of Example 3.2 and whose type rules are the same as for the assignment statement example of Section 3.4.5.
21. Using the virtual machine instructions given in Section 3.5.1.1, give an operational semantic definition of the following:

    a. Java **`do-while`**

    b. Ada **`for`**

    c. C++ **`if-then-else`**

    d. C **`for`**

    e. C **`switch`**
22. Write a denotational semantics mapping function for the following statements:

    a. Ada **`for`**

    b. Java **`do-while`**

    c. Java Boolean expressions

    d. Java **`for`**

    e. C **`switch`**
23. Compute the weakest precondition for each of the following assignment statements and postconditions:

    a. `a = 2 * (b - 1) - 1 {a > 0}`

    b. `b = (c + 10) / 3 {b > 6}`

    c. `a = a + 2 * b - 1 {a > 1}`

    d. `x = 2 * y + x - 1 {x > 11}`

24. Compute the weakest precondition for each of the following sequences of assignment statements and their postconditions:

a.
```
a = 2 * b + 1;
b = a - 3
{b < 0}
```

b.
```
a = 3 * (2 * b + a);
b = 2 * a - 1
{b > 5}
```

25. Compute the weakest precondition for each of the following selection constructs and their postconditions:

a.
```
if (a == b)
   b = 2 * a + 1
else
   b = 2 * a;
{b > 1}
```

b.
```
if (x < y)
    x = x + 1
else
    x = 3 * x
{x < 0}
```

c.
```
if (x > y)
    y = 2 * x + 1
else
    y = 3 * x - 1;
{y > 3}
```

26. Explain the four criteria for proving the correctness of a logical pretest loop construct of the form **while** B **do** S **end**.

27. Prove that (n + 1)* . . . * n = 1.

28. Prove the following program is correct:

```
{n > 0}
count = n;
sum = 0;
while  count <> 0  do
  sum = sum + count;
  count = count - 1;
end
{sum = 1 + 2 + . . . + n}
```